

```
it('22.3 All core whiteboard types export without syntax or type errors', () => {
  trackFeature('feature_22_strict_typescript_build');
  expect(typesModule.DEFAULT_STYLE).toBeTruthy();
  expect(typesModule.PEN_TYPE_PRESETS).toBeTruthy();
  expect(typesModule.STICKY_COLORS).toBeTruthy();
});

it('22.4 Geometry library exports all required math functions', () => {
  trackFeature('feature_22_strict_typescript_build');
  const required = ['worldToScreen', 'screenToWorld', 'hitTestElement', 'getNodeAnchorPorts', 'getOptimalConnectionAnchors', 'getCurvedBezierPath', 'getOrthogonalPath', 'getStraightPath'];
  for (const fn of required) {
    expect(typeof geometryModule[fn]).toBe('function');
  }
});

it('22.5 Links and Wikilinks intelligence exports parser and graph builder', () => {
  trackFeature('feature_22_strict_typescript_build');
  expect(typeof linksModule.parseText).toBe('function');
  expect(typeof linksModule.segmentText).toBe('function');
  expect(typeof linksModule.getBacklinks).toBe('function');
  expect(typeof linksModule.buildGraph).toBe('function');
});

// -----
// Feature 23: S Pen Hardware Inking, Coalescence, Barrel Button & Palm Isolation
// -----
describe('Feature 23: S Pen Hardware Inking, Coalescence, Barrel Button & Palm Isolation', () => {
  it('23.1 Coalesced events stream ingestion extracts sub-frame hardware points with monotonic order', () => {
    {
      trackFeature('feature_23_spen_hardware_inking');
      const rawStream = [
        { clientX: 50, clientY: 50, pressure: 0.25, timeStamp: 100 },
        { clientX: 55, clientY: 52, pressure: 0.35, timeStamp: 104 },
        { clientX: 62, clientY: 56, pressure: 0.45, timeStamp: 108 },
        { clientX: 70, clientY: 60, pressure: 0.55, timeStamp: 112 }
      ];
      for (let i = 1; i < rawStream.length; i++) {
        expect(rawStream[i].timeStamp > rawStream[i - 1].timeStamp).toBe(true);
      }
      const points = rawStream.map(e => [e.clientX, e.clientY, strokeModule.calibratePressure(e.pressure, 65, 'standard')]);
      expect(points.length).toBe(4);
      expect(points[0][0]).toBe(50);
      expect(points[3][0]).toBe(70);
      expect(points[3][2] > points[0][2]).toBe(true);
    }
  });

  it('23.2 Drawing pointer ID isolation rejects non-matching pointer events', () => {
    trackFeature('feature_23_spen_hardware_inking');
    const drawingPointerId = 101;
    const strokePoints = [];

    const handleMove = (ev) => {
      if (ev.pointerId !== drawingPointerId) return;
      strokePoints.push([ev.clientX, ev.clientY, ev.pressure]);
    };

    handleMove({ pointerId: 101, clientX: 10, clientY: 10, pressure: 0.5 });
    handleMove({ pointerId: 999, clientX: 500, clientY: 400, pressure: 0.1 }); // Palm touch rejected
    handleMove({ pointerId: 101, clientX: 20, clientY: 20, pressure: 0.5 });

    expect(strokePoints.length).toBe(2);
    expect(strokePoints[0][0]).toBe(10);
    expect(strokePoints[1][0]).toBe(20);
  });

  it('23.3 Stylus barrel button touchdown engages eraser without stroke artifact', () => {
    trackFeature('feature_23_spen_hardware_inking');
    const NIB = 1;
    const BARREL = inputModule.STYLUS_BARREL_BUTTON;
    const penDownWithButton = { pointerType: 'pen', buttons: NIB | BARREL };

    expect(inputModule.isStylusEraseRequest(penDownWithButton)).toBe(true);
    const action = inputModule.stylusEraseTransition(false, penDownWithButton);
    expect(action).toBe('engage');
  });
});
```

```
it('23.4 Draw-and-hold auto-straightening detects straight line with chord ratio >= 0.88', () => {
  trackFeature('feature_23_spen_hardware_inking');
  const straightLine = [
    [10, 50, 0.5],
    [30, 50, 0.5],
    [60, 51, 0.5],
    [90, 49, 0.5],
    [120, 50, 0.5],
    [150, 50, 0.5]
  ];
  expect(strokeModule.isStrokeStraight(straightLine, 0.88)).toBe(true);
  const interpolated = strokeModule.straightenStrokePoints(straightLine, 8);
  expect(interpolated.length).toBe(9);
  expect(interpolated[0][0]).toBe(10);
  expect(interpolated[8][0]).toBe(150);
});

it('23.5 All 8 pen engine geometries render valid SVG paths matching physical specs', () => {
  trackFeature('feature_23_spen_hardware_inking');
  const engines = ['ballpoint', 'gel', 'fineliner', 'fountain', 'brush', 'marker', 'highlighter', 'pencil'];
  for (const eng of engines) {
    expect(typesModule.PEN_TYPE_PRESETS[eng]).toBeDefined();
    const strokeEl = {
      id: `test-${eng}`,
      type: 'stroke',
      x: 0,
      y: 0,
      rotation: 0,
      opacity: 1,
      color: '#1e293b',
      strokeWidth: 4,
      strokeStyle: eng,
      points: [[0, 0, 0.5], [20, 20, 0.6], [40, 50, 0.7]],
      createdAt: 1,
      updatedAt: 1
    };
    const pathData = strokeModule.buildStrokePath(strokeEl, false);
    expect(pathData.startsWith('M')).toBe(true);
    expect(pathData.endsWith('Z')).toBe(true);
  }
});

it('23.6 Palm contact >= 45px or within 800ms suppression window is rejected', () => {
  trackFeature('feature_23_spen_hardware_inking');
  expect(palmModule.contactLooksLikePalm({ width: 50, height: 30 })).toBe(true);
  expect(palmModule.contactLooksLikePalm({ width: 20, height: 20 })).toBe(false);

  const now = 20000;
  const verdictRecent = palmModule.classifyTouch(
    { width: 20, height: 20 },
    { penDown: false, lastPenAt: now - 300, now }
  );
  expect(verdictRecent).toBe('pen-recent');

  const verdictAllowed = palmModule.classifyTouch(
    { width: 20, height: 20 },
    { penDown: false, lastPenAt: now - 900, now }
  );
  expect(verdictAllowed).toBe('allow');
});

it('23.7 Velocity thinning thins out rapid strokes while preserving deliberate slow writing', () => {
  trackFeature('feature_23_spen_hardware_inking');
  const slowPoints = [[0, 0, 0.5], [2, 0, 0.5], [4, 0, 0.5]];
  const fastPoints = [[0, 0, 0.5], [30, 0, 0.5], [70, 0, 0.5]];

  const slowDyn = strokeModule.applyPenDynamics(slowPoints, 'ballpoint', 65, 'standard');
  const fastDyn = strokeModule.applyPenDynamics(fastPoints, 'ballpoint', 65, 'standard');

  expect(fastDyn[2][2] < slowDyn[2][2]).toBe(true);
});

it('23.8 Anti-hook pruning removes terminal lift-off hooks without affecting checkmarks', () => {
  trackFeature('feature_23_spen_hardware_inking');
  const hookPoints = [
    [0, 0, 0.5], [30, 0, 0.5], [60, 0, 0.5], [90, 0, 0.5], [120, 0, 0.5], [105, 5, 0.15]
  ]
}
```

```
    ];
    const pruned = strokeModule.pruneTerminalHooks(hookPoints);
    expect(pruned.length).toBe(5);

    const checkmark = [
      [0, 50, 0.5], [20, 70, 0.5], [40, 90, 0.6], [60, 60, 0.5], [80, 30, 0.4]
    ];
    const preserved = strokeModule.pruneTerminalHooks(checkmark);
    expect(preserved.length).toBe(5);
  });
});
});

=====
// === END FILE: tests/e2e/tier1-features.test.js ===
=====

=====
// === START FILE: tests/e2e/tier2-boundaries.test.js ===
// File 149 of 194 | Lines: 1158 | Size: 57539 bytes
=====

/**
 * Tier 2: Boundary & Corner Cases Test Suite (ProNotes E2E)
 * =====
 * Covers all 22 features with boundary, edge, stress, and corner case testing
 * (>=5 test cases per feature, >=110 test assertions total).
 */

const fs = require('fs');
const path = require('path');
const {
  describe,
  it,
  expect,
  beforeEach,
  trackFeature,
  setupMockDom,
  loadTsModule
} = require('./test-helpers');

describe('Tier 2: Boundary & Corner Cases (22 Features)', () => {
  let strokeModule;
  let geometryModule;
  let linksModule;
  let persistenceModule;
  let storeModule;
  let typesModule;
  let palmModule;

  beforeEach(() => {
    setupMockDom();
    strokeModule = loadTsModule('src/lib/whiteboard/stroke.ts');
    geometryModule = loadTsModule('src/lib/whiteboard/geometry.ts');
    linksModule = loadTsModule('src/lib/whiteboard/links.ts');
    persistenceModule = loadTsModule('src/lib/whiteboard/persistence.ts');
    storeModule = loadTsModule('src/store/whiteboard.ts');
    typesModule = loadTsModule('src/lib/whiteboard/types.ts');
    palmModule = loadTsModule('src/lib/whiteboard/palm.ts');
  });

  // -----
  // Feature 1: Freehand Strokes Boundary & Stress Cases
  // -----
  describe('Feature 1: Freehand Strokes Boundary & Stress', () => {
    it('1.1 Empty points array returns empty path string "" without throwing', () => {
      trackFeature('feature_1_freehand_stroke_rendering');
      const emptyStroke = { id: 's-empty', type: 'stroke', x: 0, y: 0, rotation: 0, opacity: 1, color: '#000',
        strokeWidth: 4, strokeStyle: 'pen', points: [], createdAt: 1, updatedAt: 1 };
      const pathData = strokeModule.buildStrokePath(emptyStroke);
      expect(pathData).toBe('');
    });

    it('1.2 2 identical points render solid circle instead of zero-area polygon', () => {
      trackFeature('feature_1_freehand_stroke_rendering');
      const identicalPointsStroke = { id: 's-id', type: 'stroke', x: 100, y: 100, rotation: 0, opacity: 1, col
        or: '#000', strokeWidth: 4, strokeStyle: 'pen', points: [[0, 0, 0.5], [0, 0, 0.5]], createdAt: 1, up
```

```
      datedAt: 1 };
      const pathData = strokeModule.buildStrokePath(identicalPointsStroke);
      expect(pathData).toContain('a 2.00,2.00');
      expect(pathData).toContain('Z');
    });

    it('1.3 5,000 points dense stroke executes within 1000ms without stack overflow', () => {
      trackFeature('feature_1_freehand_stroke_rendering');
      const densePoints = [];
      for (let i = 0; i < 5000; i++) {
        densePoints.push([i * 0.1, Math.sin(i * 0.05) * 50, 0.5]);
      }
      const denseStroke = { id: 's-dense', type: 'stroke', x: 0, y: 0, rotation: 0, opacity: 1, color: '#000',
        strokeWidth: 3, strokeStyle: 'pen', points: densePoints, createdAt: 1, updatedAt: 1 };
      const start = Date.now();
      const pathData = strokeModule.buildStrokePath(denseStroke);
      const elapsed = Date.now() - start;
      expect(pathData).toBeTruthy();
      expect(elapsed).toBeLessThan(1500);
    });

    it('1.4 Extreme coordinate values (+1e6, -1e6) do not produce NaN or Infinity', () => {
      trackFeature('feature_1_freehand_stroke_rendering');
      const extremeStroke = { id: 's-ext', type: 'stroke', x: 1e6, y: -1e6, rotation: 0, opacity: 1, color: '#000',
        strokeWidth: 5, strokeStyle: 'pen', points: [[0, 0, 0.5], [100, 100, 0.5]], createdAt: 1, updatedAt: 1 };
      const pathData = strokeModule.buildStrokePath(extremeStroke);
      expect(pathData).not.toContain('NaN');
      expect(pathData).not.toContain('Infinity');
    });

    it('1.5 Extreme pressure values (0.0 vs 1.0) maintain valid outline path', () => {
      trackFeature('feature_1_freehand_stroke_rendering');
      const pressureStroke = { id: 's-press', type: 'stroke', x: 0, y: 0, rotation: 0, opacity: 1, color: '#000',
        strokeWidth: 6, strokeStyle: 'brush', points: [[0, 0, 0.0], [50, 50, 1.0], [100, 100, 0.0]], createdAt: 1, updatedAt: 1 };
      const pathData = strokeModule.buildStrokePath(pressureStroke);
      expect(pathData.startsWith('M')).toBe(true);
      expect(pathData.endsWith('Z')).toBe(true);
    });
  });

  // -----
  // Feature 2: Single-Tap Dots Boundary Cases
  // -----
  describe('Feature 2: Single-Tap Dots Boundary Cases', () => {
    it('2.1 strokeWidth 0 falls back to minimum 1px stroke width', () => {
      trackFeature('feature_2_singletap_dot_inking');
      const dot = { id: 'dot-0w', type: 'stroke', x: 0, y: 0, rotation: 0, opacity: 1, color: '#000', strokeWidth: 0,
        strokeStyle: 'pen', points: [[0, 0, 0.5]], createdAt: 1, updatedAt: 1 };
      const pathData = strokeModule.buildStrokePath(dot);
      expect(pathData).toContain('a 2.00,2.00');
    });

    it('2.2 Floating point micro-jitter (<0.01px) renders exact circle', () => {
      trackFeature('feature_2_singletap_dot_inking');
      const jitterDot = { id: 'dot-jit', type: 'stroke', x: 50, y: 50, rotation: 0, opacity: 1, color: '#000',
        strokeWidth: 4, strokeStyle: 'pen', points: [[0, 0, 0.5], [0.005, 0.005, 0.5]], createdAt: 1, updatedAt: 1 };
      const pathData = strokeModule.buildStrokePath(jitterDot);
      expect(pathData).toContain('a 2.00,2.00');
    });

    it('2.3 Negative strokeWidth input is clamped safely to minimum', () => {
      trackFeature('feature_2_singletap_dot_inking');
      const negDot = { id: 'dot-neg', type: 'stroke', x: 0, y: 0, rotation: 0, opacity: 1, color: '#000', strokeWidth: -10,
        strokeStyle: 'pen', points: [[0, 0, 0.5]], createdAt: 1, updatedAt: 1 };
      const pathData = strokeModule.buildStrokePath(negDot);
      expect(pathData).toContain('a 2.00,2.00');
    });

    it('2.4 Dot with zero pressure renders solid circle dot with standard radius', () => {
      trackFeature('feature_2_singletap_dot_inking');
      const zeroPressDot = { id: 'dot-zp', type: 'stroke', x: 10, y: 10, rotation: 0, opacity: 1, color: '#000',
        strokeWidth: 5, strokeStyle: 'pen', points: [[0, 0, 0]], createdAt: 1, updatedAt: 1 };
      const pathData = strokeModule.buildStrokePath(zeroPressDot);
      expect(pathData).toContain('a 2.50,2.50');
    });
  });
```

```

it('2.5 Rapid consecutive single taps at identical coordinate produce unique elements', () => {
  trackFeature('feature_2_singletap_dot_inking');
  const { useWhiteboard } = storeModule;
  useWhiteboard.getState().init();
  const dot1 = { id: 'tap-1', type: 'stroke', x: 100, y: 100, strokeWidth: 4, points: [[0, 0, 0.5]], rotation: 0, opacity: 1, color: '#000', strokeStyle: 'pen', createdAt: 1, updatedAt: 1 };
  const dot2 = { id: 'tap-2', type: 'stroke', x: 100, y: 100, strokeWidth: 4, points: [[0, 0, 0.5]], rotation: 0, opacity: 1, color: '#000', strokeStyle: 'pen', createdAt: 2, updatedAt: 2 };
  useWhiteboard.getState().addElements([dot1, dot2], true);
  const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
  expect(board.elements.filter((e) => e.x === 100 && e.y === 100).length).toBe(2);
});
});

// -----
// Feature 3: Coordinate Math Boundary Cases
// -----
describe('Feature 3: Coordinate Math Boundary Cases', () => {
  it('3.1 Zoom below MIN_ZOOM (0.0001) is clamped strictly to 0.05', () => {
    trackFeature('feature_3_coordinate_placement_and_zoom');
    const clamped = geometryModule.clamp(0.0001, typesModule.MIN_ZOOM, typesModule.MAX_ZOOM);
    expect(clamped).toBe(0.05);
  });

  it('3.2 Zoom above MAX_ZOOM (99.0) is clamped strictly to 8.0', () => {
    trackFeature('feature_3_coordinate_placement_and_zoom');
    const clamped = geometryModule.clamp(99.0, typesModule.MIN_ZOOM, typesModule.MAX_ZOOM);
    expect(clamped).toBe(8.0);
  });

  it('3.3 Viewport dimension 0x0 does not produce NaN in coordinate transforms', () => {
    trackFeature('feature_3_coordinate_placement_and_zoom');
    const camera = { x: 0, y: 0, zoom: 1 };
    const zeroViewport = { width: 0, height: 0 };
    const world = geometryModule.screenToWorld(0, 0, camera, zeroViewport);
    expect(Number.isNaN(world.x)).toBe(false);
    expect(Number.isNaN(world.y)).toBe(false);
    expect(world.x).toBe(0);
    expect(world.y).toBe(0);
  });

  it('3.4 Extreme camera coordinates (+1e8, -1e8) maintain delta arithmetic integrity', () => {
    trackFeature('feature_3_coordinate_placement_and_zoom');
    const camera = { x: 1e8, y: -1e8, zoom: 2 };
    const viewport = { width: 1000, height: 800 };
    const screen = geometryModule.worldToScreen(1e8 + 50, -1e8 + 50, camera, viewport);
    expect(screen.x).toBe(600);
    expect(screen.y).toBe(500);
  });

  it('3.5 rotatePoint with angle 0 returns exact point unmodified', () => {
    trackFeature('feature_3_coordinate_placement_and_zoom');
    const pt = geometryModule.rotatePoint(123.45, 678.9, 50, 50, 0);
    expect(pt.x).toBe(123.45);
    expect(pt.y).toBe(678.9);
  });
});

// -----
// Feature 4: Touchscreen Gestures Boundary Cases
// -----
describe('Feature 4: Touchscreen Gestures Boundary Cases', () => {
  it('4.1 0 active touches is handled gracefully without crash', () => {
    trackFeature('feature_4_touchscreen_modes_gestures');
    const touches = new Map();
    expect(touches.size).toBe(0);
  });

  it('4.2 Stylus Only mode with touch event type is rejected', () => {
    trackFeature('feature_4_touchscreen_modes_gestures');
    const touchSettings = { mode: 'stylus-only', palmRejection: true };
    const isTouchAllowed = (pointerType) => {
      if (touchSettings.mode === 'stylus-only' && pointerType === 'touch') return false;
      return true;
    };
    expect(isTouchAllowed('touch')).toBe(false);
  });
});

```

```

    expect(isTouchAllowed('pen')).toBe(true);
    expect(isTouchAllowed('mouse')).toBe(true);
  });

  it('4.3 3+ simultaneous touches cancels active draft inking', () => {
    trackFeature('feature_4_touchscreen_modes_gestures');
    const touchCount = 3;
    const shouldCancelDraft = touchCount >= 2;
    expect(shouldCancelDraft).toBe(true);
  });

  it('4.4 0-distance pinch (overlapping fingers) does not divide by zero', () => {
    trackFeature('feature_4_touchscreen_modes_gestures');
    const t1 = { x: 100, y: 100 };
    const t2 = { x: 100, y: 100 };
    const dist = Math.hypot(t1.x - t2.x, t1.y - t2.y);
    const safeDistance = Math.max(dist, 1);
    const scale = safeDistance / 100;
    expect(Number.isNaN(scale)).toBe(false);
    expect(scale).toBe(0.01);
  });

  it('4.5 Fast tap duration (<30ms) qualifies for tap undo/redo gestures', () => {
    trackFeature('feature_4_touchscreen_modes_gestures');
    const tapDuration = 25;
    const isQuickTap = tapDuration < 250;
    expect(isQuickTap).toBe(true);
  });
});

// -----
// Feature 5: Eraser Boundary Cases
// -----
describe('Feature 5: Eraser Boundary Cases', () => {
  it('5.1 Clicking empty canvas where no element exists returns false for all elements', () => {
    trackFeature('feature_5_object_drag_eraser');
    const elements = [
      { id: '1', type: 'rectangle', x: 0, y: 0, width: 50, height: 50, rotation: 0, opacity: 1, strokeColor:
        '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, c
        reatedAt: 1, updatedAt: 1 }
    ];
    const clicked = elements.find((el) => geometryModule.hitTestElement(el, 500, 500));
    expect(clicked).toBeUndefined();
  });

  it('5.2 Locked element returns false for hitTestElement protecting it from erasure', () => {
    trackFeature('feature_5_object_drag_eraser');
    const lockedEl = { id: 'lck', type: 'rectangle', x: 100, y: 100, width: 100, height: 100, locked: true,
      rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', da
      shStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    expect(geometryModule.hitTestElement(lockedEl, 150, 150)).toBe(false);
  });

  it('5.3 Deleting non-existent ID does not alter existing elements in store', () => {
    trackFeature('feature_5_object_drag_eraser');
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().init();
    const el = { id: 'safe-el', type: 'rectangle', x: 0, y: 0, width: 50, height: 50, rotation: 0, opacity:
      1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', stro
      keWidth: 2, createdAt: 1, updatedAt: 1 };
    useWhiteboard.getState().addElement(el, true);
    useWhiteboard.getState().deleteElements(['ghost-id-999'], true);
    const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardI
      d);
    expect(board.elements.some((e) => e.id === 'safe-el')).toBe(true);
  });

  it('5.4 Drag eraser path through empty region hit-tests 0 elements', () => {
    trackFeature('feature_5_object_drag_eraser');
    const el = { id: 'el-far', type: 'rectangle', x: 1000, y: 1000, width: 50, height: 50, rotation: 0, opac
      ity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid',
      strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    const pathPts = [{ x: 10, y: 10 }, { x: 20, y: 20 }, { x: 30, y: 30 }];
    const hit = pathPts.some((pt) => geometryModule.hitTestElement(el, pt.x, pt.y));
    expect(hit).toBe(false);
  });

  it('5.5 Batch erasing 50 elements executes in a single store transaction', () => {

```

```

    trackFeature('feature_5_object_drag_eraser');
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().init();
    const batch = [];
    const ids = [];
    for (let i = 0; i < 50; i++) {
      const id = `batch-${i}`;
      ids.push(id);
      batch.push({ id, type: 'rectangle', x: i * 10, y: 0, width: 8, height: 8, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 });
    }
    useWhiteboard.getState().addElements(batch, true);
    useWhiteboard.getState().deleteElements(ids, true);
    const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
    expect(board.elements.some((e) => ids.includes(e.id))).toBe(false);
  });
});

// -----
// Feature 6: 8 Geometric Shapes Boundary Cases
// -----
describe('Feature 6: 8 Geometric Shapes Boundary Cases', () => {
  it('6.1 Degenerate 0 width / 0 height rectangle bounds clamp to non-negative', () => {
    trackFeature('feature_6_geometric_shapes');
    const zeroRect = { id: 'zr', type: 'rectangle', x: 50, y: 50, width: 0, height: 0, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    const bounds = geometryModule.getElementBounds(zeroRect);
    expect(bounds.width).toBe(0);
    expect(bounds.height).toBe(0);
  });

  it('6.2 Inverted drag with negative width/height normalizes correctly', () => {
    trackFeature('feature_6_geometric_shapes');
    const startX = 200;
    const startY = 200;
    const currentX = 100;
    const currentY = 120;
    const normX = Math.min(startX, currentX);
    const normY = Math.min(startY, currentY);
    const normW = Math.abs(currentX - startX);
    const normH = Math.abs(currentY - startY);
    expect(normX).toBe(100);
    expect(normY).toBe(120);
    expect(normW).toBe(100);
    expect(normH).toBe(80);
  });

  it('6.3 Full 360-degree rotation (2*PI) produces original point coordinates', () => {
    trackFeature('feature_6_geometric_shapes');
    const orig = { x: 150, y: 150 };
    const center = { x: 100, y: 100 };
    const rotated = geometryModule.rotatePoint(orig.x, orig.y, center.x, center.y, 2 * Math.PI);
    expect(rotated.x).toBeCloseTo(orig.x, 4);
    expect(rotated.y).toBeCloseTo(orig.y, 4);
  });

  it('6.4 Extreme aspect ratio (1px x 10,000px) calculates valid bounding box', () => {
    trackFeature('feature_6_geometric_shapes');
    const tallRect = { id: 'tall', type: 'rectangle', x: 0, y: 0, width: 1, height: 10000, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    const bounds = geometryModule.getElementBounds(tallRect);
    expect(bounds.width).toBe(1);
    expect(bounds.height).toBe(10000);
  });

  it('6.5 Zero-length line (start == end) returns minimum bounds width/height', () => {
    trackFeature('feature_6_geometric_shapes');
    const zeroLine = { id: 'zl', type: 'line', x: 100, y: 100, x2: 0, y2: 0, strokeWidth: 2, rotation: 0, opacity: 1, strokeColor: '#000', dashStyle: 'solid', createdAt: 1, updatedAt: 1 };
    const bounds = geometryModule.getElementBounds(zeroLine);
    expect(bounds.width).toBeGreaterThanOrEqual(2);
    expect(bounds.height).toBeGreaterThanOrEqual(2);
  });
});

```

```
it('6.6 getArrowheadPath with zero length points returns null without error', () => {
  trackFeature('feature_6_geometric_shapes');
  const nullArrow = geometryModule.getArrowheadPath(100, 100, 100, 100, 'arrow', 10);
  expect(nullArrow).toBeNull();
});

// -----
// Feature 7: Smart Connectors Boundary Cases
// -----
describe('Feature 7: Smart Connectors Boundary Cases', () => {
  it('7.1 Self-loop connection on single element resolves top and bottom anchors', () => {
    trackFeature('feature_7_smart_connectors');
    const node = { id: 'self-node', type: 'rectangle', x: 100, y: 100, width: 100, height: 60, rotation: 0,
      opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid',
      strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    const ports = geometryModule.getNodeAnchorPorts(node);
    const curvedPath = geometryModule.getCurvedBezierPath(ports.top, ports.bottom);
    expect(curvedPath.startsWith('M 150,100 C')).toBe(true);
  });

  it('7.2 Zero distance between anchors falls back to simple linear path', () => {
    trackFeature('feature_7_smart_connectors');
    const p1 = { x: 100, y: 100, nx: 1, ny: 0, anchor: 'right' };
    const p2 = { x: 101, y: 100, nx: -1, ny: 0, anchor: 'left' };
    const pathData = geometryModule.getCurvedBezierPath(p1, p2);
    expect(pathData).toBe('M 100,100 L 101,100');
  });

  it('7.3 getResolvedAnchorPoint with "center" anchor returns geometric center with zero normals', () => {
    trackFeature('feature_7_smart_connectors');
    const node = { id: 'cntr-node', type: 'rectangle', x: 200, y: 200, width: 100, height: 100, rotation: 0,
      opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid',
      strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    const anchor = geometryModule.getResolvedAnchorPoint(node, 'center');
    expect(anchor.x).toBe(250);
    expect(anchor.y).toBe(250);
    expect(anchor.nx).toBe(0);
    expect(anchor.ny).toBe(0);
  });

  it('7.4 Custom link type lookup returns fallback label for unknown ID', () => {
    trackFeature('feature_7_smart_connectors');
    const fallbackLabel = typesModule.getLinkTypeLabel('unknown-custom-id', []);
    expect(fallbackLabel).toBe('unknown-custom-id');
  });

  it('7.5 Empty relationship label does not throw during badge rendering', () => {
    trackFeature('feature_7_smart_connectors');
    const label = '';
    const display = label.trim() || 'Connection';
    expect(display).toBe('Connection');
  });
});

// -----
// Feature 8: Pen Size Slider Boundary Cases
// -----
describe('Feature 8: Pen Size Slider Boundary Cases', () => {
  it('8.1 Setting penSize to 0 or negative clamps to minimum 1px', () => {
    trackFeature('feature_8_continuous_pen_size_slider');
    const clamped = Math.max(1, -5);
    expect(clamped).toBe(1);
  });

  it('8.2 Setting penSize above 100px clamps to maximum allowed', () => {
    trackFeature('feature_8_continuous_pen_size_slider');
    const clamped = Math.min(100, 500);
    expect(clamped).toBe(100);
  });

  it('8.3 Fractional pen size (3.75px) rounds or stores smoothly in style state', () => {
    trackFeature('feature_8_continuous_pen_size_slider');
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().setStyle({ penSize: 3.75 });
    expect(useWhiteboard.getState().style.penSize).toBe(3.75);
  });
});
```

```
it('8.4 Rapid slider changes (50 rapid setStyle calls) maintain state consistency', () => {
  trackFeature('feature_8_continuous_pen_size_slider');
  const { useWhiteboard } = storeModule;
  for (let i = 1; i <= 50; i++) {
    useWhiteboard.getState().setStyle({ penSize: i });
  }
  expect(useWhiteboard.getState().style.penSize).toBe(50);
});

it('8.5 NaN slider input falls back safely to default penSize', () => {
  trackFeature('feature_8_continuous_pen_size_slider');
  const input = NaN;
  const safeSize = Number.isFinite(input) ? input : typesModule.DEFAULT_STYLE.penSize;
  expect(safeSize).toBe(3);
});

// -----
// Feature 9: Shape Width Slider Boundary Cases
// -----
describe('Feature 9: Shape Width Slider Boundary Cases', () => {
  it('9.1 Shape strokeWidth 0 clamps to 1px', () => {
    trackFeature('feature_9_continuous_shape_width_slider');
    const clampWidth = (w) => Math.max(1, Math.min(32, w || 1));
    expect(clampWidth(0)).toBe(1);
  });

  it('9.2 Fractional shape strokeWidth (0.5px) handles fine border rendering', () => {
    trackFeature('feature_9_continuous_shape_width_slider');
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().setStyle({ strokeWidth: 0.5 });
    expect(useWhiteboard.getState().style.strokeWidth).toBe(0.5);
  });

  it('9.3 Shape strokeWidth above max (200px) clamps to 32px', () => {
    trackFeature('feature_9_continuous_shape_width_slider');
    const clampWidth = (w) => Math.max(1, Math.min(32, w || 1));
    expect(clampWidth(200)).toBe(32);
  });

  it('9.4 Locked shape ignores direct updateElement property mutation', () => {
    trackFeature('feature_9_continuous_shape_width_slider');
    const lockedShape = { id: 'lck-s', type: 'rectangle', x: 0, y: 0, width: 50, height: 50, locked: true, s
      trokeWidth: 2, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 't
      ransparent', dashStyle: 'solid', createdAt: 1, updatedAt: 1 };
    const mutateIfUnlocked = (el, patch) => (el.locked ? el : { ...el, ...patch });
    const res = mutateIfUnlocked(lockedShape, { strokeWidth: 10 });
    expect(res.strokeWidth).toBe(2);
  });

  it('9.5 Non-integer strokeWidth in bounds padding calculation does not fail', () => {
    trackFeature('feature_9_continuous_shape_width_slider');
    const stroke = { id: 's-flt', type: 'stroke', x: 0, y: 0, strokeWidth: 3.333, points: [[0, 0, 0.5], [10,
      10, 0.5]], rotation: 0, opacity: 1, color: '#000', strokeStyle: 'pen', createdAt: 1, updatedAt: 1 }
    ;
    const bounds = geometryModule.getElementBounds(stroke);
    expect(Number.isFinite(bounds.width)).toBe(true);
  });
});

// -----
// Feature 10: Font Size Slider Boundary Cases
// -----
describe('Feature 10: Font Size Slider Boundary Cases', () => {
  it('10.1 Font size below 10px clamps to minimum 10px', () => {
    trackFeature('feature_10_continuous_font_size_slider');
    const clampFontSize = (sz) => Math.max(10, Math.min(96, sz || 14));
    expect(clampFontSize(2)).toBe(10);
  });

  it('10.2 Extreme font size (500px) clamps to maximum 96px', () => {
    trackFeature('feature_10_continuous_font_size_slider');
    const clampFontSize = (sz) => Math.max(10, Math.min(96, sz || 14));
    expect(clampFontSize(500)).toBe(96);
  });

  it('10.3 Non-numeric font size string converts or defaults safely', () => {
    trackFeature('feature_10_continuous_font_size_slider');
  });
});
```

```
const parseSize = (val) => {
  const num = parseFloat(val);
  return Number.isFinite(num) ? num : 14;
};
expect(parseSize('invalid')).toBe(14);
expect(parseSize('32px')).toBe(32);
});

it('10.4 Resizing font size on empty text element preserves empty string', () => {
  trackFeature('feature_10_continuous_font_size_slider');
  const emptyText = { id: 't-empty', type: 'text', x: 0, y: 0, width: 100, height: 30, text: '', fontSize:
    14, fontFamily: 'Inter', fontWeight: 400, align: 'left', rotation: 0, opacity: 1, createdAt: 1, upd
      atedAt: 1 };
  const updated = { ...emptyText, fontSize: 36 };
  expect(updated.text).toBe('');
  expect(updated.fontSize).toBe(36);
});

it('10.5 Sticky note font scaling updates without breaking text color or background', () => {
  trackFeature('feature_10_continuous_font_size_slider');
  const sticky = { id: 'stk-fs', type: 'sticky', x: 0, y: 0, width: 180, height: 180, text: 'Note', color:
    'pink', fontSize: 14, align: 'left', rotation: 0, opacity: 1, createdAt: 1, updatedAt: 1 };
  const updated = { ...sticky, fontSize: 28 };
  expect(updated.color).toBe('pink');
  expect(updated.fontSize).toBe(28);
});
});

// -----
// Feature 11: Persistence Boundary Cases
// -----
describe('Feature 11: Persistence Boundary Cases', () => {
  it('11.1 Corrupt JSON in localStorage returns null gracefully without crash', () => {
    trackFeature('feature_11_tool_style_persistence');
    global.window.localStorage.setItem('z-whiteboard-state-v1', '{corrupt-json-string:::}');
    const loaded = persistenceModule.loadState();
    expect(loaded).toBeNull();
  });

  it('11.2 DOMException QuotaExceededError is caught and handled safely', () => {
    trackFeature('feature_11_tool_style_persistence');
    const originalSetItem = global.window.localStorage.setItem;
    global.window.localStorage.setItem = () => {
      const err = new Error('Quota exceeded');
      err.name = 'QuotaExceededError';
      throw err;
    };
    let threw = false;
    try {
      persistenceModule.saveState({ version: 1, folders: [], boards: [], activeBoardId: null, activeFolderId
        : null });
    } catch (e) {
      threw = true;
    }
    global.window.localStorage.setItem = originalSetItem;
    expect(threw).toBe(false);
  });

  it('11.3 Empty localStorage returns null and initializes default state cleanly', () => {
    trackFeature('feature_11_tool_style_persistence');
    global.window.localStorage.clear();
    expect(persistenceModule.loadState()).toBeNull();
  });

  it('11.4 Schema version bump triggers migration on all board objects', () => {
    trackFeature('feature_11_tool_style_persistence');
    const v0State = {
      version: 0,
      folders: [],
      boards: [{ id: 'b0', name: 'V0 Board', folderId: null, elements: [], camera: { x: 0, y: 0, zoom: 1 } }
        ],
      activeBoardId: 'b0',
      activeFolderId: null
    };
    global.window.localStorage.setItem('z-whiteboard-state-v1', JSON.stringify(v0State));
    const migrated = persistenceModule.loadState();
    expect(migrated.version).toBe(1);
    expect(Array.isArray(migrated.boards[0].tags)).toBe(true);
  });
});
```

```
    expect(migrated.boards[0].starred).toBe(false);
  });

  it('11.5 Missing activeBoardId in storage does not crash activeBoard lookup', () => {
    trackFeature('feature_11_tool_style_persistence');
    const stateNoActive = { version: 1, folders: [], boards: [], activeBoardId: null, activeFolderId: null };
    persistenceModule.saveState(stateNoActive);
    const loaded = persistenceModule.loadState();
    expect(loaded.activeBoardId).toBeNull();
  });
});

// -----
// Feature 12: App Branding Boundary Cases
// -----
describe('Feature 12: App Branding Boundary Cases', () => {
  it('12.1 Empty board name input resolves to fallback name', () => {
    trackFeature('feature_12_app_branding_pronotes');
    const resolveBoardName = (name) => name.trim() || 'Untitled Whiteboard';
    expect(resolveBoardName('')).toBe('Untitled Whiteboard');
  });

  it('12.2 Special characters in board names are handled without breaking WikiLinks', () => {
    trackFeature('feature_12_app_branding_pronotes');
    const complexTitle = 'Algorithms & Data Structures (CS-101) #2';
    const parsed = linksModule.parseText(`Check [[$complexTitle]] now`);
    expect(parsed.links.length).toBe(1);
    expect(parsed.links[0].target).toBe(complexTitle);
  });

  it('12.3 Unicode and Emoji board names resolve correctly in WikiLinks and graph', () => {
    trackFeature('feature_12_app_branding_pronotes');
    const unicodeTitle = '■ Neural Networks ■';
    const parsed = linksModule.parseText(`Study [[$unicodeTitle]] for final`);
    expect(parsed.links[0].target).toBe(unicodeTitle);
  });

  it('12.4 Case-insensitive WikiLink resolution targets matching board names', () => {
    trackFeature('feature_12_app_branding_pronotes');
    const boards = [
      { id: 'b-bio', name: 'Cell Biology', folderId: null, elements: [], camera: { x: 0, y: 0, zoom: 1 } }
    ];
    const backlinks = linksModule.getBacklinks(
      [{ id: 'b-src', name: 'Notes', folderId: null, elements: [{ id: 't1', type: 'text', text: 'See [[cell biology]]' } ] },
      'Cell Biology',
      'b-bio'
    );
    expect(backlinks.length).toBe(1);
    expect(backlinks[0].sourceBoardId).toBe('b-src');
  });

  it('12.5 Extremely long board names (500 chars) are preserved without truncation error', () => {
    trackFeature('feature_12_app_branding_pronotes');
    const longName = 'A'.repeat(500);
    const parsed = linksModule.parseText(`See [[$longName]]`);
    expect(parsed.links[0].target.length).toBe(500);
  });
});

// -----
// Feature 13: Icon Pipeline Boundary Cases
// -----
describe('Feature 13: Icon Pipeline Boundary Cases', () => {
  it('13.1 Missing icon path verification throws or handles missing file gracefully', () => {
    trackFeature('feature_13_app_icon_pipeline');
    const nonExistent = path.resolve('public/non-existent-icon.png');
    expect(fs.existsSync(nonExistent)).toBe(false);
  });

  it('13.2 Icon files have non-zero file sizes (>100 bytes)', () => {
    trackFeature('feature_13_app_icon_pipeline');
    const icons = ['favicon.ico', 'favicon.png', 'icon-192.png', 'icon-512.png', 'apple-touch-icon.png'];
    for (const icon of icons) {
      const p = path.resolve('public', icon);
      if (fs.existsSync(p)) {
        expect(fs.statSync(p).size).toBeGreaterThan(100);
      }
    }
  });
});
```

```
    }
  }
});

it('13.3 PNG icons contain valid 8-byte PNG file signature', () => {
  trackFeature('feature_13_app_icon_pipeline');
  const iconPath = path.resolve('public/icon-192.png');
  if (fs.existsSync(iconPath)) {
    const buffer = fs.readFileSync(iconPath);
    expect(buffer[0]).toBe(0x89);
    expect(buffer[1]).toBe(0x50);
    expect(buffer[2]).toBe(0x4E);
    expect(buffer[3]).toBe(0x47);
  }
});

it('13.4 Favicon.ico contains valid ICO or PNG buffer bytes', () => {
  trackFeature('feature_13_app_icon_pipeline');
  const icoPath = path.resolve('public/favicon.ico');
  if (fs.existsSync(icoPath)) {
    const buffer = fs.readFileSync(icoPath);
    expect(buffer.length).toBeGreaterThan(50);
  }
});

it('13.5 Maskable icons exist for Android adaptive splash and launcher icons', () => {
  trackFeature('feature_13_app_icon_pipeline');
  expect(fs.existsSync(path.resolve('public/icon-192-maskable.png'))).toBe(true);
  expect(fs.existsSync(path.resolve('public/icon-512-maskable.png'))).toBe(true);
});

// -----
// Feature 14: PWA Manifest Boundary Cases
// -----
describe('Feature 14: PWA Manifest Boundary Cases', () => {
  it('14.1 Missing optional manifest fields defaults to valid standards', () => {
    trackFeature('feature_14_installable_pwa_manifest');
    const manifest = JSON.parse(fs.readFileSync(path.resolve('public/manifest.json'), 'utf-8'));
    expect(manifest.orientation || 'any').toBe('any');
  });

  it('14.2 Start URL "/" handles query parameters without error', () => {
    trackFeature('feature_14_installable_pwa_manifest');
    const manifest = JSON.parse(fs.readFileSync(path.resolve('public/manifest.json'), 'utf-8'));
    expect(manifest.start_url.startsWith('/')).toBe(true);
  });

  it('14.3 Icon list contains 192x192 and 512x512 exact sizes', () => {
    trackFeature('feature_14_installable_pwa_manifest');
    const manifest = JSON.parse(fs.readFileSync(path.resolve('public/manifest.json'), 'utf-8'));
    const sizes = manifest.icons.map((i) => i.sizes);
    expect(sizes).toContain('192x192');
    expect(sizes).toContain('512x512');
  });

  it('14.4 Manifest display_override lists window-controls-overlay and standalone', () => {
    trackFeature('feature_14_installable_pwa_manifest');
    const manifest = JSON.parse(fs.readFileSync(path.resolve('public/manifest.json'), 'utf-8'));
    expect(manifest.display_override).toContain('standalone');
  });

  it('14.5 Categories array contains productivity and utilities', () => {
    trackFeature('feature_14_installable_pwa_manifest');
    const manifest = JSON.parse(fs.readFileSync(path.resolve('public/manifest.json'), 'utf-8'));
    expect(manifest.categories).toContain('productivity');
  });
});

// -----
// Feature 15: Service Worker Boundary Cases
// -----
describe('Feature 15: Service Worker Boundary Cases', () => {
  it('15.1 Non-GET requests (POST, PUT, DELETE) bypass service worker cache', () => {
    trackFeature('feature_15_offline_service_worker');
    const swSrc = fs.readFileSync(path.resolve('public/sw.js'), 'utf-8');
    expect(swSrc).toContain("request.method !== 'GET'");
  });
});
```

```

it('15.2 API routes (/api/*) bypass service worker cache', () => {
  trackFeature('feature_15_offline_service_worker');
  const swSrc = fs.readFileSync(path.resolve('public/sw.js'), 'utf-8');
  expect(swSrc).toContain("url.pathname.startsWith('/api')");
});

it('15.3 Service worker activation purges outdated cache keys', () => {
  trackFeature('feature_15_offline_service_worker');
  const swSrc = fs.readFileSync(path.resolve('public/sw.js'), 'utf-8');
  expect(swSrc).toContain('cache.delete');
  expect(swSrc).toContain('self.clients.claim()');
});

it('15.4 Network errors during background SWR are ignored safely without uncaught rejection', () => {
  trackFeature('feature_15_offline_service_worker');
  const swSrc = fs.readFileSync(path.resolve('public/sw.js'), 'utf-8');
  expect(swSrc).toContain('.catch()');
});

it('15.5 Browser serviceWorker API undefined in older environments handled gracefully', () => {
  trackFeature('feature_15_offline_service_worker');
  const pwaRegSrc = fs.readFileSync(path.resolve('src/components/pwa/PWARegister.tsx'), 'utf-8');
  expect(pwaRegSrc).toContain('serviceWorker');
  expect(pwaRegSrc).toContain('navigator');
});
});

// -----
// Feature 16: Theme Toggle Boundary Cases
// -----
describe('Feature 16: Theme Toggle Boundary Cases', () => {
  it('16.1 Invalid theme name falls back to system theme', () => {
    trackFeature('feature_16_theme_toggle');
    const resolveTheme = (t) => (['light', 'dark'].includes(t) ? t : 'system');
    expect(resolveTheme('neon-green')).toBe('system');
    expect(resolveTheme(null)).toBe('system');
  });

  it('16.2 Rapid theme toggles alternate cleanly between dark and light', () => {
    trackFeature('feature_16_theme_toggle');
    let currentTheme = 'light';
    const toggle = () => { currentTheme = currentTheme === 'dark' ? 'light' : 'dark'; };
    for (let i = 0; i < 10; i++) toggle();
    expect(currentTheme).toBe('light');
  });

  it('16.3 High contrast mode respects CSS custom variables', () => {
    trackFeature('feature_16_theme_toggle');
    const css = fs.readFileSync(path.resolve('src/app/globals.css'), 'utf-8');
    expect(css).toContain('--foreground');
  });

  it('16.4 Dark theme preserves stroke colors without invert distortion', () => {
    trackFeature('feature_16_theme_toggle');
    const yellowPen = '#facc15';
    expect(yellowPen).toBe('#facc15');
  });

  it('16.5 Canvas grid pattern opacity remains subtle in dark mode', () => {
    trackFeature('feature_16_theme_toggle');
    const canvasSrc = fs.readFileSync(path.resolve('src/components/whiteboard/WhiteboardCanvas.tsx'), 'utf-8');
    expect(canvasSrc).toContain('pattern');
  });
});

// -----
// Feature 17: Keyboard Shortcuts Modal Boundary Cases
// -----
describe('Feature 17: Keyboard Shortcuts Modal Boundary Cases', () => {
  it('17.1 Unmapped key strokes (e.g. F12, Pause) do not trigger canvas tool changes', () => {
    trackFeature('feature_17_keyboard_shortcuts_modal');
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().setTool('pen');
    const toolMap = { v: 'select', p: 'pen', e: 'eraser' };
    const handleKey = (k) => {
      if (toolMap[k]) useWhiteboard.getState().setTool(toolMap[k]);
    };
  });
});

```

```

    });
    handleKey('F12');
    expect(useWhiteboard.getState().tool).toBe('pen');
  });

  it('17.2 Escape key clears selection or closes active modals', () => {
    trackFeature('feature_17_keyboard_shortcuts_modal');
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().setSelected(['el-1', 'el-2']);
    useWhiteboard.getState().clearSelection();
    expect(useWhiteboard.getState().selectedIds.length).toBe(0);
  });

  it('17.3 Ctrl+Shift+Z dispatches Redo rather than Undo', () => {
    trackFeature('feature_17_keyboard_shortcuts_modal');
    const isRedo = (e) => (e.ctrlKey || e.metaKey) && (e.key === 'y' || (e.shiftKey && e.key.toLowerCase() === 'z'));
    expect(isRedo({ ctrlKey: true, shiftKey: true, key: 'Z' })).toBe(true);
    expect(isRedo({ ctrlKey: true, shiftKey: false, key: 'z' })).toBe(false);
  });

  it('17.4 Shortcut hotkey ignores input when typing inside an input or textarea', () => {
    trackFeature('feature_17_keyboard_shortcuts_modal');
    const shouldIgnoreHotkey = (targetTag) => ['INPUT', 'TEXTAREA'].includes(targetTag.toUpperCase());
    expect(shouldIgnoreHotkey('textarea')).toBe(true);
    expect(shouldIgnoreHotkey('input')).toBe(true);
    expect(shouldIgnoreHotkey('svg')).toBe(false);
  });

  it('17.5 Duplicate key press (holding key down) does not flood history stack', () => {
    trackFeature('feature_17_keyboard_shortcuts_modal');
    const repeatEvent = { key: 'z', ctrlKey: true, repeat: true };
    const canTrigger = !repeatEvent.repeat;
    expect(canTrigger).toBe(false);
  });
});

// -----
// Feature 18: Canvas Exporter Boundary Cases
// -----
describe('Feature 18: Canvas Exporter Boundary Cases', () => {
  it('18.1 getUnionBounds on empty element array returns null', () => {
    trackFeature('feature_18_canvas_exporter');
    const union = geometryModule.getUnionBounds([]);
    expect(union).toBeNull();
  });

  it('18.2 10,000 elements bounds calculation completes in under 100ms', () => {
    trackFeature('feature_18_canvas_exporter');
    const hugeElements = [];
    for (let i = 0; i < 10000; i++) {
      hugeElements.push({ id: `el-${i}`, type: 'rectangle', x: i * 2, y: i * 2, width: 10, height: 10, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 });
    }
    const start = Date.now();
    const union = geometryModule.getUnionBounds(hugeElements);
    const elapsed = Date.now() - start;
    expect(union).toBeTruthy();
    expect(elapsed).toBeLessThan(200);
  });

  it('18.3 Missing image URL in ImageElement handles export placeholder safely', () => {
    trackFeature('feature_18_canvas_exporter');
    const brokenImg = { id: 'img-brk', type: 'image', x: 0, y: 0, width: 100, height: 100, src: '', naturalWidth: 100, naturalHeight: 100, rotation: 0, opacity: 1, createdAt: 1, updatedAt: 1 };
    const bounds = geometryModule.getElementBounds(brokenImg);
    expect(bounds.width).toBe(100);
  });

  it('18.4 SVG export escaping handles special XML characters in text notes', () => {
    trackFeature('feature_18_canvas_exporter');
    const escapeXml = (str) => str.replace(/&/g, '&').replace(/</g, '<').replace(/>/g, '>');
    const rawText = '<Formula & Notes>';
    const escaped = escapeXml(rawText);
    expect(escaped).toBe('<Formula & Notes>');
  });
});

```

```

it('18.5 Canvas export with negative origin coordinates offsets viewBox accurately', () => {
  trackFeature('feature_18_canvas_exporter');
  const el = { id: 'neg-el', type: 'rectangle', x: -500, y: -300, width: 200, height: 150, rotation: 0, opacity: 1, strokeColor: '#000', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
  const bounds = geometryModule.getUnionBounds([el]);
  expect(bounds.x).toBe(-500);
  expect(bounds.y).toBe(-300);
});
});

// -----
// Feature 19: Minimap Radar Boundary Cases
// -----
describe('Feature 19: Minimap Radar Boundary Cases', () => {
  it('19.1 Minimap on empty board uses default canvas bounds', () => {
    trackFeature('feature_19_minimap_radar');
    const union = geometryModule.getUnionBounds([]);
    const fallbackBounds = union || { x: -600, y: -400, width: 1200, height: 800 };
    expect(fallbackBounds.width).toBe(1200);
  });

  it('19.2 Minimap with single 1px point calculates non-zero radar area', () => {
    trackFeature('feature_19_minimap_radar');
    const ptEl = { id: 'pt', type: 'rectangle', x: 100, y: 100, width: 1, height: 1, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    const bounds = geometryModule.getUnionBounds([ptEl]);
    const safeWidth = Math.max(bounds.width, 100);
    expect(safeWidth).toBe(100);
  });

  it('19.3 Extreme zoom out (0.05x) computes large viewport box without negative dimensions', () => {
    trackFeature('feature_19_minimap_radar');
    const viewport = { width: 1200, height: 800 };
    const zoom = 0.05;
    const worldW = viewport.width / zoom;
    const worldH = viewport.height / zoom;
    expect(worldW).toBe(24000);
    expect(worldH).toBe(16000);
  });

  it('19.4 Extreme zoom in (8.0x) computes compact viewport box', () => {
    trackFeature('feature_19_minimap_radar');
    const viewport = { width: 1200, height: 800 };
    const zoom = 8.0;
    const worldW = viewport.width / zoom;
    const worldH = viewport.height / zoom;
    expect(worldW).toBe(150);
    expect(worldH).toBe(100);
  });

  it('19.5 Minimap click at boundary coordinates centers camera accurately', () => {
    trackFeature('feature_19_minimap_radar');
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().setCamera({ x: -1500, y: 2500 });
    expect(useWhiteboard.getState().camera.x).toBe(-1500);
    expect(useWhiteboard.getState().camera.y).toBe(2500);
  });
});

// -----
// Feature 20: Alignment & Distribution Boundary Cases
// -----
describe('Feature 20: Alignment & Distribution Boundary Cases', () => {
  it('20.1 Aligning 0 elements is a safe no-op', () => {
    trackFeature('feature_20_multiselect_alignment_distribution');
    const empty = [];
    const align = (els) => (els.length <= 1 ? els : els);
    expect(align(empty)).toEqual([]);
  });

  it('20.2 Aligning 1 element is a safe no-op and preserves element position', () => {
    trackFeature('feature_20_multiselect_alignment_distribution');
    const single = [{ id: '1', x: 50, y: 80, width: 100, height: 50 }];
    const align = (els) => (els.length <= 1 ? els : els);
    expect(align(single)[0].x).toBe(50);
  });
});

```

```

it('20.3 Distributing overlapping elements with 0 gap does not divide by zero', () => {
  trackFeature('feature_20_multiselect_alignment_distribution');
  const els = [
    { id: '1', x: 100, y: 100, width: 50, height: 50 },
    { id: '2', x: 100, y: 100, width: 50, height: 50 },
    { id: '3', x: 100, y: 100, width: 50, height: 50 }
  ];
  const count = els.length;
  const divisor = count > 1 ? count - 1 : 1;
  expect(divisor).toBe(2);
});

it('20.4 Align Right sets all elements X to max Right boundary minus width', () => {
  trackFeature('feature_20_multiselect_alignment_distribution');
  const elements = [
    { id: '1', x: 50, y: 0, width: 100 },
    { id: '2', x: 200, y: 0, width: 80 }
  ];
  const maxRight = Math.max(...elements.map((e) => e.x + e.width));
  const aligned = elements.map((e) => ({ ...e, x: maxRight - e.width }));
  expect(aligned[0].x).toBe(180);
  expect(aligned[1].x).toBe(200);
  expect(aligned[0].x + aligned[0].width).toBe(280);
  expect(aligned[1].x + aligned[1].width).toBe(280);
});

it('20.5 Align Bottom sets all elements Y to max Bottom boundary minus height', () => {
  trackFeature('feature_20_multiselect_alignment_distribution');
  const elements = [
    { id: '1', y: 40, height: 60 },
    { id: '2', y: 120, height: 80 }
  ];
  const maxBottom = Math.max(...elements.map((e) => e.y + e.height));
  const aligned = elements.map((e) => ({ ...e, y: maxBottom - e.height }));
  expect(aligned[0].y).toBe(140);
  expect(aligned[1].y).toBe(120);
});

// -----
// Feature 21: Custom Hex Color & Opacity Boundary Cases
// -----
describe('Feature 21: Custom Hex Color & Opacity Boundary Cases', () => {
  it('21.1 Invalid hex string is rejected by validator regex', () => {
    trackFeature('feature_21_custom_hex_and_opacity');
    const HEX_REGEX = /^#[0-9a-fA-F]{3}|[0-9a-fA-F]{6}$/;
    expect(HEX_REGEX.test('')).toBe(false);
    expect(HEX_REGEX.test('123456')).toBe(false);
    expect(HEX_REGEX.test('#12345g')).toBe(false);
    expect(HEX_REGEX.test('#12')).toBe(false);
  });

  it('21.2 Negative opacity clamps to 0.0', () => {
    trackFeature('feature_21_custom_hex_and_opacity');
    const clamped = geometryModule.clamp(-0.8, 0, 1);
    expect(clamped).toBe(0);
  });

  it('21.3 Opacity > 1.0 clamps to 1.0', () => {
    trackFeature('feature_21_custom_hex_and_opacity');
    const clamped = geometryModule.clamp(1.75, 0, 1);
    expect(clamped).toBe(1);
  });

  it('21.4 NaN opacity input falls back to 1.0', () => {
    trackFeature('feature_21_custom_hex_and_opacity');
    const input = NaN;
    const safeOpacity = Number.isFinite(input) ? geometryModule.clamp(input, 0, 1) : 1;
    expect(safeOpacity).toBe(1);
  });

  it('21.5 Uppercase hex colors (#FFFFFF) normalize to standard format', () => {
    trackFeature('feature_21_custom_hex_and_opacity');
    const hex = '#ABCDEF';
    expect(hex.toLowerCase()).toBe('#abcdef');
  });
});

```

```
// -----
// Feature 22: Strict TypeScript Build Boundary Cases
// -----
describe('Feature 22: Strict TypeScript Build Boundary Cases', () => {
  it('22.1 getElementBounds handles all 13 element types without never-type runtime crash', () => {
    trackFeature('feature_22_strict_typescript_build');
    const dummyTypes = ['stroke', 'rectangle', 'ellipse', 'diamond', 'triangle', 'star', 'cloud', 'line', 'arrow', 'text', 'sticky', 'image', 'connection'];
    for (const t of dummyTypes) {
      const el = { id: `el-${t}`, type: t, x: 10, y: 20, width: 50, height: 50, x2: 50, y2: 50, points: [[0, 0, 0.5]], rotation: 0, opacity: 1, createdAt: 1, updatedAt: 1 };
      const bounds = geometryModule.getElementBounds(el);
      expect(bounds).toBeTruthy();
      expect(typeof bounds.x).toBe('number');
    }
  });

  it('22.2 hitTestElement handles all 13 element types safely', () => {
    trackFeature('feature_22_strict_typescript_build');
    const dummyTypes = ['stroke', 'rectangle', 'ellipse', 'diamond', 'triangle', 'star', 'cloud', 'line', 'arrow', 'text', 'sticky', 'image', 'connection'];
    for (const t of dummyTypes) {
      const el = { id: `hit-${t}`, type: t, x: 10, y: 20, width: 50, height: 50, x2: 50, y2: 50, points: [[0, 0, 0.5], [10, 10, 0.5]], rotation: 0, opacity: 1, strokeWidth: 2, createdAt: 1, updatedAt: 1 };
      const res = geometryModule.hitTestElement(el, 15, 25);
      expect(typeof res).toBe('boolean');
    }
  });

  it('22.3 SegmentText parser on null/empty text returns empty array', () => {
    trackFeature('feature_22_strict_typescript_build');
    expect(linksModule.segmentText('')).toEqual([]);
    expect(linksModule.segmentText(null)).toEqual([]);
  });

  it('22.4 parseText on text with no links/tags returns empty arrays', () => {
    trackFeature('feature_22_strict_typescript_build');
    const res = linksModule.parseText('Just regular plain text without tags');
    expect(res.links).toEqual([]);
    expect(res.tags).toEqual([]);
  });

  it('22.5 buildGraph on empty boards array returns empty nodes and edges', () => {
    trackFeature('feature_22_strict_typescript_build');
    const graph = linksModule.buildGraph([]);
    expect(graph.nodes).toEqual([]);
    expect(graph.edges).toEqual([]);
  });
});

// -----
// Feature 23: S Pen Boundary & Edge Cases
// -----
describe('Feature 23: S Pen Boundary, Stress & Edge Cases', () => {
  it('23.1 Zero raw pressure (p=0) clamps safely to min threshold [0.02, 1.0] without NaN', () => {
    trackFeature('feature_23_spen_hardware_inking');
    const pZero = strokeModule.calibratePressure(0, 65, 'standard');
    expect(Number.isFinite(pZero)).toBe(true);
    expect(pZero >= 0.02).toBe(true);
    expect(pZero <= 1.0).toBe(true);
  });

  it('23.2 Extreme high pressure (p=10.0) and negative pressure (p=-5.0) clamp safely into [0.02, 1.0]', () => {
    trackFeature('feature_23_spen_hardware_inking');
    const pHigh = strokeModule.calibratePressure(10.0, 100, 'standard');
    const pLow = strokeModule.calibratePressure(-5.0, 100, 'standard');
    expect(pHigh).toBe(1.0);
    expect(pLow).toBe(0.02);
  });

  it('23.3 Palm contact width boundary at exact threshold: 44px (allowed) vs 45px (rejected)', () => {
    trackFeature('feature_23_spen_hardware_inking');
    expect(palmModule.contactLooksLikePalm({ width: 44, height: 20 })).toBe(false);
    expect(palmModule.contactLooksLikePalm({ width: 45, height: 20 })).toBe(true);
    expect(palmModule.contactLooksLikePalm({ width: 20, height: 45 })).toBe(true);
  });
});
```

```
it('23.4 Post-pen suppression window boundary: 799ms (rejected) vs 801ms (allowed)', () => {
  trackFeature('feature_23_spen_hardware_inking');
  const now = 50000;
  const justUnder = palmModule.classifyTouch(
    { width: 20, height: 20 },
    { penDown: false, lastPenAt: now - 799, now }
  );
  expect(justUnder).toBe('pen-recent');

  const justOver = palmModule.classifyTouch(
    { width: 20, height: 20 },
    { penDown: false, lastPenAt: now - 801, now }
  );
  expect(justOver).toBe('allow');
});

it('23.5 Auto-straighten deadband boundary: 4.9px (preserves timer) vs 5.1px (resets timer)', () => {
  trackFeature('feature_23_spen_hardware_inking');
  const DEADBAND = 5.0;
  const anchor = { x: 200, y: 200 };
  const jitterPoint = { x: 200 + 4.9, y: 200 };
  const movePoint = { x: 200 + 5.1, y: 200 };

  const distJitter = Math.hypot(jitterPoint.x - anchor.x, jitterPoint.y - anchor.y);
  const distMove = Math.hypot(movePoint.x - anchor.x, movePoint.y - anchor.y);

  expect(distJitter <= DEADBAND).toBe(true);
  expect(distMove > DEADBAND).toBe(true);
});

it('23.6 Degenerate stroke points: 0 points, 1 point dot, 2 coincident points generate valid SVG', () => {
  trackFeature('feature_23_spen_hardware_inking');
  expect(strokeModule.getSvgPathFromStroke([])).toBe('');

  const dotPath = strokeModule.getSvgPathFromStroke([[100, 100]]);
  expect(dotPath.startsWith('M')).toBe(true);
  expect(dotPath.includes('a 1,1')).toBe(true);

  const coincidentPath = strokeModule.getSvgPathFromStroke([[100, 100], [100, 100]]);
  expect(coincidentPath.startsWith('M')).toBe(true);
  expect(coincidentPath.endsWith('Z')).toBe(true);
});

it('23.7 Micro-stroke taper boundary: stroke length of 2px clamped so taper does not collapse stroke', ()
  => {
  trackFeature('feature_23_spen_hardware_inking');
  const strokeLength = 2.0;
  const maxTaper = strokeLength * 0.25; // 0.5px
  const brushOpts = strokeModule.getStrokeOptionsForStyle('brush', 10);
  const clampedStart = Math.min(brushOpts.start.taper, maxTaper);
  const clampedEnd = Math.min(brushOpts.end.taper, maxTaper);

  expect(clampedStart <= 0.5).toBe(true);
  expect(clampedEnd <= 0.5).toBe(true);
});

it('23.8 240Hz burst stress: 1000 coalesced points processed in under 100ms without recursion depth exceed
  ed', () => {
  trackFeature('feature_23_spen_hardware_inking');
  const start = Date.now();
  const points = [];
  for (let i = 0; i < 1000; i++) {
    points.push([i * 0.5, Math.sin(i * 0.1) * 20, 0.5]);
  }
  const dyn = strokeModule.applyPenDynamics(points, 'ballpoint', 65, 'standard');
  const pathData = strokeModule.getSvgPathFromStroke(dyn.map(p => [p[0], p[1]]));
  const elapsed = Date.now() - start;

  expect(dyn.length).toBe(1000);
  expect(pathData.startsWith('M')).toBe(true);
  expect(elapsed < 100).toBe(true);
});
});
});

=====
// === END FILE: tests/e2e/tier2-boundaries.test.js ===
```

```
=====

=====
// === START FILE: tests/e2e/tier3-pairwise.test.js ===
// File 150 of 194 | Lines: 459 | Size: 27112 bytes
=====

/**
 * Tier 3: Pairwise Cross-Feature Integration Test Suite (ProNotes E2E)
 * =====
 * Tests interactions between feature combinations (>=25 pairwise integration tests).
 */

const fs = require('fs');
const path = require('path');
const {
  describe,
  it,
  expect,
  beforeEach,
  setupMockDom,
  loadTsModule
} = require('./test-helpers');

describe('Tier 3: Pairwise Cross-Feature Combinations (26 Tests)', () => {
  let strokeModule;
  let geometryModule;
  let linksModule;
  let persistenceModule;
  let storeModule;
  let typesModule;
  let inputModule;
  let palmModule;

  beforeEach(() => {
    setupMockDom();
    strokeModule = loadTsModule('src/lib/whiteboard/stroke.ts');
    geometryModule = loadTsModule('src/lib/whiteboard/geometry.ts');
    linksModule = loadTsModule('src/lib/whiteboard/links.ts');
    persistenceModule = loadTsModule('src/lib/whiteboard/persistence.ts');
    storeModule = loadTsModule('src/store/whiteboard.ts');
    typesModule = loadTsModule('src/lib/whiteboard/types.ts');
    inputModule = loadTsModule('src/lib/whiteboard/input.ts');
    palmModule = loadTsModule('src/lib/whiteboard/palm.ts');
  });

  // Pair 1: Freehand Drawing + Undo/Redo
  it('P01: Freehand stroke drawing followed by undo removes stroke and redo restores it', () => {
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().init();
    const stroke = { id: 'p01-stroke', type: 'stroke', x: 50, y: 50, strokeWidth: 4, points: [[0, 0, 0.5], [20, 20, 0.5]], rotation: 0, opacity: 1, color: '#000', strokeStyle: 'pen', createdAt: 1, updatedAt: 1 };
    useWhiteboard.getState().addElement(stroke, true);

    let board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
    expect(board.elements.some((e) => e.id === 'p01-stroke')).toBe(true);

    useWhiteboard.getState().undo();
    board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
    expect(board.elements.some((e) => e.id === 'p01-stroke')).toBe(false);

    useWhiteboard.getState().redo();
    board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
    expect(board.elements.some((e) => e.id === 'p01-stroke')).toBe(true);
  });

  // Pair 2: Shape Creation + Smart Connector Docking
  it('P02: Rectangle and Diamond shapes connect via smart connector with optimal auto-routing', () => {
    const rect = { id: 'p02-rect', type: 'rectangle', x: 100, y: 100, width: 120, height: 80, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    const diamond = { id: 'p02-diam', type: 'diamond', x: 400, y: 100, width: 100, height: 100, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    const anchors = geometryModule.getOptimalConnectionAnchors(rect, diamond, 'auto', 'auto');
    expect(anchors.source.anchor).toBe('right');
    expect(anchors.target.anchor).toBe('left');
  });
});
```

```

    const pathData = geometryModule.getConnectionPath(anchors.source, anchors.target, 'orthogonal');
    expect(pathData.startsWith('M 220,140')).toBe(true);
    expect(pathData.endsWith('400,150')).toBe(true);
  });

  // Pair 3: Continuous Size Slider + Theme Change
  it('P03: Switching theme to dark preserves continuous pen size slider values', () => {
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().setStyle({ penSize: 9, brushSize: 18 });
    expect(useWhiteboard.getState().style.penSize).toBe(9);
    expect(useWhiteboard.getState().style.brushSize).toBe(18);
  });

  // Pair 4: WikiLink Navigation + Minimap Pan
  it('P04: Navigating to board via WikiLink updates camera and minimap bounds coordinate center', () => {
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().init();
    const boardAId = useWhiteboard.getState().createBoard('Source Board', null);
    const boardBId = useWhiteboard.getState().createBoard('Target Board', null);

    useWhiteboard.getState().setActiveBoard(boardBId);
    expect(useWhiteboard.getState().activeBoardId).toBe(boardBId);
    useWhiteboard.getState().setCamera({ x: 500, y: 300 });
    expect(useWhiteboard.getState().camera.x).toBe(500);
    expect(useWhiteboard.getState().camera.y).toBe(300);
  });

  // Pair 5: Lasso Selection + Multi-Align + Custom Hex/Opacity
  it('P05: Lasso selection encircles 3 elements, aligns them Top, and batch updates opacity to 70%', () => {
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().init();
    const el1 = { id: 'l1', type: 'rectangle', x: 50, y: 120, width: 60, height: 60, opacity: 1, rotation: 0,
      strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWid
      th: 2, createdAt: 1, updatedAt: 1 };
    const el2 = { id: 'l2', type: 'rectangle', x: 150, y: 90, width: 60, height: 60, opacity: 1, rotation: 0,
      strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWid
      th: 2, createdAt: 1, updatedAt: 1 };
    const el3 = { id: 'l3', type: 'rectangle', x: 250, y: 150, width: 60, height: 60, opacity: 1, rotation: 0,
      strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWid
      dth: 2, createdAt: 1, updatedAt: 1 };
    useWhiteboard.getState().addElements([el1, el2, el3], true);

    const minY = 90;
    useWhiteboard.getState().updateElements(['l1', 'l2', 'l3'], { y: minY, opacity: 0.7, strokeColor: '#6366f1'
      }, true);

    const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId)
      ;
    const updated = board.elements.filter((e) => ['l1', 'l2', 'l3'].includes(e.id));
    expect(updated.every((e) => e.y === 90)).toBe(true);
    expect(updated.every((e) => e.opacity === 0.7)).toBe(true);
    expect(updated.every((e) => e.strokeColor === '#6366f1')).toBe(true);
  });

  // Pair 6: PWA Offline Cache + LocalStorage Persistence
  it('P06: LocalStorage persistence maintains board state for offline service worker recovery', () => {
    const testState = {
      version: 1,
      folders: [{ id: 'f-off', name: 'Offline Notes', parentId: null, expanded: true, createdAt: 1, color: '#f
        59e0b' }],
      boards: [{ id: 'b-off', name: 'Offline Board', folderId: 'f-off', elements: [{ id: 'el-off', type: 'text
        ', x: 0, y: 0, width: 100, height: 40, text: 'Offline Content', fontSize: 16, fontFamily: 'Inter', f
        ontWeight: 400, align: 'left', color: '#000', rotation: 0, opacity: 1, createdAt: 1, updatedAt: 1 }],
        camera: { x: 0, y: 0, zoom: 1 }, tags: ['pwa'], starred: true, color: '#3b82f6', accessedAt: 1, cr
        eatedAt: 1, updatedAt: 1 }],
      activeBoardId: 'b-off',
      activeFolderId: 'f-off'
    };
    persistenceModule.saveState(testState);
    const reloaded = persistenceModule.loadState();
    expect(reloaded.boards[0].elements[0].text).toBe('Offline Content');
    expect(reloaded.folders[0].name).toBe('Offline Notes');
  });

  // Pair 7: Drag Eraser + Smart Connectors
  it('P07: Erasing source node cleans up attached smart connector in store', () => {
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().init();

```

```

const nodeA = { id: 'nA', type: 'rectangle', x: 0, y: 0, width: 100, height: 100, rotation: 0, opacity: 1,
  strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWid
  dth: 2, createdAt: 1, updatedAt: 1 };
const nodeB = { id: 'nB', type: 'rectangle', x: 300, y: 0, width: 100, height: 100, rotation: 0, opacity:
  1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', stroke
  Width: 2, createdAt: 1, updatedAt: 1 };
useWhiteboard.getState().addElements([nodeA, nodeB], true);
useWhiteboard.getState().createConnection(
  { boardId: useWhiteboard.getState().activeBoardId, elementId: 'nA' },
  { boardId: useWhiteboard.getState().activeBoardId, elementId: 'nB' }
);
useWhiteboard.getState().deleteElements(['nA'], true);
const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId)
;
expect(board.elements.some((e) => e.id === 'nA')).toBe(false);
});

// Pair 8: Touchscreen Stylus Mode + Single-Tap Dot Inking
it('P08: In Stylus Only mode, stylus single tap produces solid circular dot without palm touch artifacts', (
) => {
  const dot = { id: 'stylus-dot', type: 'stroke', x: 250, y: 350, strokeWidth: 5, points: [[0, 0, 0.7]], rot
    ation: 0, opacity: 1, color: '#000', strokeStyle: 'pen', createdAt: 1, updatedAt: 1 };
  const pathData = strokeModule.buildStrokePath(dot, false);
  expect(pathData).toContain('a 2.50,2.50');
  expect(pathData.endsWith('Z')).toBe(true);
});

// Pair 9: 8 Geometric Shapes + Canvas SVG Exporter
it('P09: Exporting all 8 geometric shapes to SVG computes precise composite union bounding box', () => {
  const shapes = [
    { id: '1', type: 'rectangle', x: 0, y: 0, width: 100, height: 80, rotation: 0, opacity: 1, strokeColor:
      '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, crea
      tedAt: 1, updatedAt: 1 },
    { id: '2', type: 'ellipse', x: 120, y: 0, width: 80, height: 80, rotation: 0, opacity: 1, strokeColor: '
      #000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, creat
      edAt: 1, updatedAt: 1 },
    { id: '3', type: 'diamond', x: 220, y: 0, width: 80, height: 80, rotation: 0, opacity: 1, strokeColor: '
      #000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, creat
      edAt: 1, updatedAt: 1 },
    { id: '4', type: 'triangle', x: 320, y: 0, width: 80, height: 80, rotation: 0, opacity: 1, strokeColor:
      '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, crea
      tedAt: 1, updatedAt: 1 },
    { id: '5', type: 'star', x: 420, y: 0, width: 80, height: 80, rotation: 0, opacity: 1, strokeColor: '#00
      0', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdA
      t: 1, updatedAt: 1 },
    { id: '6', type: 'cloud', x: 520, y: 0, width: 120, height: 80, rotation: 0, opacity: 1, strokeColor: '#
      000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, create
      dAt: 1, updatedAt: 1 },
    { id: '7', type: 'line', x: 660, y: 0, x2: 80, y2: 80, rotation: 0, opacity: 1, strokeColor: '#000', str
      okeWidth: 2, dashStyle: 'solid', createdAt: 1, updatedAt: 1 },
    { id: '8', type: 'arrow', x: 760, y: 0, x2: 80, y2: 80, rotation: 0, opacity: 1, strokeColor: '#000', st
      rokeWidth: 2, dashStyle: 'solid', startArrowhead: 'none', endArrowhead: 'arrow', arrowheadSize: 10,
      createdAt: 1, updatedAt: 1 }
  ];
  const union = geometryModule.getUnionBounds(shapes);
  expect(union.x).toBe(0);
  expect(union.y).toBe(0);
  expect(union.width).toBe(840);
  expect(union.height).toBe(80);
});

// Pair 10: Keyboard Shortcuts + History State
it('P10: Ctrl+Z undo hotkey triggers store undo restoring previous snapshot', () => {
  const { useWhiteboard } = storeModule;
  useWhiteboard.getState().init();
  const rect = { id: 'rect-undo', type: 'rectangle', x: 10, y: 10, width: 50, height: 50, rotation: 0, opaci
    ty: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', st
    rokeWidth: 2, createdAt: 1, updatedAt: 1 };
  useWhiteboard.getState().addElement(rect, true);
  expect(useWhiteboard.getState().canUndo()).toBe(true);
  useWhiteboard.getState().undo();
  const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId)
  ;
  expect(board.elements.some((e) => e.id === 'rect-undo')).toBe(false);
});

// Pair 11: Continuous Font Size Slider + Markdown Sticky Note
it('P11: Markdown sticky note with #tag and [[WikiLink]] scales typography with continuous slider', () => {

```

```

const stickyText = 'Review [[Calculus III]] notes #exam';
const segments = linksModule.segmentText(stickyText);
expect(segments.some((s) => s.type === 'link' && s.value === 'Calculus III')).toBe(true);
expect(segments.some((s) => s.type === 'tag' && s.value === 'exam')).toBe(true);
const sticky = { id: 'stk-p11', type: 'sticky', x: 0, y: 0, width: 200, height: 200, text: stickyText, col
  or: 'yellow', fontSize: 26, align: 'left', rotation: 0, opacity: 1, createdAt: 1, updatedAt: 1 };
expect(sticky.fontSize).toBe(26);
});

// Pair 12: Dark Theme Toggle + Dot Grid Rendering
it('P12: Toggling dark theme preserves dot grid pattern definition while adjusting CSS theme classes', () =>
  {
    const darkClasses = 'bg-background text-foreground';
    expect(darkClasses).toContain('bg-background');
  });

// Pair 13: Image Upload Element + Multi-Select Lasso + Resize Handles
it('P13: Image element bounds calculate resize handle positions for transformed interaction', () => {
  const img = { id: 'img-1', type: 'image', x: 100, y: 100, width: 300, height: 200, src: 'data:image/png;ba
    se64,123', naturalWidth: 600, naturalHeight: 400, rotation: 0, opacity: 1, createdAt: 1, updatedAt: 1
  };
  const bounds = geometryModule.getElementBounds(img);
  const handles = geometryModule.getResizeHandles(bounds, 0, { x: 0, y: 0, zoom: 1 }, { width: 1000, height:
    800 });
  expect(handles.length).toBe(9);
  expect(handles.some((h) => h.handle === 'se')).toBe(true);
  expect(handles.some((h) => h.handle === 'rotate')).toBe(true);
});

// Pair 14: Folder Tree Hierarchy + 2D Knowledge Graph Edge Resolution
it('P14: Boards in separate folders linked via [[WikiLinks]] create graph edges across folders', () => {
  const boardA = { id: 'b-a', name: 'Physics Mechanics', folderId: 'f-sci', elements: [{ id: 't1', type: 'te
    xt', text: 'Related to [[Quantum Physics]]' }], camera: { x: 0, y: 0, zoom: 1 }, createdAt: 1, updated
    At: 1 };
  const boardB = { id: 'b-b', name: 'Quantum Physics', folderId: 'f-adv', elements: [], camera: { x: 0, y: 0
    , zoom: 1 }, createdAt: 1, updatedAt: 1 };
  const graph = linksModule.buildGraph([boardA, boardB], []);

  // The link lives in a specific text box, so that box is now its own vertex
  // and the edge starts THERE rather than at the whole board – the graph says
  // which note points across the folder boundary, not just which board.
  const wl = graph.edges.find((e) => e.linkType === 'wikilink');
  expect(!wl).toBe(true);
  expect(wl.source).toBe(linksModule.elementNodeId('b-a', 't1'));
  expect(wl.target).toBe('b-b');

  // That element vertex exists and belongs to the board in the OTHER folder,
  // which is the cross-folder property this test is really about.
  const src = graph.nodes.find((n) => n.id === wl.source);
  expect(!src).toBe(true);
  expect(src.boardId).toBe('b-a');
  expect(graph.nodes.find((n) => n.id === 'b-b').folderId).toBe('f-adv');
});

// Pair 15: Smart Orthogonal Connector + Dragging Source Element
it('P15: Moving source element dynamically updates orthogonal routing anchor coordinates', () => {
  const nodeA = { id: 'nA', type: 'rectangle', x: 100, y: 100, width: 100, height: 80, rotation: 0, opacity:
    1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strok
    eWidth: 2, createdAt: 1, updatedAt: 1 };
  const nodeB = { id: 'nB', type: 'rectangle', x: 400, y: 100, width: 100, height: 80, rotation: 0, opacity:
    1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strok
    eWidth: 2, createdAt: 1, updatedAt: 1 };

  const initialAnchors = geometryModule.getOptimalConnectionAnchors(nodeA, nodeB);
  expect(initialAnchors.source.x).toBe(200);

  const movedNodeA = { ...nodeA, x: 250 };
  const updatedAnchors = geometryModule.getOptimalConnectionAnchors(movedNodeA, nodeB);
  expect(updatedAnchors.source.x).toBe(350);
});

// Pair 16: Single-Tap Dot + Move Translation
it('P16: Translating single-tap dot updates its origin without distorting circular radius', () => {
  const dot = { id: 'dot-move', type: 'stroke', x: 100, y: 100, strokeWidth: 6, points: [[0, 0, 0.5]], rotat
    ion: 0, opacity: 1, color: '#000', strokeStyle: 'pen', createdAt: 1, updatedAt: 1 };
  const movedDot = { ...dot, x: 200, y: 250 };
  const pathData = strokeModule.buildStrokePath(movedDot, false);
  expect(pathData).toContain('a 3.00,3.00');
});

```

```

});

// Pair 17: Hex Color Picker + Continuous Shape Width Slider
it('P17: Ellipse with custom hex #10b981 and 7px stroke width renders with semi-fill alpha', () => {
  const ellipse = { id: 'el-p17', type: 'ellipse', x: 100, y: 100, width: 120, height: 80, strokeColor: '#10b981', fillColor: '#10b981', fillStyle: 'semi', strokeWidth: 7, dashStyle: 'solid', rotation: 0, opacity: 1, createdAt: 1, updatedAt: 1 };
  expect(ellipse.strokeColor).toBe('#10b981');
  expect(ellipse.strokeWidth).toBe(7);
  expect(ellipse.fillStyle).toBe('semi');
});

// Pair 18: ZoomToFit + Minimap Viewport Synchronization
it('P18: zoomToFit computes bounding box and scales camera to fit viewport with padding', () => {
  const { useWhiteboard } = storeModule;
  useWhiteboard.getState().init();
  const el = { id: 'fit-rect', type: 'rectangle', x: 0, y: 0, width: 800, height: 400, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
  useWhiteboard.getState().addElement(el, true);
  useWhiteboard.getState().zoomToFit();
  const camera = useWhiteboard.getState().camera;
  expect(camera.zoom).toBeGreaterThanOrEqual(0.05);
  expect(camera.zoom).toBeLessThanOrEqual(8.0);
});

// Pair 19: Arrow Element with Stealth Head + Opacity Slider
it('P19: Stealth arrow with 45% opacity generates valid arrowhead polygon', () => {
  const arrowHead = geometryModule.getArrowheadPath(200, 200, 100, 200, 'stealth', 14);
  expect(arrowHead).toBeTruthy();
  expect(arrowHead.path.startsWith('M 200,200')).toBe(true);
  expect(arrowHead.path.toContain('L')).toBe(true);
  expect(arrowHead.path.endsWith('Z')).toBe(true);
});

// Pair 20: Touchscreen 2-Finger Pan + Freehand Brush Inking
it('P20: 2-finger pan translates camera without polluting brush stroke points array', () => {
  const { useWhiteboard } = storeModule;
  const initialCam = { x: 0, y: 0, zoom: 1 };
  useWhiteboard.getState().setCamera(initialCam);
  useWhiteboard.getState().panCamera(40, -30);
  expect(useWhiteboard.getState().camera.x).toBe(-40);
  expect(useWhiteboard.getState().camera.y).toBe(30);
});

// Pair 21: Duplicate Board with WikiLinks + Backlink Aggregation
it('P21: Duplicating a board with WikiLinks registers additional backlinks to target boards', () => {
  const board1 = { id: 'b1', name: 'Original Board', folderId: null, elements: [{ id: 't1', type: 'text', text: 'Links to [[Shared Node]]' }], camera: { x: 0, y: 0, zoom: 1 }, createdAt: 1, updatedAt: 1 };
  const board2 = { id: 'b2', name: 'Duplicated Board', folderId: null, elements: [{ id: 't2', type: 'text', text: 'Links to [[Shared Node]]' }], camera: { x: 0, y: 0, zoom: 1 }, createdAt: 2, updatedAt: 2 };
  const targetBoard = { id: 'b-target', name: 'Shared Node', folderId: null, elements: [], camera: { x: 0, y: 0, zoom: 1 }, createdAt: 3, updatedAt: 3 };

  const backlinks = linksModule.getBacklinks([board1, board2], 'Shared Node', 'b-target', []);
  expect(backlinks.length).toBe(2);
  expect(backlinks.map((b) => b.sourceBoardId)).toContain('b1');
  expect(backlinks.map((b) => b.sourceBoardId)).toContain('b2');
});

// Pair 22: Continuous Pen Slider + 6 Pen Presets
it('P22: every pen in the picker retains its own independent size', () => {
  const { useWhiteboard } = storeModule;
  const st = () => useWhiteboard.getState();

  // Give each offered pen a distinct size, written the way the app writes it.
  const sizes = {};
  typesModule.PEN_TYPES.forEach((tool, i) => {
    const size = 3 + i * 2;
    sizes[tool] = size;
    st().setStyle(typesModule.setToolInk(st().style, tool, { size }));
  });

  // Every one must read back its own value – the previous version only
  // covered the six pre-rework ids, and the four newer pens were silently
  // sharing one size with each other and with shape borders.
  for (const tool of typesModule.PEN_TYPES) {
    expect(typesModule.getToolSize(st().style, tool)).toBe(sizes[tool]);
  }
});

```

```

    }

    // ...and none of it leaked into the shape border width.
    expect(st().style.strokeWidth === sizes[typesModule.PEN_TYPES[0]]).toBe(false);
  });

  // Pair 23: Text In-place Editing + Escape Key Commit
  it('P23: Editing text element in place and setting editingId commits text and exits edit mode', () => {
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().init();
    const textEl = { id: 'txt-edit', type: 'text', x: 0, y: 0, width: 150, height: 40, text: 'Initial', fontSize: 16, fontFamily: 'Inter', fontWeight: 400, align: 'left', color: '#000', rotation: 0, opacity: 1, createdAt: 1, updatedAt: 1 };
    useWhiteboard.getState().addElement(textEl, true);
    useWhiteboard.getState().setEditingId('txt-edit');
    expect(useWhiteboard.getState().editingId).toBe('txt-edit');

    useWhiteboard.getState().updateElement('txt-edit', { text: 'Committed Text' }, true);
    useWhiteboard.getState().setEditingId(null);
    expect(useWhiteboard.getState().editingId).toBeNull();
    const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
    expect(board.elements.find((e) => e.id === 'txt-edit').text).toBe('Committed Text');
  });

  // Pair 24: Service Worker Cache + PWA Manifest Install Prompt
  it('P24: PWA manifest icon assets match pre-cached static asset routes in service worker', () => {
    const manifest = JSON.parse(fs.readFileSync(path.resolve('public/manifest.json'), 'utf-8'));
    const swSrc = fs.readFileSync(path.resolve('public/sw.js'), 'utf-8');
    for (const icon of manifest.icons) {
      expect(swSrc).toContain(icon.src);
    }
  });

  // Pair 25: Distribute Horizontally + Grid Snapping
  it('P25: Distribute Horizontally spaces 5 elements evenly across coordinate grid', () => {
    const elements = [
      { id: '1', x: 0, width: 40 },
      { id: '2', x: 80, width: 40 },
      { id: '3', x: 150, width: 40 },
      { id: '4', x: 300, width: 40 },
      { id: '5', x: 440, width: 40 }
    ];
    const totalWidth = 40 * 5;
    const span = (elements[4].x + elements[4].width) - elements[0].x;
    const gap = (span - totalWidth) / 4;
    expect(gap).toBe(70);
  });

  // Pair 26: Star & Cloud Shapes + Rotation Angle + Anchor Port Recalculation
  it('P26: Rotating Star shape by 90 degrees (PI/2) recalculates anchor ports with rotated normals', () => {
    const star = { id: 'star-rot', type: 'star', x: 100, y: 100, width: 100, height: 100, rotation: Math.PI / 2, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    const ports = geometryModule.getNodeAnchorPorts(star);
    expect(ports.top.nx).toBeCloseTo(1, 4);
    expect(ports.top.ny).toBeCloseTo(0, 4);
  });

  // Pair 27: S Pen Inking + Barrel Button Erase Toggle during Active Motion
  it('P27: S Pen inking transitions to erase on button press and resumes pen on button release mid-stroke', () => {
    const NIB = 1;
    const BARREL = inputModule.STYLUS_BARREL_BUTTON;
    let borrowed = false;

    // Inking
    expect(inputModule.stylusEraseTransition(borrowed, { pointerType: 'pen', buttons: NIB })).toBe('none');

    // Press barrel button while nib is down
    const engage = inputModule.stylusEraseTransition(borrowed, { pointerType: 'pen', buttons: NIB | BARREL });
    expect(engage).toBe('engage');
    borrowed = true;

    // Release barrel button while nib remains on screen
    const release = inputModule.stylusEraseTransition(borrowed, { pointerType: 'pen', buttons: NIB });
    expect(release).toBe('release');
    borrowed = false;
  });

```

```
    expect(borrowed).toBe(false);
  });

// Pair 28: Drawing Pointer ID Isolation + Interleaved Touch Moves from Resting Palm
it('P28: Active stylus stroke maintains integrity when palm touches move and release concurrently', () => {
  const stylusPointerId = 55;
  const palmPointerId = 88;
  const points = [];
  let isDrawing = true;

  const onPointerMove = (e) => {
    if (e.pointerId !== stylusPointerId) return;
    points.push([e.x, e.y, e.pressure]);
  };

  const onPointerUp = (e) => {
    if (e.pointerId === stylusPointerId) {
      isDrawing = false;
    }
  };

  // Pen moves
  onPointerMove({ pointerId: stylusPointerId, x: 100, y: 100, pressure: 0.5 });
  // Palm moves across screen
  onPointerMove({ pointerId: palmPointerId, x: 450, y: 350, pressure: 0.1 });
  // Palm lifts off
  onPointerUp({ pointerId: palmPointerId });
  // Pen moves again
  onPointerMove({ pointerId: stylusPointerId, x: 120, y: 110, pressure: 0.5 });

  expect(isDrawing).toBe(true);
  expect(points.length).toBe(2);
  expect(points[1][0]).toBe(120);

  // Pen lifts
  onPointerUp({ pointerId: stylusPointerId });
  expect(isDrawing).toBe(false);
});

// Pair 29: Draw-and-Hold Auto-Straighten + Micro-Jitter Spatial Deadband vs Intentional Rubber-Banding
it('P29: Hold timer remains engaged under micro-jitter within 5px deadband and rubber-bands on move', () => {
  {
    const DEADBAND = 5.0;
    const anchor = { x: 50, y: 50 };
    const jitter = { x: 52, y: 51 }; // Euclidean dist = sqrt(4 + 1) = 2.23px <= 5px
    const moved = { x: 90, y: 50 }; // Euclidean dist = 40px > 5px

    const distJitter = Math.hypot(jitter.x - anchor.x, jitter.y - anchor.y);
    const distMoved = Math.hypot(moved.x - anchor.x, moved.y - anchor.y);

    expect(distJitter <= DEADBAND).toBe(true);
    expect(distMoved > DEADBAND).toBe(true);

    const initialStroke = [[50, 50, 0.5], [60, 50, 0.5], [70, 50, 0.5], [80, 50, 0.5], [90, 50, 0.5]];
    expect(strokeModule.isStrokeStraight(initialStroke, 0.88)).toBe(true);
    const rubberBanded = strokeModule.straightenStrokePoints([50, 50, 0.5], [150, 50, 0.5], 8);
    expect(rubberBanded.length).toBe(9);
    expect(rubberBanded[8][0]).toBe(150);
  }
});

// Pair 30: Multi-Touch Palm Handoff vs Inking Pointer Protection
it('P30: Secondary capacitive contacts during active pen inking do not trigger canvas pan/zoom', () => {
  const now = 30000;
  const proximity = { penDown: true, lastPenAt: now, now };
  const palmVerdict = palmModule.classifyTouch({ width: 50, height: 35 }, proximity);
  expect(palmVerdict).toBe('pen-down');
  expect(palmModule.isRejectedTouch({ width: 50, height: 35 }, proximity)).toBe(true);
});

=====
// === END FILE: tests/e2e/tier3-pairwise.test.js ===
=====

=====
// === START FILE: tests/e2e/tier4-scenarios.test.js ===
// File 151 of 194 | Lines: 603 | Size: 23107 bytes
```

```

=====

/**
 * Tier 4: Real-World Application Scenarios Test Suite (ProNotes E2E)
 * =====
 * Covers 5 end-to-end multi-step user workflows:
 * 1. Academic Lecture Note-taking (WikiLinks, Stickies, Brush Sketching, Export)
 * 2. Software Architecture Diagramming (Shapes, Orthogonal Smart Connectors, Auto-Routing)
 * 3. Touchscreen Tablet Freehand Math (Stylus Mode, Single-Tap Dots, 2-Finger Pan/Undo)
 * 4. Offline PWA Installation & Theme Switching (Manifest, Dark Mode, Cache Persistence)
 * 5. Multi-Element Layout & Alignment (Lasso, Align Top, Distribute Horizontally, Batch Styling)
 */

const fs = require('fs');
const path = require('path');
const {
  describe,
  it,
  expect,
  beforeEach,
  setupMockDom,
  loadTsModule
} = require('./test-helpers');

describe('Tier 4: Real-World Application Scenarios (5 Comprehensive Workflows)', () => {
  let strokeModule;
  let geometryModule;
  let linksModule;
  let persistenceModule;
  let storeModule;
  let typesModule;
  let inputModule;
  let palmModule;

  beforeEach(() => {
    setupMockDom();
    strokeModule = loadTsModule('src/lib/whiteboard/stroke.ts');
    geometryModule = loadTsModule('src/lib/whiteboard/geometry.ts');
    linksModule = loadTsModule('src/lib/whiteboard/links.ts');
    persistenceModule = loadTsModule('src/lib/whiteboard/persistence.ts');
    storeModule = loadTsModule('src/store/whiteboard.ts');
    typesModule = loadTsModule('src/lib/whiteboard/types.ts');
    inputModule = loadTsModule('src/lib/whiteboard/input.ts');
    palmModule = loadTsModule('src/lib/whiteboard/palm.ts');
  });

  // =====
  // Scenario 1: Academic Lecture Note-taking
  // =====
  it('Scenario 1: Academic Lecture Note-taking with WikiLinks, stickies, brush sketching, and export', () => {
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().init();

    // Step 1: Create "Biology 101" folder
    const folderId = useWhiteboard.getState().createFolder('Biology 101', null, '#10b981');
    expect(folderId).toBeTruthy();

    // Step 2: Create whiteboard "Cell Biology & Membranes" inside folder
    const boardId = useWhiteboard.getState().createBoard('Cell Biology & Membranes', folderId, '#3b82f6');
    useWhiteboard.getState().setActiveBoard(boardId);
    expect(useWhiteboard.getState().activeBoardId).toBe(boardId);

    // Step 3: Insert titled header text with [[WikiLinks]]
    const headerText = '# Cell Biology & Membranes\nSee also [[Cell Membrane]] and [[Mitochondria]] for exam.'
    ;
    const headerEl = {
      id: 'hdr-1',
      type: 'text',
      x: 100,
      y: 80,
      width: 400,
      height: 60,
      text: headerText,
      color: '#0f172a',
      fontSize: 24,
      fontFamily: 'Inter',
      fontWeight: 600,
      align: 'left',
    }
  });
}

```

```
rotation: 0,
opacity: 1,
createdAt: Date.now(),
updatedAt: Date.now()
};
useWhiteboard.getState().addElement(headerEl, true);

// Step 4: Sketch a phospholipid bilayer using Brush pen with 4px stroke width
const brushStroke = {
  id: 'sketch-brush',
  type: 'stroke',
  x: 100,
  y: 180,
  strokeWidth: 4,
  strokeStyle: 'brush',
  color: '#3b82f6',
  points: [
    [0, 0, 0.4],
    [50, 10, 0.7],
    [100, 5, 0.8],
    [150, 20, 0.6],
    [200, 10, 0.4]
  ],
  rotation: 0,
  opacity: 1,
  createdAt: Date.now(),
  updatedAt: Date.now()
};
const brushPath = strokeModule.buildStrokePath(brushStroke, false);
expect(brushPath.startsWith('M')).toBe(true);
expect(brushPath.endsWith('Z')).toBe(true);
useWhiteboard.getState().addElement(brushStroke, true);

// Step 5: Add yellow Sticky Note #notes
const yellowSticky = {
  id: 'stk-notes',
  type: 'sticky',
  x: 100,
  y: 260,
  width: 200,
  height: 180,
  text: 'Hydrophilic heads outside, hydrophobic tails inside. #notes',
  color: 'yellow',
  fontSize: 16,
  align: 'left',
  rotation: 0,
  opacity: 1,
  createdAt: Date.now(),
  updatedAt: Date.now()
};
useWhiteboard.getState().addElement(yellowSticky, true);

// Step 6: Add pink Sticky Note #exam-tip
const pinkSticky = {
  id: 'stk-exam',
  type: 'sticky',
  x: 320,
  y: 260,
  width: 200,
  height: 180,
  text: 'Fluid mosaic model proposed by Singer & Nicolson in 1972. #exam-tip',
  color: 'pink',
  fontSize: 16,
  align: 'left',
  rotation: 0,
  opacity: 1,
  createdAt: Date.now(),
  updatedAt: Date.now()
};
useWhiteboard.getState().addElement(pinkSticky, true);

// Step 7: Verify WikiLinks & tags are parsed into backlinks
const activeBoard = useWhiteboard.getState().boards.find((b) => b.id === boardId);
const boardLinks = linksModule.getBoardLinks(activeBoard);
expect(boardLinks).toContain('cell membrane');
expect(boardLinks).toContain('mitochondria');

const boardTags = linksModule.getBoardTags(activeBoard);
```

```
expect(boardTags).toContain('notes');
expect(boardTags).toContain('exam-tip');

// Step 8: Calculate union bounding box for Retina PNG / SVG Export
const union = geometryModule.getUnionBounds(activeBoard.elements);
expect(union).toBeTruthy();
expect(union.x).toBeLessThanOrEqual(100);
expect(union.width).toBeGreaterThanOrEqual(400);
expect(union.height).toBeGreaterThanOrEqual(300);
});

// =====
// Scenario 2: Software System Architecture Diagramming
// =====
it('Scenario 2: Software System Architecture Diagramming with microservice shapes, smart orthogonal connectors, and auto-routing', () => {
  const { useWhiteboard } = storeModule;
  useWhiteboard.getState().init();

  // Step 1: Create board "E-Commerce Microservices"
  const boardId = useWhiteboard.getState().createBoard('E-Commerce Microservices', null, '#6366f1');
  useWhiteboard.getState().setActiveBoard(boardId);

  // Step 2: Create 4 microservice rectangle shapes
  const services = [
    { id: 'svc-gw', name: 'API Gateway', x: 100, y: 200, width: 140, height: 70 },
    { id: 'svc-auth', name: 'Auth Service', x: 350, y: 100, width: 140, height: 70 },
    { id: 'svc-order', name: 'Order Service', x: 350, y: 300, width: 140, height: 70 },
    { id: 'svc-pay', name: 'Payment Service', x: 600, y: 300, width: 140, height: 70 }
  ];

  for (const svc of services) {
    const rectEl = {
      id: svc.id,
      type: 'rectangle',
      x: svc.x,
      y: svc.y,
      width: svc.width,
      height: svc.height,
      strokeColor: '#3b82f6',
      fillColor: '#eff6ff',
      fillStyle: 'semi',
      dashStyle: 'solid',
      strokeWidth: 2,
      rotation: 0,
      opacity: 1,
      createdAt: Date.now(),
      updatedAt: Date.now()
    };
    useWhiteboard.getState().addElement(rectEl, true);
  }

  // Step 3: Create 1 Database Diamond shape "PostgreSQL Cluster"
  const dbEl = {
    id: 'db-postgres',
    type: 'diamond',
    x: 370,
    y: 450,
    width: 100,
    height: 90,
    strokeColor: '#059669',
    fillColor: '#ecfdf5',
    fillStyle: 'semi',
    dashStyle: 'solid',
    strokeWidth: 2,
    rotation: 0,
    opacity: 1,
    createdAt: Date.now(),
    updatedAt: Date.now()
  };
  useWhiteboard.getState().addElement(dbEl, true);

  // Step 4: Connect API Gateway -> Auth Service (Orthogonal)
  const gwNode = services[0];
  const authNode = services[1];
  const anchorsGwAuth = geometryModule.getOptimalConnectionAnchors(
    { ...gwNode, type: 'rectangle', rotation: 0 },
    { ...authNode, type: 'rectangle', rotation: 0 }
  );
```

```

    );
    expect(anchorsGwAuth.source.anchor).toBe('right');
    const pathGwAuth = geometryModule.getOrthogonalPath(anchorsGwAuth.source, anchorsGwAuth.target);
    expect(pathGwAuth.startsWith('M')).toBe(true);

    // Step 5: Connect API Gateway -> Order Service (Orthogonal)
    const orderNode = services[2];
    const anchorsGwOrder = geometryModule.getOptimalConnectionAnchors(
      { ...gwNode, type: 'rectangle', rotation: 0 },
      { ...orderNode, type: 'rectangle', rotation: 0 }
    );
    expect(anchorsGwOrder.source.anchor).toBe('right');
    expect(anchorsGwOrder.target.anchor).toBe('left');

    // Step 6: Connect Order Service -> Payment Service (Curved)
    const payNode = services[3];
    const anchorsOrderPay = geometryModule.getOptimalConnectionAnchors(
      { ...orderNode, type: 'rectangle', rotation: 0 },
      { ...payNode, type: 'rectangle', rotation: 0 }
    );
    const pathOrderPay = geometryModule.getCurvedBezierPath(anchorsOrderPay.source, anchorsOrderPay.target);
    expect(pathOrderPay.startsWith('M 490,335 C')).toBe(true);

    // Step 7: Connect Order Service -> Database (Orthogonal)
    const anchorsOrderDb = geometryModule.getOptimalConnectionAnchors(
      { ...orderNode, type: 'rectangle', rotation: 0 },
      dbEl
    );
    expect(anchorsOrderDb.source.anchor).toBe('bottom');
    expect(anchorsOrderDb.target.anchor).toBe('top');

    // Step 8: Move Order Service to (350, 320) and verify auto-routing updates
    const movedOrderNode = { ...orderNode, y: 320 };
    const updatedAnchors = geometryModule.getOptimalConnectionAnchors(
      { ...gwNode, type: 'rectangle', rotation: 0 },
      { ...movedOrderNode, type: 'rectangle', rotation: 0 }
    );
    expect(updatedAnchors.target.y).toBe(355);
  });

  // =====
  // Scenario 3: Touchscreen Tablet Freehand Math Session
  // =====
  it('Scenario 3: Touchscreen Tablet Freehand Math Session with stylus-only mode, single-tap decimal dots, 2-finger pan/zoom, and 2-finger undo', () => {
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().init();

    // Step 1: Enable Stylus Only mode with palm rejection
    useWhiteboard.getState().setTouchSettings({
      mode: 'stylus-only',
      palmRejection: true,
      twoFingerUndo: true,
      twoFingerPan: true,
      pinchZoom: true
    });
    const touchSettings = useWhiteboard.getState().touchSettings;
    expect(touchSettings.mode).toBe('stylus-only');
    expect(touchSettings.palmRejection).toBe(true);

    // Step 2: Write math formula with Pencil tool (e.g. integral stroke)
    const pencilStroke = {
      id: 'math-integral',
      type: 'stroke',
      x: 100,
      y: 200,
      strokeWidth: 3,
      strokeStyle: 'pencil',
      color: '#000000',
      points: [
        [0, 50, 0.5],
        [10, 20, 0.6],
        [15, 0, 0.7],
        [12, -20, 0.5],
        [5, -30, 0.4]
      ],
      rotation: 0,
      opacity: 1,
    };
  });

```

```

    createdAt: Date.now(),
    updatedAt: Date.now()
  };
  const integralPath = strokeModule.buildStrokePath(pencilStroke, false);
  expect(integralPath.startsWith('M')).toBe(true);
  useWhiteboard.getState().addElement(pencilStroke, true);

  // Step 3: Ink decimal dots (e.g. "3 . 1 4") with single-tap precision
  const decimalDot = {
    id: 'math-dot-pi',
    type: 'stroke',
    x: 180,
    y: 230,
    strokeWidth: 5,
    strokeStyle: 'pen',
    color: '#000000',
    points: [[0, 0, 0.6]],
    rotation: 0,
    opacity: 1,
    createdAt: Date.now(),
    updatedAt: Date.now()
  };
  const dotPath = strokeModule.buildStrokePath(decimalDot, false);
  expect(dotPath).toContain('a 2.50,2.50');
  useWhiteboard.getState().addElement(decimalDot, true);

  // Step 4: Perform 2-finger pan to navigate right (simulate panCamera)
  const initialCam = useWhiteboard.getState().camera;
  useWhiteboard.getState().panCamera(150, 0);
  expect(useWhiteboard.getState().camera.x).toBe(initialCam.x - 150);

  // Step 5: Perform pinch-to-zoom to inspect handwriting
  useWhiteboard.getState().zoomTo(1.5, 600, 400);
  expect(useWhiteboard.getState().camera.zoom).toBe(1.5);

  // Step 6: Trigger 2-finger tap undo to undo decimal dot mistake
  expect(useWhiteboard.getState().canUndo()).toBe(true);
  useWhiteboard.getState().undo();

  // Step 7: Verify undo removed decimal dot while maintaining integral stroke
  const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
  expect(board.elements.some((e) => e.id === 'math-dot-pi')).toBe(false);
  expect(board.elements.some((e) => e.id === 'math-integral')).toBe(true);
});

// =====
// Scenario 4: Offline PWA Installation & Theme Switching
// =====
it('Scenario 4: Offline PWA Installation & Theme Switching with manifest validation, dark mode switch, and c
  ache verification', () => {
    // Step 1: Validate Web App Manifest
    const manifestPath = path.resolve('public/manifest.json');
    expect(fs.existsSync(manifestPath)).toBe(true);
    const manifest = JSON.parse(fs.readFileSync(manifestPath, 'utf-8'));
    expect(manifest.name).toContain('ProNotes');
    expect(manifest.display).toBe('standalone');
    expect(manifest.icons.length).toBeGreaterThanOrEqual(4);

    // Step 2: Validate Service Worker cache manifest
    const swPath = path.resolve('public/sw.js');
    expect(fs.existsSync(swPath)).toBe(true);
    const swSrc = fs.readFileSync(swPath, 'utf-8');
    expect(swSrc).toContain('STATIC_ASSETS');

    // Step 3: Switch theme to dark mode and verify UI styles
    const darkBg = '#0f172a';
    const darkFg = '#f8fafc';
    expect(darkBg).toBe('#0f172a');
    expect(darkFg).toBe('#f8fafc');

    // Step 4: Create dark theme diagrams in Zustand store
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().init();
    const darkNode = {
      id: 'dark-node-1',
      type: 'rectangle',
      x: 200,

```

```

    y: 200,
    width: 150,
    height: 100,
    strokeColor: '#818cf8',
    fillColor: '#1e1b4b',
    fillStyle: 'solid',
    dashStyle: 'solid',
    strokeWidth: 2,
    rotation: 0,
    opacity: 1,
    createdAt: Date.now(),
    updatedAt: Date.now()
  });
  useWhiteboard.getState().addElement(darkNode, true);

  // Step 5: Flush save to localStorage
  persistenceModule.flushSave(() => ({
    version: 1,
    folders: useWhiteboard.getState().folders,
    boards: useWhiteboard.getState().boards,
    activeBoardId: useWhiteboard.getState().activeBoardId,
    activeFolderId: useWhiteboard.getState().activeFolderId
  }));

  // Step 6: Simulate offline browser reload by loading state from localStorage
  const restoredState = persistenceModule.loadState();
  expect(restoredState).toBeTruthy();
  const activeBoard = restoredState.boards.find((b) => b.id === restoredState.activeBoardId);
  expect(activeBoard.elements.some((e) => e.id === 'dark-node-1')).toBe(true);
});

// =====
// Scenario 5: Multi-Element Layout & Alignment
// =====
it('Scenario 5: Multi-Element Layout & Alignment with lasso multi-selection, top alignment, horizontal distribution, hex color batch updates, and opacity slider', () => {
  const { useWhiteboard } = storeModule;
  useWhiteboard.getState().init();

  // Step 1: Create 8 scattered sticky notes
  const scatteredNotes = [];
  for (let i = 0; i < 8; i++) {
    scatteredNotes.push({
      id: `note-${i}`,
      type: 'sticky',
      x: i * 70 + (i % 2 === 0 ? 15 : -10),
      y: 100 + (i * 25) % 80,
      width: 120,
      height: 120,
      text: `Idea #${i + 1}`,
      color: 'yellow',
      fontSize: 14,
      align: 'left',
      rotation: 0,
      opacity: 1,
      createdAt: Date.now(),
      updatedAt: Date.now()
    });
  }
  useWhiteboard.getState().addElements(scatteredNotes, true);

  // Step 2: Lasso select all 8 notes
  const noteIds = scatteredNotes.map((n) => n.id);
  useWhiteboard.getState().setSelected(noteIds);
  expect(useWhiteboard.getState().selectedIds.length).toBe(8);

  // Step 3: Execute "Align Top" (align all notes to minY)
  const minY = Math.min(...scatteredNotes.map((n) => n.y));
  useWhiteboard.getState().updateElements(noteIds, { y: minY }, true);

  let board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
  const topAligned = board.elements.filter((e) => noteIds.includes(e.id));
  expect(topAligned.every((e) => e.y === minY)).toBe(true);

  // Step 4: Execute "Distribute Horizontally" (even gaps across 8 notes)
  const noteWidth = 120;
  const startX = 50;
  const gap = 30;

```

```

noteIds.forEach((id, idx) => {
  useWhiteboard.getState().updateElement(id, { x: startX + idx * (noteWidth + gap) }, false);
});

board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
const distributed = board.elements.filter((e) => noteIds.includes(e.id));
expect(distributed[0].x).toBe(50);
expect(distributed[1].x).toBe(50 + 120 + 30); // 200
expect(distributed[7].x).toBe(50 + 7 * 150); // 1100

// Step 5: Batch update color to sky blue (color: "blue") and opacity to 80% (0.80)
useWhiteboard.getState().updateElements(noteIds, { color: 'blue', opacity: 0.8 }, true);

board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
const styledNotes = board.elements.filter((e) => noteIds.includes(e.id));
expect(styledNotes.every((e) => e.color === 'blue')).toBe(true);
expect(styledNotes.every((e) => e.opacity === 0.8)).toBe(true);

// Step 6: Union bounds wrap the aligned and distributed row
const finalUnion = geometryModule.getUnionBounds(styledNotes);
expect(finalUnion.x).toBe(50);
expect(finalUnion.y).toBe(minY);
expect(finalUnion.width).toBe(1100 + 120 - 50); // 1170
expect(finalUnion.height).toBe(120);
});

// =====
// Scenario 6: High-Frequency S Pen Digital Inking, Cursive Note-Taking & Precision Gesture Flow
// =====
it('Scenario 6: High-Frequency S Pen Cursive Note-Taking, Barrel Button Erasing, Palm Suppression, and Auto-
  Straightening', () => {
  const { useWhiteboard } = storeModule;
  useWhiteboard.getState().init();

  // Step 1: Open a new whiteboard for inking
  const boardId = useWhiteboard.getState().createBoard('S Pen Lecture Notes', null, '#ffffff');
  useWhiteboard.getState().setActiveBoard(boardId);
  expect(useWhiteboard.getState().activeBoardId).toBe(boardId);

  // Step 2: Select Fountain Pen with chisel nib
  useWhiteboard.getState().setTool('fountain');
  expect(useWhiteboard.getState().tool).toBe('fountain');

  // Step 3: Stream high-frequency 240Hz coalesced cursive handwriting
  const cursiveCoalescedStream = [
    { x: 100, y: 150, p: 0.3, t: 1000 },
    { x: 103, y: 145, p: 0.4, t: 1004 },
    { x: 108, y: 138, p: 0.6, t: 1008 },
    { x: 115, y: 135, p: 0.7, t: 1012 },
    { x: 122, y: 140, p: 0.5, t: 1016 },
    { x: 126, y: 148, p: 0.4, t: 1020 },
    { x: 130, y: 155, p: 0.3, t: 1024 }
  ];
  const cursivePoints = cursiveCoalescedStream.map(pt => [pt.x, pt.y, strokeModule.calibratePressure(pt.p, 6
    5, 'standard')]);
  const cursiveStroke = {
    id: 'spen-cursive-1',
    type: 'stroke',
    x: 0,
    y: 0,
    rotation: 0,
    opacity: 1,
    color: '#1e293b',
    strokeWidth: 4,
    strokeStyle: 'fountain',
    points: cursivePoints,
    createdAt: 1,
    updatedAt: 1
  };
  useWhiteboard.getState().addElement(cursiveStroke, true);

  let board = useWhiteboard.getState().boards.find((b) => b.id === boardId);
  expect(board.elements.some(e => e.id === 'spen-cursive-1')).toBe(true);

  // Step 4: Palm contact lands while writing (classified as rejected palm)
  const palmTouch = { width: 55, height: 40 };
  const now = 10030;
  const penProx = { penDown: true, lastPenAt: now, now };

```

```
const verdict = palmModule.classifyTouch(palmTouch, penProx);
expect(verdict).toBe('pen-down');
expect(palmModule.isRejectedTouch(palmTouch, penProx)).toBe(true);

// Step 5: S Pen barrel button pressed mid-stroke to erase mistake, then released
const NIB = 1;
const BARREL = inputModule.STYLUS_BARREL_BUTTON;
let borrowed = false;

// Button engages eraser
const actEngage = inputModule.stylusEraseTransition(borrowed, { pointerType: 'pen', buttons: NIB | BARREL
});
expect(actEngage).toBe('engage');
borrowed = true;

// Erase the erroneous stroke
useWhiteboard.getState().deleteElements(['spen-cursive-1']);
board = useWhiteboard.getState().boards.find((b) => b.id === boardId);
expect(board.elements.some(e => e.id === 'spen-cursive-1')).toBe(false);

// Button released mid-stroke with nib down -> restores pen
const actRelease = inputModule.stylusEraseTransition(borrowed, { pointerType: 'pen', buttons: NIB });
expect(actRelease).toBe('release');
borrowed = false;
expect(borrowed).toBe(false);

// Step 6: Draw-and-hold auto-straightening for an underline rule
const underlineRaw = [
  [100, 200, 0.5],
  [150, 201, 0.5],
  [200, 199, 0.5],
  [250, 200, 0.5],
  [300, 200, 0.5]
];
expect(strokeModule.isStrokeStraight(underlineRaw, 0.88)).toBe(true);
const straightened = strokeModule.straightenStrokePoints(underlineRaw, 8);
expect(straightened.length).toBe(9);
expect(straightened[0][0]).toBe(100);
expect(straightened[8][0]).toBe(300);

const ruleStroke = {
  id: 'spen-rule-1',
  type: 'stroke',
  x: 0,
  y: 0,
  rotation: 0,
  opacity: 1,
  color: '#3b82f6',
  strokeWidth: 3,
  strokeStyle: 'fineliner',
  points: straightened,
  createdAt: 2,
  updatedAt: 2
};
useWhiteboard.getState().addElement(ruleStroke, true);

board = useWhiteboard.getState().boards.find((b) => b.id === boardId);
expect(board.elements.some(e => e.id === 'spen-rule-1')).toBe(true);

// Step 7: Switch to highlighter for title emphasis
useWhiteboard.getState().setTool('highlighter');
expect(useWhiteboard.getState().tool).toBe('highlighter');
const hiOpts = strokeModule.getStrokeOptionsForStyle('highlighter', 10);
expect(hiOpts.size).toBe(30);
expect(typesModule.PEN_TYPE_PRESETS.highlighter.mixBlendMode).toBe('multiply');
expect(typesModule.PEN_TYPE_PRESETS.highlighter.cap).toBe('flat');
});
});

=====
// === END FILE: tests/e2e/tier4-scenarios.test.js ===
=====

=====
// === START FILE: tests/eraser_settle.test.ts ===
// File 152 of 194 | Lines: 104 | Size: 3596 bytes
=====
```

```

/**
 * Pixel eraser: the ink beside a cut must not change shape.
 *
 * A stroke is drawn from its raw points after smoothing that depends on the
 * points before each one. Cutting a stroke and letting each piece be smoothed
 * again from its new start bent the writing beside every cut - measured on a
 * zig-zag line, the piece after the cut came out 20px tall instead of 28. The
 * eraser now cuts the drawn line itself (`settledStrokePoints`) and the pieces
 * are drawn as they are.
 */

import assert from "node:assert/strict";
import { buildStrokePath, eraseFromStroke, settledStrokePoints } from "../src/lib/whiteboard/stroke";
import type { StrokeElement } from "../src/lib/whiteboard/types";

const NL = String.fromCharCode(10);
let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`${(err as Error).message}`);
  }
}

console.log(NL + "■■ Pixel eraser keeps the ink in place ■■");

/** Handwriting-like zig-zag: sharp turns every few points. */
function zigzag(): StrokeElement {
  const points: [number, number, number][] = [];
  for (let t = 0; t <= 60; t++) {
    const k = t % 6;
    points.push([t * 8 + (k < 3 ? 0 : -6), k < 3 ? -k * 12 : (k - 6) * -12 - 36, 0.5]);
  }
  return {
    id: "s",
    type: "stroke",
    x: 0,
    y: 0,
    rotation: 0,
    opacity: 1,
    points,
    color: "#000",
    strokeWidth: 3,
    strokeStyle: "gel",
    createdAt: 0,
    updatedAt: 0,
  };
}

/** Bounding box of a path's coordinates, optionally only within [x0, x1]. */
function box(d: string, x0 = -Infinity, x1 = Infinity) {
  const nums = (d.match(/-?\d+(\.\d+)?/g) ?? []).map(Number);
  let minY = Infinity;
  let maxY = -Infinity;
  let minX = Infinity;
  let maxX = -Infinity;
  for (let i = 0; i + 1 < nums.length; i += 2) {
    const [x, y] = [nums[i], nums[i + 1]];
    if (x < x0 || x > x1) continue;
    minX = Math.min(minX, x);
    maxX = Math.max(maxX, x);
    minY = Math.min(minY, y);
    maxY = Math.max(maxY, y);
  }
  return { minX, maxX, minY, maxY, h: maxY - minY };
}

test("a settled stroke draws the same line as the original", () => {
  const s = zigzag();
  const settled: StrokeElement = { ...s, points: settledStrokePoints(s), settled: true };
  const a = box(buildStrokePath(s));

```

```

const b = box(buildStrokePath(settled));
for (const k of ["minX", "maxX", "minY", "maxY"] as const) {
  assert.ok(Math.abs(a[k] - b[k]) < 1.5, `${k}: ${a[k]} vs ${b[k]}`);
}
});

test("the piece after a cut keeps the height of the writing it came from", () => {
  const s = zigzag();
  const pts = settledStrokePoints(s);
  // A vertical swipe through the middle.
  const runs = eraseFromStroke(pts, 240, -80, 240, 60, 10);
  assert.ok(runs && runs.length === 2, "cut in two");
  const right = runs![1];
  const piece: StrokeElement = { ...s, points: right, settled: true, cutStart: true };
  const whole = box(buildStrokePath(s), 300, 460);
  const part = box(buildStrokePath(piece), 300, 460);
  assert.ok(Math.abs(whole.h - part.h) < 2, `height ${whole.h.toFixed(1)} vs ${part.h.toFixed(1)}`);
  assert.ok(Math.abs(whole.minY - part.minY) < 2 && Math.abs(whole.maxY - part.maxY) < 2, "same place");
});

test("settling is done once: a settled stroke's points are kept as they are", () => {
  const s = { ...zigzag(), settled: true };
  assert.equal(settledStrokePoints(s), s.points);
});

console.log(` ${passed} passed, ${failed} failed`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/eraser_settle.test.ts ===
=====

=====
// === START FILE: tests/export_geometry.test.ts ===
// File 153 of 194 | Lines: 199 | Size: 7814 bytes
=====

/**
 * Where exported ink actually LANDS.
 *
 * The bug this pins lost people's handwriting. A stroke stores its points
 * RELATIVE to the element's own origin - the on-screen renderer wraps every
 * element in `translate(x, y)` and draws the path inside it. The exporter
 * emitted the same path with NO translate, so every stroke on every page
 * collapsed into the top-left corner of the sheet and was clipped away.
 *
 * On screen the notebook looked perfect. The exported PDF came out with the
 * text and sticky notes in place and the handwriting simply gone. It survived
 * review because the test fixtures had been written with ABSOLUTE points,
 * which real strokes never have - so the fixtures below deliberately use the
 * real convention, taken from a device's stored notebook:
 *
 * stroke.x = 252, stroke.y = 159, points = [[-26, -7], [10, 28], ...]
 *
 * The general rule these tests defend: an element's exported geometry must sit
 * where `getElementBounds` says it sits. Anything else means the file does not
 * match the screen.
 */

import assert from "node:assert/strict";

import { buildExportSvg } from "../src/lib/whiteboard/export";
import { getElementBounds } from "../src/lib/whiteboard/geometry";
import type { Board, NotebookPage, StrokeElement, WhiteboardElement } from "../src/lib/whiteboard/types";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`${(err as Error).message}`);
  }
}

```

```

}

const A4 = { w: 794, h: 1123 };

/** A stroke the way the app really stores one: points around a local origin. */
function realStroke(x: number, y: number, extra: Partial<StrokeElement> = {}): StrokeElement {
  return {
    id: `s_${x}_${y}`,
    type: "stroke",
    x,
    y,
    width: 40,
    height: 35,
    rotation: 0,
    opacity: 1,
    points: [
      [-26, -7, 0.5],
      [-10, 12, 0.6],
      [10, 28, 0.6],
      [14, 5, 0.5],
    ],
    color: "#111111",
    strokeWidth: 3,
    strokeStyle: "pen",
    createdAt: 1,
    updatedAt: 1,
    ...extra,
  } as StrokeElement;
}

function page(elements: WhiteboardElement[]): NotebookPage {
  return { id: "pg1", elements, createdAt: 1, updatedAt: 1 };
}

function notebook(elements: WhiteboardElement[]): Board {
  return {
    id: "b1",
    name: "N",
    folderId: null,
    elements: [],
    camera: { x: 0, y: 0, zoom: 1 },
    kind: "notebook",
    pages: [page(elements)],
    activePageId: "pg1",
    pageSize: "a4",
    createdAt: 1,
    updatedAt: 1,
  } as Board;
}

/** Every number inside the `d` of the stroke's path. */
function pathNumbers(svg: string): number[] {
  const m = /<g transform="translate\[^\]*\"><path d="\[^\]*"/.exec(svg);
  if (!m) return [];
  return (m[1].match(/-?\d+(\.\d+)?/g) ?? []).map(Number);
}

console.log(`\n███ Exported strokes land where the screen puts them ███`);

test("a stroke is placed by a translate, not drawn at the origin", () => {
  const svg = buildExportSvg(notebook([realStroke(252, 159)]));
  assert.ok(
    svg.includes('transform="translate(252, 159)"'),
    "the stroke carries the same translate the renderer applies"
  );
});

test("the path itself stays in LOCAL coordinates", () => {
  // If someone "fixes" this by baking x/y into the points instead, the
  // translate would double the offset. The path must stay small and local.
  const nums = pathNumbers(buildExportSvg(notebook([realStroke(252, 159)]));
  assert.ok(nums.length > 0, "the stroke produced a path");
  const maxAbs = Math.max(...nums.map(Math.abs));
  assert.ok(maxAbs < 120, `path coordinates should be local, saw max |${maxAbs}|`);
});

test("ink does NOT collapse into the top-left corner", () => {
  // The exact failure users saw: every stroke stacked in the corner, clipped.

```

```

const svg = buildExportSvg(notebook([realStroke(252, 159), realStroke(600, 900)]).svg;
const translates = [...svg.matchAll(/translate\((\d+(?:\.\d+)?), (\d+(?:\.\d+)?))\)/g)]
  .map((m) => ({ x: +m[1], y: +m[2] }));
  .filter((t) => !(t.x === 0)); // sheet groups translate(0, offset)
assert.equal(translates.length, 2, "both strokes are placed");
assert.ok(
  translates.some((t) => t.x === 600 && t.y === 900),
  "the far stroke keeps its own position"
);
});

test("the placed stroke matches getElementBounds", () => {
  // The invariant that generalises past strokes: what the exporter draws has
  // to occupy the same box the rest of the app thinks the element occupies.
  const s = realStroke(252, 159);
  const bounds = getElementBounds(s);
  const svg = buildExportSvg(notebook([s])).svg;
  const m = /translate\((\d+(?:\.\d+)?), (\d+(?:\.\d+)?))"><path d="([^\"]*)"\/.exec(svg);
  assert.ok(m, "found the placed stroke");
  const tx = +m[1], ty = +m[2];
  const nums = (m[3].match(/-?\d+(\.\d+)?/g) ?? []).map(Number);
  let minX = Infinity, minY = Infinity, maxX = -Infinity, maxY = -Infinity;
  for (let i = 0; i + 1 < nums.length; i += 2) {
    const x = nums[i] + tx, y = nums[i + 1] + ty;
    if (x < minX) minX = x; if (x > maxX) maxX = x;
    if (y < minY) minY = y; if (y > maxY) maxY = y;
  }
  // Stroke outlines are fattened by width, so allow generous slack; the point
  // is that it sits at ~(252,159), not at ~(0,0).
  assert.ok(Math.abs(minX - bounds.x) < 60, `x ${minX} vs bounds ${bounds.x}`);
  assert.ok(Math.abs(minY - bounds.y) < 60, `y ${minY} vs bounds ${bounds.y}`);
});

test("strokes stay ON the sheet", () => {
  const svg = buildExportSvg(notebook([realStroke(252, 159), realStroke(400, 700)]).svg;
  for (const m of svg.matchAll(/translate\((\d+(?:\.\d+)?), (\d+(?:\.\d+)?))"><path/g)) {
    const x = +m[1], y = +m[2];
    assert.ok(x > 0 && x < A4.w, `stroke x ${x} is on the sheet`);
    assert.ok(y > 0 && y < A4.h, `stroke y ${y} is on the sheet`);
  }
});

console.log("\n■■ The stale path cache is never trusted ■■");

test("a legacy cached `path` is ignored in favour of the points", () => {
  // `path` froze a stroke's geometry at draw time and is stripped on load.
  // The renderer always rebuilds from points; the exporter must agree, or a
  // file can disagree with the screen.
  const s = realStroke(252, 159, { path: "M 9999,9999 L 9998,9998 Z" } as Partial<StrokeElement>);
  const svg = buildExportSvg(notebook([s])).svg;
  assert.ok(!svg.includes("9999"), "the frozen path was not used");
  assert.ok(svg.includes('translate(252, 159)'), "the stroke is still placed correctly");
});

console.log("\n■■ Other element types keep their absolute coordinates ■■");

test("text is written at its own x/y, not translated twice", () => {
  const text = {
    id: "t1", type: "text", x: 300, y: 500, width: 200, height: 40, rotation: 0, opacity: 1,
    text: "Hello", color: "#000", fontSize: 20, fontFamily: "Inter", fontWeight: 400,
    align: "left", createdAt: 1, updatedAt: 1,
  } as unknown as WhiteboardElement;
  const svg = buildExportSvg(notebook([text])).svg;
  const m = /<text x="([\d.]+)"\/.exec(svg);
  assert.ok(m, "the text was exported");
  const x = +m[1];
  assert.ok(Math.abs(x - 300) < 20, `text x ${x} should be ~300, not re-offset`);
});

test("a line spans from its origin to its far end", () => {
  const line = {
    id: "l1", type: "line", x: 100, y: 200, x2: 150, y2: 60, width: 150, height: 60,
    rotation: 0, opacity: 1, strokeColor: "#000", strokeWidth: 2, dashStyle: "solid",
    createdAt: 1, updatedAt: 1,
  } as unknown as WhiteboardElement;
  const svg = buildExportSvg(notebook([line])).svg;
  assert.ok(/x1="100" y1="200" x2="150" y2="60"\/.test(svg), "line endpoints are absolute");
});

```

```
console.log(`\n ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);
```

```
=====
// === END FILE: tests/export_geometry.test.ts ===
=====
```

```
=====
// === START FILE: tests/fixtures/shape_user_strokes.json ===
// File 154 of 194 | Lines: 1 | Size: 48172 bytes
=====
```

```
{
  "note": "Strokes drawn by hand on the laptop (2026-09-30) that the first version missed; 'none' = must stay as drawn (an S, a C, a question-mark curve, a V, an arc).",
  "strokes": [
    {
      "id": "cr0frZ0",
      "expect": "ellipse",
      "points": [
        [0, 0, 0.5], [0.0, 3.9, 0.5], [-1.1, 12.2, 0.5], [-2.8, 23.9, 0.5], [-5.5, 34.4, 0.5], [-8.3, 43.9, 0.5], [-11.1, 51.7, 0.5], [-13.3, 59.4, 0.5], [-15.5, 66.1, 0.5], [-17.8, 72.8, 0.5], [-20.5, 79.4, 0.5], [-22.8, 86.1, 0.5], [-26.1, 92.8, 0.5], [-29.4, 100.0, 0.5], [-33.3, 106.1, 0.5], [-37.8, 112.8, 0.5], [-43.9, 119.4, 0.5], [-50.5, 126.7, 0.5], [-57.2, 133.9, 0.5], [-65.0, 140.6, 0.5], [-72.8, 146.7, 0.5], [-81.1, 152.2, 0.5], [-89.4, 156.7, 0.5], [-98.3, 159.4, 0.5], [-107.2, 161.7, 0.5], [-116.1, 163.3, 0.5], [-125.0, 163.9, 0.5], [-133.9, 163.9, 0.5], [-142.2, 163.3, 0.5], [-151.1, 162.2, 0.5], [-159.4, 160.0, 0.5], [-167.2, 155.6, 0.5], [-175.0, 151.1, 0.5], [-182.2, 146.7, 0.5], [-188.9, 141.7, 0.5], [-195.0, 135.0, 0.5], [-200.5, 127.2, 0.5], [-206.7, 120.0, 0.5], [-211.7, 112.2, 0.5], [-215.5, 105.0, 0.5], [-219.4, 97.2, 0.5], [-231.1, 76.1, 0.5], [-236.7, 66.7, 0.5], [-239.4, 59.4, 0.5], [-241.1, 48.9, 0.5], [-242.8, 40.6, 0.5], [-243.9, 32.2, 0.5], [-244.4, 21.7, 0.5], [-244.4, 13.9, 0.5], [-244.4, 3.9, 0.5], [-243.3, -5.6, 0.5], [-242.2, -12.2, 0.5], [-240.5, -21.7, 0.5], [-238.3, -30.0, 0.5], [-235.5, -35.6, 0.5], [-231.7, -40.6, 0.5], [-227.2, -48.9, 0.5], [-223.3, -55.0, 0.5], [-218.9, -57.8, 0.5], [-213.3, -61.1, 0.5], [-207.8, -64.4, 0.5], [-200.5, -68.9, 0.5], [-194.4, -71.7, 0.5], [-187.8, -73.9, 0.5], [-180.0, -75.6, 0.5], [-172.8, -76.1, 0.5], [-165.0, -76.1, 0.5], [-155.5, -75.6, 0.5], [-147.8, -73.3, 0.5], [-139.4, -70.0, 0.5], [-130.0, -66.1, 0.5], [-121.7, -61.7, 0.5], [-112.8, -57.8, 0.5], [-103.9, -52.8, 0.5], [-96.7, -43.9, 0.5], [-88.3, -36.1, 0.5], [-80.5, -29.4, 0.5], [-73.3, -19.4, 0.5], [-65.5, -11.1, 0.5], [-58.3, -2.8, 0.5], [-52.8, 8.3, 0.5], [-47.2, 17.2, 0.5], [-41.7, 28.3, 0.5], [-36.7, 37.8, 0.5], [-32.2, 46.7, 0.5], [-28.3, 57.8, 0.5], [-25.0, 65.6, 0.5], [-22.2, 73.9, 0.5], [-19.4, 82.2, 0.5], [-17.2, 88.3, 0.5], [-16.1, 91.1, 0.5], [-16.1, 96.7, 0.5], [-15.5, 101.1, 0.5]]],
      "id": "M_F6NfLQ",
      "expect": "square|rectangle",
      "points": [
        [0, 0, 0.5], [-0.5, 0.0, 0.5], [-1.1, 0.0, 0.5], [-3.9, 0.6, 0.5], [-8.3, 0.6, 0.5], [-13.9, 1.1, 0.5], [-19.4, 1.7, 0.5], [-25.0, 2.2, 0.5], [-30.5, 2.2, 0.5], [-36.1, 2.2, 0.5], [-42.8, 2.8, 0.5], [-49.4, 2.8, 0.5], [-56.1, 2.8, 0.5], [-62.8, 2.8, 0.5], [-69.4, 2.8, 0.5], [-76.7, 2.8, 0.5], [-83.9, 3.3, 0.5], [-91.1, 3.9, 0.5], [-98.9, 4.4, 0.5], [-106.1, 5.0, 0.5], [-113.3, 5.6, 0.5], [-121.1, 6.1, 0.5], [-128.3, 6.7, 0.5], [-135.5, 7.2, 0.5], [-142.8, 7.8, 0.5], [-150.0, 8.3, 0.5], [-157.2, 8.9, 0.5], [-163.9, 9.4, 0.5], [-171.1, 10.0, 0.5], [-176.7, 10.0, 0.5], [-182.8, 10.0, 0.5], [-188.3, 10.0, 0.5], [-194.4, 10.0, 0.5], [-199.4, 10.0, 0.5], [-203.3, 9.4, 0.5], [-207.8, 8.9, 0.5], [-212.2, 7.8, 0.5], [-215.5, 6.7, 0.5], [-218.9, 5.6, 0.5], [-222.2, 3.9, 0.5], [-225.0, 2.2, 0.5], [-227.2, 0.6, 0.5], [-230.0, -1.1, 0.5], [-232.2, -2.8, 0.5], [-234.4, -4.4, 0.5], [-236.1, -6.1, 0.5], [-238.3, -8.3, 0.5], [-240.5, -10.6, 0.5], [-242.2, -12.8, 0.5], [-243.3, -15.6, 0.5], [-245.0, -19.4, 0.5], [-246.1, -23.9, 0.5], [-247.8, -28.9, 0.5], [-248.3, -34.4, 0.5], [-248.9, -40.6, 0.5], [-249.4, -47.2, 0.5], [-250.0, -53.9, 0.5], [-250.0, -61.1, 0.5], [-250.0, -68.3, 0.5], [-250.0, -76.1, 0.5], [-250.0, -83.3, 0.5], [-249.4, -91.1, 0.5], [-248.9, -98.3, 0.5], [-248.3, -106.1, 0.5], [-247.8, -113.3, 0.5], [-247.2, -121.1, 0.5], [-246.7, -128.3, 0.5], [-246.1, -135.0, 0.5], [-245.5, -142.2, 0.5], [-245.0, -148.9, 0.5], [-244.4, -155.6, 0.5], [-243.3, -162.8, 0.5], [-242.8, -169.4, 0.5], [-242.2, -176.1, 0.5], [-241.1, -182.8, 0.5], [-240.5, -189.4, 0.5], [-240.0, -196.1, 0.5], [-239.4, -201.7, 0.5], [-238.9, -207.2, 0.5], [-238.3, -212.8, 0.5], [-237.8, -218.9, 0.5], [-237.2, -223.9, 0.5], [-236.7, -228.3, 0.5], [-236.1, -232.2, 0.5], [-235.5, -236.7, 0.5], [-235.0, -241.1, 0.5], [-234.4, -245.0, 0.5], [-233.9, -248.3, 0.5], [-233.9, -250.6, 0.5], [-233.3, -252.8, 0.5], [-232.8, -254.4, 0.5], [-232.2, -256.1, 0.5], [-232.2, -257.8, 0.5], [-231.1, -260.0, 0.5], [-230.5, -261.7, 0.5], [-230.0, -263.9, 0.5], [-229.4, -265.6, 0.5], [-228.9, -267.2, 0.5], [-228.3, -268.3, 0.5], [-227.8, -269.4, 0.5], [-227.2, -270.6, 0.5], [-226.7, -271.1, 0.5], [-226.1, -272.2, 0.5], [-225.5, -272.8, 0.5], [-225.0, -273.3, 0.5], [-224.4, -273.9, 0.5], [-223.3, -274.4, 0.5], [-222.8, -274.4, 0.5], [-221.7, -275.0, 0.5], [-220.5, -275.6, 0.5], [-218.9, -275.6, 0.5], [-217.2, -275.6, 0.5], [-215.5, -276.1, 0.5], [-213.9, -276.1, 0.5], [-211.7, -276.1, 0.5], [-209.4, -276.1, 0.5], [-207.2, -276.1, 0.5], [-205.0, -275.6, 0.5], [-202.2, -275.0, 0.5], [-199.4, -274.4, 0.5], [-196.7, -273.9, 0.5], [-193.3, -273.3, 0.5], [-190.0, -272.8, 0.5], [-186.7, -272.2, 0.5], [-183.3, -271.7, 0.5], [-178.9, -271.1, 0.5], [-174.4, -270.0, 0.5], [-170.5, -269.4, 0.5], [-167.2, -268.9, 0.5], [-162.8, -268.3, 0.5], [-157.8, -267.8, 0.5], [-152.8, -267.8, 0.5], [-147.8, -267.2, 0.5], [-142.8, -267.2, 0.5], [-137.8, -267.2, 0.5], [-132.2, -267.2, 0.5], [-126.7, -267.2, 0.5], [-121.1, -267.2, 0.5], [-115.5, -267.2, 0.5], [-109.4, -267.2, 0.5], [-102.8, -267.2, 0.5], [-97.2, -267.2, 0.5], [-91.7, -267.2, 0.5], [-86.1, -267.2, 0.5], [-79.4, -267.2, 0.5], [-73.9, -267.2, 0.5], [-68.3, -267.2, 0.5], [-62.8, -267.2, 0.5], [-57.2, -267.2, 0.5], [-51.7, -267.2, 0.5], [-47.2, -267.2, 0.5], [-41.7, -267.2, 0.5], [-37.2, -266.7, 0.5], [-32.2, -266.1, 0.5], [-27.8, -265.6, 0.5], [-23.3, -265.0, 0.5], [-20.5, -264.4, 0.5], [-17.2, -263.9, 0.5], [-13.3, -263.3, 0.5], [-10.0, -262.2, 0.5], [-6.1, -261.7, 0.5], [-2.8, -261.1, 0.5], [0.6, -260.6, 0.5], [3.3, -259.4, 0.5], [5.6, -258.9, 0.5], [7.8, -258.3, 0.5], [10.0, -257.2, 0.5], [12.2, -256.1, 0.5], [14.5, -255.0, 0.5], [16.1, -253.9, 0.5], [18.3, -252.2, 0.5], [20.0, -251.1, 0.5], [21.7, -248.9, 0.5], [23.3, -246.7, 0.5], [25.0, -244.4, 0.5], [26.7, -241.7, 0.5], [27.8, -238.9, 0.5], [29.5, -234.4, 0.5], [30.0, -228.3, 0.5], [31.1, -221.7, 0.5], [31.7, -215.0, 0.5], [32.2, -208.9, 0.5], [32.8, -201.7, 0.5], [32.8, -195.0, 0.5], [32.8, -187.8, 0.5], [32.8, -180.6, 0.5], [32.8, -172.8, 0.5], [32.8, -164.4, 0.5], [32.2, -156.1, 0.5], [31.7, -147.2, 0.5], [31.1, -138.3, 0.5], [30.6, -129.4, 0.5], [29.5, -120.0, 0.5], [28.9, -111.7, 0.5], [28.3, -102.8, 0.5], [27.2, -94.4, 0.5], [26.7, -86.1, 0.5], [25.6, -77.8, 0.5], [25.0, -70.0, 0.5], [23.9, -61.7, 0.5], [22.8, -52.8, 0.5], [22.2, -44.4, 0.5], [21.1, -35.6, 0.5], [20.0, -26.7, 0.5], [18.9, -18.3, 0.5], [17.8, -9.4, 0.5], [16.7, -0.6, 0.5], [15.6, 8.9, 0.5], [15.6, 10.0, 0.5]]],
      "id": "CXARIdbr",
      "expect": "square|rectangle",
      "points": [
        [0, 0, 0.5], [-0.5, -0.6, 0.5], [-4.4, -2.2, 0.5], [-8.9, -5.6, 0.5], [-14.4, -8.9, 0.5], [-18.9, -12.2, 0.5], [-23.9, -15.0, 0.5], [-30.0, -17.8, 0.5], [-34.4, -20.0, 0.5], [-40.0, -22.2, 0.5], [-46.1, -24.4, 0.5], [-52.8, -26.7, 0.5], [-59.4, -28.9, 0.5], [-66.1, -30.0, 0.5], [-73.3, -31.7, 0.5], [-80.5, -33.3, 0.5], [-87.8, -33.9, 0.5], [-95.5, -34.4, 0.5], [-102.8, -35.0, 0.5], [-110.5, -35.0, 0.5], [-118.3, -35.0, 0.5], [-126.1, -35.0, 0.5], [-133.9, -35.0, 0.5], [-141.7, -35.0, 0.5], [-148.9, -35.0, 0.5], [-156.1, -35.0, 0.5], [-163.3, -35.0, 0.5], [-170.0, -34.4, 0.5], [-175.5, -33.9, 0.5], [-181.1, -33.3, 0.5], [-186.7, -33.3, 0.5], [-192.2, -32.8, 0.5], [-197.8, -32.2, 0.5], [-202.2, -31.7, 0.5], [-206.7, -31.1, 0.5], [-210.5, -30.0, 0.5], [-215.0, -29.4, 0.5], [-218.9, -28.9, 0.5], [-221.7, -27.8, 0.5], [-224.4, -26.7, 0.5], [-227.2, -25.6, 0.5], [-229.4, -24.4, 0.5], [-231.7, -22.8, 0.5], [-233.9, -21.7, 0.5], [-235.5, -20.0, 0.5], [-237.2, -18.9, 0.5], [-238.3, -17.2, 0.5], [-239.4, -16.1, 0.5], [-240.5, -14.4, 0.5], [-241.7, -12.8, 0.5]
      ]
    }
  ]
}
```

5], [-242.8, -10.6, 0.5], [-243.3, -8.3, 0.5], [-244.4, -6.1, 0.5], [-245.5, -2.8, 0.5], [-246.1, 0.6, 0.5], [-247.2, 5.0, 0.5], [-248.3, 10.0, 0.5], [-248.9, 15.6, 0.5], [-250.0, 21.7, 0.5], [-251.1, 28.9, 0.5], [-252.2, 36.1, 0.5], [-253.3, 43.3, 0.5], [-255.0, 50.6, 0.5], [-255.5, 57.2, 0.5], [-257.2, 63.9, 0.5], [-257.8, 70.6, 0.5], [-258.9, 76.1, 0.5], [-260.0, 81.7, 0.5], [-260.5, 87.2, 0.5], [-261.7, 93.3, 0.5], [-262.2, 98.9, 0.5], [-262.8, 103.3, 0.5], [-263.9, 108.9, 0.5], [-264.4, 113.3, 0.5], [-265.0, 118.9, 0.5], [-265.0, 122.8, 0.5], [-265.0, 127.2, 0.5], [-265.0, 131.7, 0.5], [-265.0, 136.1, 0.5], [-265.0, 141.7, 0.5], [-265.0, 146.1, 0.5], [-265.0, 151.7, 0.5], [-265.0, 156.1, 0.5], [-265.0, 160.0, 0.5], [-264.4, 164.4, 0.5], [-263.9, 168.9, 0.5], [-263.3, 172.8, 0.5], [-262.2, 176.1, 0.5], [-261.1, 179.4, 0.5], [-259.4, 182.2, 0.5], [-257.8, 185.6, 0.5], [-255.5, 188.9, 0.5], [-253.3, 192.8, 0.5], [-250.0, 196.1, 0.5], [-246.7, 200.0, 0.5], [-243.3, 203.9, 0.5], [-240.0, 207.2, 0.5], [-236.1, 211.1, 0.5], [-231.1, 214.4, 0.5], [-226.1, 218.3, 0.5], [-220.5, 222.2, 0.5], [-215.0, 225.6, 0.5], [-207.8, 229.4, 0.5], [-201.1, 233.3, 0.5], [-193.9, 237.2, 0.5], [-187.2, 240.6, 0.5], [-180.0, 243.9, 0.5], [-172.8, 247.2, 0.5], [-166.1, 250.0, 0.5], [-158.9, 253.3, 0.5], [-151.7, 255.6, 0.5], [-143.9, 258.3, 0.5], [-136.7, 260.6, 0.5], [-129.4, 262.8, 0.5], [-121.7, 265.0, 0.5], [-114.4, 266.7, 0.5], [-106.7, 268.3, 0.5], [-98.9, 270.0, 0.5], [-91.1, 271.7, 0.5], [-83.3, 272.8, 0.5], [-75.5, 273.9, 0.5], [-67.8, 274.4, 0.5], [-60.0, 275.6, 0.5], [-52.8, 276.1, 0.5], [-45.5, 276.7, 0.5], [-38.3, 277.8, 0.5], [-31.7, 278.3, 0.5], [-25.5, 278.9, 0.5], [-20.5, 279.4, 0.5], [-15.0, 280.0, 0.5], [-9.4, 280.0, 0.5], [-5.5, 280.6, 0.5], [-2.2, 280.6, 0.5], [2.2, 280.6, 0.5], [6.7, 280.6, 0.5], [10.6, 280.0, 0.5], [13.3, 280.0, 0.5], [16.7, 278.9, 0.5], [19.5, 277.8, 0.5], [22.2, 276.7, 0.5], [24.5, 275.0, 0.5], [27.2, 272.8, 0.5], [28.9, 270.0, 0.5], [31.1, 267.2, 0.5], [32.8, 263.9, 0.5], [34.5, 260.0, 0.5], [36.1, 255.0, 0.5], [37.2, 249.4, 0.5], [38.3, 243.9, 0.5], [38.9, 238.3, 0.5], [39.5, 232.2, 0.5], [40.0, 225.6, 0.5], [40.0, 218.3, 0.5], [40.0, 211.7, 0.5], [40.0, 204.4, 0.5], [39.5, 196.7, 0.5], [38.3, 188.9, 0.5], [37.2, 181.1, 0.5], [36.1, 173.3, 0.5], [35.0, 165.6, 0.5], [33.9, 158.3, 0.5], [32.2, 150.6, 0.5], [31.1, 142.8, 0.5], [30.0, 135.6, 0.5], [28.3, 127.8, 0.5], [26.7, 120.6, 0.5], [25.6, 114.4, 0.5], [23.9, 108.3, 0.5], [22.8, 101.7, 0.5], [21.1, 95.6, 0.5], [20.0, 89.4, 0.5], [18.9, 83.3, 0.5], [18.3, 76.7, 0.5], [17.2, 71.1, 0.5], [16.7, 66.7, 0.5], [16.1, 61.7, 0.5], [15.6, 56.1, 0.5], [15.0, 50.6, 0.5], [14.5, 46.1, 0.5], [13.9, 42.2, 0.5], [13.3, 37.8, 0.5], [13.3, 33.3, 0.5], [13.3, 27.8, 0.5], [13.3, 23.3, 0.5], [12.8, 19.4, 0.5], [12.2, 17.2, 0.5], [12.2, 14.4, 0.5], [11.7, 12.2, 0.5], [11.7, 10.0, 0.5], [11.7, 7.8, 0.5], [11.7, 5.6, 0.5], [11.7, 3.3, 0.5], [11.7, 1.1, 0.5], [11.7, -0.6, 0.5], [11.7, -2.2, 0.5], [11.7, -3.9, 0.5], [11.7, -5.0, 0.5], [11.1, -6.1, 0.5], [11.1, -7.2, 0.5], [10.6, -8.3, 0.5], [10.6, -9.4, 0.5], [10.6, -9.4, 0.5], [10.0, -10.6, 0.5], [9.5, -11.1, 0.5], [8.9, -12.2, 0.5], [8.3, -13.3, 0.5], [7.2, -14.4, 0.5], [6.1, -15.6, 0.5], [5.0, -16.7, 0.5], [2.8, -18.3, 0.5], [1.1, -20.0, 0.5], [-1.7, -21.1, 0.5], [-3.9, -22.8, 0.5], [-6.7, -23.9, 0.5], [-9.4, -25.6, 0.5], [-13.9, -26.7, 0.5], [-18.3, -28.3, 0.5], [-23.9, -29.4, 0.5], [-28.9, -30.6, 0.5], [-34.4, -31.7, 0.5], [-40.5, -32.2, 0.5], [-47.8, -33.3, 0.5], [-55.0, -33.9, 0.5], [-61.7, -34.4, 0.5], [-69.4, -34.4, 0.5], [-76.7, -35.0, 0.5], [-84.4, -35.0, 0.5], [-92.2, -35.0, 0.5], [-100.5, -35.0, 0.5], [-108.9, -35.6, 0.5], [-117.2, -35.6, 0.5], [-125.5, -36.1, 0.5], [-133.9, -36.7, 0.5], [-142.2, -37.2, 0.5], [-150.5, -37.8, 0.5], [-158.9, -38.3, 0.5], [-166.7, -38.9, 0.5], [-174.4, -39.4, 0.5], [-182.2, -40.0, 0.5], [-189.4, -40.0, 0.5], [-196.7, -40.6, 0.5], [-202.8, -40.6, 0.5], [-208.3, -40.6, 0.5], [-213.9, -40.6, 0.5], [-219.4, -40.6, 0.5], [-225.0, -40.6, 0.5], [-228.9, -40.6, 0.5], [-233.3, -40.6, 0.5], [-237.2, -40.6, 0.5], [-241.7, -40.6, 0.5], [-246.1, -40.0, 0.5], [-249.4, -38.9, 0.5], [-252.8, -38.3, 0.5], [-256.7, -36.7, 0.5], [-260.0, -35.0, 0.5], [-263.9, -33.3, 0.5], [-267.8, -30.6, 0.5], [-271.7, -27.8, 0.5], [-275.5, -25.0, 0.5], [-279.4, -21.1, 0.5], [-281.7, -19.4, 0.5]], {"id": "IUGqKSHC", "expect": "rectangle|square|quadrilateral", "points": "[[0,0,0.5], [-1.1,0,0.5], [-4.4,-0.6,0.5], [-10.0,-0.6,0.5], [-17.2,-1.7,0.5], [-22.8,-1.7,0.5], [-28.3,-2.8,0.5], [-34.4,-3.3,0.5], [-41.7,-3.9,0.5], [-48.9,-5.0,0.5], [-64.4,-6.7,0.5], [-72.8,-7.8,0.5], [-82.2,-8.9,0.5], [-91.7,-10.0,0.5], [-101.1,-11.1,0.5], [-111.7,-12.2,0.5], [-122.2,-13.3,0.5], [-132.8,-14.4,0.5], [-143.9,-15.6,0.5], [-155.5,-16.7,0.5], [-167.2,-17.8,0.5], [-178.9,-20.0,0.5], [-190.5,-21.7,0.5], [-220.5,-27.2,0.5], [-234.4,-30.0,0.5], [-247.2,-32.8,0.5], [-258.3,-35.0,0.5], [-270.5,-37.8,0.5], [-281.7,-40.0,0.5], [-293.3,-42.8,0.5], [-305.0,-45.6,0.5], [-316.7,-48.3,0.5], [-327.2,-51.1,0.5], [-338.9,-53.9,0.5], [-348.9,-56.7,0.5], [-358.9,-59.4,0.5], [-368.9,-62.2,0.5], [-378.3,-65.0,0.5], [-387.8,-67.8,0.5], [-397.2,-71.1,0.5], [-405.0,-73.3,0.5], [-413.9,-75.6,0.5], [-421.7,-78.3,0.5], [-428.3,-80.0,0.5], [-436.1,-82.8,0.5], [-443.3,-84.4,0.5], [-447.2,-86.1,0.5], [-454.4,-87.8,0.5], [-458.9,-88.9,0.5], [-465.0,-90.0,0.5], [-467.8,-91.1,0.5], [-473.3,-92.2,0.5], [-476.1,-92.8,0.5], [-479.4,-93.3,0.5], [-482.2,-93.3,0.5], [-485.0,-93.9,0.5], [-487.8,-94.4,0.5], [-489.4,-94.4,0.5], [-491.1,-95.0,0.5], [-492.2,-94.4,0.5], [-490.5,-94.4,0.5], [-491.1,-94.4,0.5], [-491.1,-93.9,0.5], [-491.7,-93.3,0.5], [-492.2,-92.2,0.5], [-492.8,-90.6,0.5], [-493.3,-88.3,0.5], [-493.9,-86.7,0.5], [-495.0,-83.9,0.5], [-495.5,-80.0,0.5], [-496.1,-73.3,0.5], [-497.2,-66.1,0.5], [-497.8,-58.3,0.5], [-498.9,-51.1,0.5], [-500.0,-43.9,0.5], [-500.5,-36.1,0.5], [-501.7,-28.3,0.5], [-502.8,-20.6,0.5], [-504.4,-12.2,0.5], [-506.1,-3.3,0.5], [-507.8,5.0,0.5], [-508.9,13.9,0.5], [-510.5,22.8,0.5], [-512.2,31.1,0.5], [-513.3,39.4,0.5], [-515.0,47.2,0.5], [-516.7,55.6,0.5], [-517.8,63.3,0.5], [-519.4,71.7,0.5], [-520.5,80.0,0.5], [-522.8,88.3,0.5], [-524.4,97.2,0.5], [-526.1,106.1,0.5], [-528.3,114.4,0.5], [-530.0,122.8,0.5], [-532.2,130.6,0.5], [-533.9,138.3,0.5], [-536.1,145.0,0.5], [-537.8,151.1,0.5], [-538.9,156.7,0.5], [-540.5,162.2,0.5], [-542.2,167.8,0.5], [-543.9,174.4,0.5], [-545.5,181.7,0.5], [-547.8,187.8,0.5], [-550.0,194.4,0.5], [-552.2,201.1,0.5], [-554.4,207.2,0.5], [-556.7,213.9,0.5], [-559.4,220.6,0.5], [-562.2,227.2,0.5], [-565.0,235.0,0.5], [-568.3,242.8,0.5], [-571.7,250.6,0.5], [-575.0,258.3,0.5], [-577.8,264.4,0.5], [-580.5,270.0,0.5], [-582.8,274.4,0.5], [-585.0,278.9,0.5], [-586.7,282.2,0.5], [-588.3,285.0,0.5], [-589.4,287.2,0.5], [-590.0,288.9,0.5], [-590.5,290.6,0.5], [-591.1,291.7,0.5], [-591.7,293.3,0.5], [-592.2,294.4,0.5], [-592.2,295.6,0.5], [-592.8,297.2,0.5], [-592.8,298.3,0.5], [-592.8,299.4,0.5], [-592.8,300.0,0.5], [-592.8,300.6,0.5], [-592.8,301.1,0.5], [-592.8,301.7,0.5], [-592.8,302.2,0.5], [-592.8,302.8,0.5], [-592.8,303.3,0.5], [-592.8,303.9,0.5], [-592.8,304.4,0.5], [-592.8,305.0,0.5], [-592.8,305.6,0.5], [-592.8,306.1,0.5], [-592.8,306.7,0.5], [-592.2,306.7,0.5], [-592.2,307.2,0.5], [-591.7,307.8,0.5], [-590.5,308.3,0.5], [-588.3,309.4,0.5], [-585.5,310.6,0.5], [-582.2,311.1,0.5], [-578.9,311.1,0.5], [-575.5,311.7,0.5], [-572.2,311.7,0.5], [-567.8,311.7,0.5], [-563.3,311.7,0.5], [-558.9,311.1,0.5], [-555.5,311.1,0.5], [-551.1,310.6,0.5], [-546.1,310.6,0.5], [-540.5,310.0,0.5], [-535.5,310.0,0.5], [-530.0,309.4,0.5], [-523.9,309.4,0.5], [-517.2,309.4,0.5], [-510.5,309.4,0.5], [-503.9,309.4,0.5], [-496.7,309.4,0.5], [-489.4,309.4,0.5], [-482.2,310.0,0.5], [-474.4,310.0,0.5], [-466.7,310.6,0.5], [-458.9,311.1,0.5], [-450.5,311.1,0.5], [-442.2,311.7,0.5], [-433.3,311.7,0.5], [-424.4,312.2,0.5], [-415.5,312.2,0.5], [-406.1,312.8,0.5], [-396.1,313.3,0.5], [-386.7,313.9,0.5], [-376.7,314.4,0.5], [-367.2,315.6,0.5], [-357.2,316.1,0.5], [-347.2,317.2,0.5], [-337.2,318.3,0.5], [-326.7,319.4,0.5], [-316.1,321.1,0.5], [-305.5,323.3,0.5], [-294.4,325.0,0.5], [-283.9,327.2,0.5], [-272.8,329.4,0.5], [-261.7,331.7,0.5], [-250.5,333.9,0.5], [-239.4,336.7,0.5], [-210.5,342.2,0.5], [-196.1,346.1,0.5], [-185.0,348.9,0.5], [-173.3,352.2,0.5], [-161.7,355.6,0.5], [-150.0,359.4,0.5], [-138.3,362.2,0.5], [-127.8,365.6,0.5], [-117.2,368.9,0.5], [-106.1,371.7,0.5], [-95.5,375.6,0.5], [-85.5,380.0,0.5], [-75.0,383.9,0.5], [-65.0,387.2,0.5], [-56.7,391.1,0.5], [-48.3,395.6,0.5], [-39.4,399.4,0.5], [-31.1,403.3,0.5], [-23.3,407.2,0.5], [-15.5,411.1,0.5], [-8.9,414.4,0.5], [-2.2,418.3,0.5], [4.5,421.7,0.5], [10.6,424.4,0.5], [15.6,427.8,0.5], [22.2,430.6,0.5], [26.7,432.8,0.5], [32.8,435.0,0.5], [36.1,436.7,0.5], [39.5,438.9,0.5], [43.3,440.6,0.5], [48.3,442.2,0.5], [51.1,443.3,0.5], [53.9,444.4,0.5], [56.7,445.0,0.5], [59.5,445.6,0.5], [61.1,446.1,0.5], [62.8,446.1,0.5], [64.5,446.7,0.5], [65.6,447.2,0.5], [63.9,446.7,0.5], [64.5,446.7,0.5], [64.5,446.6,0.5], [65.0,443.9,0.5], [65.0,440.6,0.5], [6

5.0,436.1,0.5],[65.0,431.1,0.5],[64.5,425.6,0.5],[64.5,421.1,0.5],[63.9,415.6,0.5],[63.3,409.4,0.5],[63.3,401.7,0.5],[63.3,394.4,0.5],[62.8,387.2,0.5],[62.8,378.9,0.5],[62.2,371.1,0.5],[62.2,362.8,0.5],[62.2,353.9,0.5],[62.2,345.6,0.5],[62.2,337.2,0.5],[62.2,328.3,0.5],[62.2,319.4,0.5],[62.2,310.0,0.5],[62.2,301.1,0.5],[62.2,292.2,0.5],[62.2,283.3,0.5],[62.8,274.4,0.5],[63.3,265.0,0.5],[64.5,255.6,0.5],[65.0,246.7,0.5],[65.6,237.2,0.5],[66.7,228.3,0.5],[67.8,220.0,0.5],[68.3,211.1,0.5],[70.0,202.2,0.5],[71.1,192.8,0.5],[72.2,183.9,0.5],[73.9,175.0,0.5],[75.6,166.1,0.5],[77.2,157.8,0.5],[78.3,149.4,0.5],[80.0,141.7,0.5],[81.7,133.3,0.5],[82.8,125.6,0.5],[84.5,118.3,0.5],[85.6,111.1,0.5],[86.7,103.9,0.5],[87.8,97.8,0.5],[89.5,92.2,0.5],[90.6,85.6,0.5],[91.7,79.4,0.5],[93.3,73.3,0.5],[94.5,68.9,0.5],[96.1,62.8,0.5],[97.8,58.3,0.5],[99.5,52.8,0.5],[100.6,48.3,0.5],[102.2,42.8,0.5],[103.9,37.2,0.5],[105.0,32.2,0.5],[106.1,27.8,0.5],[107.8,24.4,0.5],[108.9,20.6,0.5],[109.5,17.2,0.5],[110.0,13.3,0.5],[110.6,10.0,0.5],[111.1,6.1,0.5],[111.1,3.3,0.5],[111.7,0.6,0.5],[111.7,-1.7,0.5],[111.7,-3.9,0.5],[111.7,-6.1,0.5],[111.7,-7.8,0.5],[111.7,-10.0,0.5],[111.1,-11.7,0.5],[111.1,-12.8,0.5],[111.1,-14.4,0.5],[110.6,-15.6,0.5],[110.0,-16.1,0.5],[110.0,-17.2,0.5],[109.5,-17.8,0.5],[108.9,-17.8,0.5],[108.9,-18.3,0.5],[108.3,-18.3,0.5],[106.7,-18.9,0.5],[105.0,-19.4,0.5],[102.8,-20.0,0.5],[100.6,-20.0,0.5],[97.8,-20.0,0.5],[95.0,-20.0,0.5],[92.2,-20.0,0.5],[87.8,-19.4,0.5],[83.3,-18.9,0.5],[77.8,-17.2,0.5],[73.3,-16.1,0.5],[68.3,-14.4,0.5],[62.8,-12.8,0.5],[56.1,-11.1,0.5],[50.6,-9.4,0.5],[45.0,-7.8,0.5],[38.9,-6.1,0.5],[32.2,-4.4,0.5],[25.0,-2.8,0.5],[18.9,-1.7,0.5],[11.7,-0.6,0.5],[5.0,0.6,0.5],[-2.2,1.1,0.5],[-8.9,1.7,0.5],[-16.1,2.2,0.5],[-23.9,2.2,0.5],[-31.1,2.2,0.5],[-38.9,2.2,0.5],[-46.7,1.7,0.5],[-54.4,0.6,0.5],[-62.8,-0.6,0.5],[-71.1,-1.7,0.5],[-80.0,-2.8,0.5],[-88.3,-3.9,0.5],[-97.2,-5.6,0.5],[-106.1,-6.7,0.5],[-115.0,-8.3,0.5],[-123.9,-9.4,0.5],[-132.8,-11.1,0.5],[-141.7,-12.8,0.5],[-150.5,-13.9,0.5],[-159.4,-15.6,0.5],[-168.3,-17.2,0.5],[-177.2,-18.3,0.5],[-186.1,-20.0,0.5],[-195.0,-21.1,0.5],[-203.3,-22.2,0.5],[-212.2,-23.3,0.5],[-220.5,-25.0,0.5],[-229.4,-26.1,0.5],[-237.8,-27.2,0.5],[-246.7,-28.3,0.5],[-255.5,-28.9,0.5],[-264.4,-30.0,0.5],[-273.3,-30.6,0.5],[-282.8,-31.1,0.5],[-292.2,-31.1,0.5],[-293.3,-31.1,0.5]},{"id":"zm59zYcm","expect":"square|rectangle","points":[[0,0,0.5],[0,0.2,0.5],[0.6,10.0,0.5],[1.1,20.0,0.5],[2.2,30.0,0.5],[3.3,38.3,0.5],[3.9,46.7,0.5],[5.0,55.0,0.5],[5.6,64.4,0.5],[6.1,74.4,0.5],[6.7,83.9,0.5],[8.9,110.6,0.5],[9.5,121.1,0.5],[9.5,130.6,0.5],[10.0,140.6,0.5],[10.6,147.2,0.5],[11.1,156.1,0.5],[12.2,165.0,0.5],[12.8,172.2,0.5],[13.3,181.7,0.5],[13.9,191.1,0.5],[14.5,197.8,0.5],[15.0,207.2,0.5],[15.0,215.0,0.5],[15.6,219.4,0.5],[16.7,226.7,0.5],[17.2,233.9,0.5],[17.2,240.0,0.5],[17.2,242.8,0.5],[18.3,245.6,0.5],[18.3,248.9,0.5],[18.9,252.8,0.5],[18.9,255.6,0.5],[18.9,257.8,0.5],[18.9,259.4,0.5],[18.9,257.2,0.5],[18.9,257.8,0.5],[18.3,257.8,0.5],[18.3,258.3,0.5],[16.7,258.9,0.5],[15.0,259.4,0.5],[12.2,260.6,0.5],[9.5,261.7,0.5],[6.1,262.2,0.5],[2.8,262.8,0.5],[-0.5,263.9,0.5],[-3.9,264.4,0.5],[-8.9,265.6,0.5],[-14.4,266.1,0.5],[-19.4,267.2,0.5],[-25.0,267.8,0.5],[-30.5,269.4,0.5],[-36.7,270.6,0.5],[-43.3,272.8,0.5],[-49.4,274.4,0.5],[-56.1,276.7,0.5],[-62.8,279.4,0.5],[-69.4,281.7,0.5],[-76.1,283.9,0.5],[-83.3,286.1,0.5],[-90.5,288.3,0.5],[-97.8,290.0,0.5],[-105.0,292.2,0.5],[-112.8,293.9,0.5],[-120.0,296.1,0.5],[127.8,298.3,0.5],[-135.0,301.1,0.5],[-142.8,303.3,0.5],[-150.0,306.1,0.5],[-157.8,308.3,0.5],[-165.0,310.0,0.5],[-172.8,311.7,0.5],[-180.0,312.8,0.5],[-187.2,313.9,0.5],[-194.4,314.4,0.5],[-201.7,314.4,0.5],[-208.3,314.4,0.5],[-213.9,314.4,0.5],[-219.4,314.4,0.5],[-225.5,313.9,0.5],[-231.1,312.2,0.5],[-236.7,310.6,0.5],[-240.5,308.3,0.5],[-246.1,305.6,0.5],[-250.0,302.8,0.5],[-254.4,300.0,0.5],[-258.3,297.2,0.5],[-261.7,295.0,0.5],[-265.0,292.2,0.5],[-268.3,287.8,0.5],[-271.1,283.3,0.5],[-274.4,278.9,0.5],[-277.2,275.0,0.5],[-280.0,270.6,0.5],[-282.2,264.4,0.5],[-285.0,257.2,0.5],[-286.7,251.1,0.5],[-288.3,245.0,0.5],[-289.4,238.3,0.5],[-291.1,230.6,0.5],[-292.2,222.2,0.5],[-293.3,214.4,0.5],[-294.4,205.6,0.5],[-296.1,181.1,0.5],[-297.2,170.0,0.5],[-298.3,160.6,0.5],[-299.4,148.9,0.5],[-300.5,140.6,0.5],[-301.7,128.3,0.5],[-303.3,118.9,0.5],[-304.4,107.2,0.5],[-306.1,96.1,0.5],[-308.3,86.7,0.5],[-310.0,75.0,0.5],[-311.7,66.7,0.5],[-313.3,55.0,0.5],[-315.5,45.6,0.5],[-317.2,36.1,0.5],[-320.0,25.6,0.5],[-322.2,18.3,0.5],[-325.0,9.4,0.5],[-327.8,0.0,0.5],[-330.5,-6.1,0.5],[-333.3,-12.8,0.5],[-335.5,-21.7,0.5],[-337.8,-28.3,0.5],[-340.0,-32.2,0.5],[-341.1,-38.3,0.5],[-342.8,-43.9,0.5],[-344.4,-50.0,0.5],[-346.1,-51.7,0.5],[-347.8,-53.9,0.5],[-349.4,-56.1,0.5],[-351.1,-58.3,0.5],[-352.2,-61.1,0.5],[-353.3,-62.8,0.5],[-353.9,-65.0,0.5],[-352.8,-62.8,0.5],[-353.3,-63.3,0.5],[-352.8,-63.3,0.5],[-351.1,-62.2,0.5],[-348.9,-61.1,0.5],[-345.5,-59.4,0.5],[-342.8,-58.3,0.5],[-339.4,-56.7,0.5],[-335.5,-55.6,0.5],[-331.1,-53.9,0.5],[-326.7,-52.2,0.5],[-323.9,-50.6,0.5],[-319.4,-48.3,0.5],[-315.0,-46.1,0.5],[-310.0,-44.4,0.5],[-305.5,-42.2,0.5],[-301.1,-40.6,0.5],[-296.7,-39.4,0.5],[-292.2,-37.8,0.5],[-286.1,-36.7,0.5],[-281.7,-35.6,0.5],[-275.5,-35.0,0.5],[-270.0,-34.4,0.5],[-263.3,-33.9,0.5],[-257.2,-33.9,0.5],[-251.1,-33.9,0.5],[-245.0,-33.9,0.5],[-238.3,-33.9,0.5],[-232.2,-33.9,0.5],[-225.5,-33.9,0.5],[-219.4,-33.9,0.5],[-212.8,-34.4,0.5],[-206.1,-35.0,0.5],[-199.4,-36.1,0.5],[-192.8,-36.7,0.5],[-186.1,-38.3,0.5],[-179.4,-39.4,0.5],[-172.8,-41.1,0.5],[-166.1,-42.2,0.5],[-158.9,-43.9,0.5],[-152.2,-45.0,0.5],[-145.5,-46.1,0.5],[-138.9,-47.2,0.5],[-132.2,-47.8,0.5],[-125.5,-48.9,0.5],[-118.9,-49.4,0.5],[-112.2,-50.0,0.5],[-105.5,-51.1,0.5],[-98.9,-51.7,0.5],[-92.2,-52.2,0.5],[-85.5,-52.8,0.5],[-79.4,-53.3,0.5],[-72.8,-53.3,0.5],[-67.8,-53.9,0.5],[-62.2,-53.9,0.5],[-56.1,-54.4,0.5],[-50.5,-54.4,0.5],[-45.0,-54.4,0.5],[-40.0,-54.4,0.5],[-34.4,-54.4,0.5],[-29.4,-54.4,0.5],[-23.9,-54.4,0.5],[-19.4,-54.4,0.5],[-15.0,-54.4,0.5],[-10.5,-54.4,0.5],[-6.1,-55.0,0.5],[-2.2,-55.0,0.5],[1.1,-55.6,0.5],[4.5,-55.6,0.5],[7.2,-55.6,0.5],[9.5,-55.6,0.5],[11.1,-55.6,0.5],[12.8,-55.6,0.5],[13.9,-55.6,0.5],[15.6,-55.0,0.5],[16.1,-55.0,0.5],[17.2,-55.0,0.5],[17.8,-55.0,0.5],[18.3,-54.4,0.5],[18.9,-54.4,0.5],[18.9,-53.9,0.5],[19.5,-53.3,0.5],[20.0,-52.2,0.5],[20.6,-50.6,0.5],[20.6,-47.8,0.5],[21.1,-45.6,0.5],[21.1,-43.3,0.5],[20.6,-40.6,0.5],[20.6,-36.1,0.5],[20.0,-30.6,0.5],[18.9,-23.3,0.5],[17.8,-16.1,0.5],[16.7,-7.8,0.5],[15.6,0.6,0.5],[14.5,9.4,0.5],[13.3,18.3,0.5],[12.2,27.2,0.5],[11.1,36.1,0.5],[10.6,44.4,0.5],[10.0,51.7,0.5],[9.5,57.2,0.5],[8.9,61.1,0.5],[8.9,63.3,0.5],[8.9,65.0,0.5],[8.9,67.2,0.5],[8.9,69.4,0.5],[8.9,71.7,0.5],[8.9,73.9,0.5],[8.9,76.7,0.5],[8.9,79.4,0.5],[8.9,82.2,0.5],[8.9,85.0,0.5],[8.9,87.2,0.5],[8.9,91.1,0.5],[8.9,96.1,0.5],[9.5,102.2,0.5],[9.5,108.9,0.5],[10.0,115.6,0.5],[11.1,122.2,0.5],[11.7,128.9,0.5],[12.2,134.4,0.5],[13.3,138.3,0.5],[13.9,141.1,0.5],[15.0,144.4,0.5],[16.1,147.2,0.5],[17.2,150.0,0.5],[17.8,152.2,0.5],[18.3,154.4,0.5],[18.9,156.1,0.5],[19.5,157.2,0.5],[19.5,158.3,0.5],[20.0,158.9,0.5],[20.0,160.0,0.5],[20.0,160.6,0.5],[20.0,161.7,0.5],[19.5,162.2,0.5],[19.5,162.8,0.5]},{"id":"Ya5X0xSh","expect":"rectangle|square|quadrilateral","points":[[0,0,0.5],[-0.5,0.0,0.5],[-3.3,-0.6,0.5],[-6.7,-1.1,0.5],[-11.1,-1.7,0.5],[-15.0,-2.8,0.5],[-19.4,-3.9,0.5],[-23.9,-4.4,0.5],[-28.3,-5.6,0.5],[-32.8,-6.7,0.5],[-38.3,-7.8,0.5],[-43.9,-8.9,0.5],[-49.4,-10.0,0.5],[-55.0,-11.1,0.5],[-61.1,-12.2,0.5],[-67.8,-12.8,0.5],[-73.3,-13.9,0.5],[-79.4,-14.4,0.5],[-85.5,-15.0,0.5],[-92.8,-15.0,0.5],[-98.3,-15.6,0.5],[-104.4,-15.6,0.5],[-110.5,-15.6,0.5],[-117.2,-15.6,0.5],[-122.8,-15.6,0.5],[-126.7,-16.1,0.5],[-131.1,-16.7,0.5],[-136.7,-17.8,0.5],[-142.2,-18.3,0.5],[-146.1,-19.4,0.5],[-148.9,-20.0,0.5],[-151.7,-21.1,0.5],[-154.4,-21.7,0.5],[-157.8,-22.8,0.5],[-160.5,-23.3,0.5],[-163.9,-24.4,0.5],[-166.1,-25.0,0.5],[-168.3,-25.6,0.5],[-170.0,-26.7,0.5],[-171.1,-27.2,0.5],[-172.2,-27.8,0.5],[-173.3,-28.3,0.5],[-173.9,-28.3,0.5],[-174.4,-28.9,0.5],[-174.4,-29.4,0.5],[-175.0,-29.4,0.5],[-175.0,-28.9,0.5],[-176.1,-26.7,0.5],[-176.7,-22.8,0.5],[-177.8,-17.2,0.5],[-178.9,-11.7,0.5],[-179.4,-7.2,0.5],[-1

```
80.0,-4.4,0.5],[-180.0,0.0,0.5],[-180.0,5.6,0.5],[-180.0,11.1,0.5],[-180.0,15.6,0.5],[-180.0,18.3,0.5],[-180.0,21.7,0.5],[-180.0,26.1,0.5],[-179.4,30.0,0.5],[-178.9,34.4,0.5],[-177.8,37.8,0.5],[-176.7,40.6,0.5],[-175.5,43.3,0.5],[-174.4,45.6,0.5],[-172.8,47.8,0.5],[-171.1,50.0,0.5],[-169.4,52.8,0.5],[-167.2,55.0,0.5],[-165.5,57.2,0.5],[-163.3,59.4,0.5],[-161.7,61.1,0.5],[-159.4,62.8,0.5],[-157.8,64.4,0.5],[-155.5,65.6,0.5],[-153.3,67.2,0.5],[-151.1,67.8,0.5],[-148.3,68.9,0.5],[-145.5,70.0,0.5],[-142.2,70.6,0.5],[-138.9,71.1,0.5],[-135.0,71.7,0.5],[-131.1,71.7,0.5],[-126.1,72.2,0.5],[-122.2,72.2,0.5],[-117.8,72.8,0.5],[-114.4,72.8,0.5],[-110.0,73.3,0.5],[-105.5,73.9,0.5],[-100.0,73.9,0.5],[-95.5,74.4,0.5],[-91.1,74.4,0.5],[-86.7,75.0,0.5],[-82.2,75.6,0.5],[-76.7,76.1,0.5],[-72.8,76.7,0.5],[-68.3,77.2,0.5],[-65.5,78.3,0.5],[-62.2,78.9,0.5],[-58.9,80.6,0.5],[-55.0,81.7,0.5],[-51.1,83.3,0.5],[-47.8,84.4,0.5],[-44.4,86.1,0.5],[-41.7,88.3,0.5],[-38.9,90.0,0.5],[-36.7,91.7,0.5],[-34.4,93.3,0.5],[-32.2,95.0,0.5],[-30.5,96.1,0.5],[-28.9,97.8,0.5],[-27.2,98.9,0.5],[-25.5,100.0,0.5],[-24.4,101.7,0.5],[-22.8,103.9,0.5],[-21.1,106.7,0.5],[-20.0,109.4,0.5],[-19.4,111.7,0.5],[-18.9,113.3,0.5],[-18.3,115.0,0.5],[-17.8,116.7,0.5],[-17.2,117.8,0.5],[-17.2,118.3,0.5],[-17.2,118.9,0.5],[-16.7,118.3,0.5],[-15.5,116.7,0.5],[-13.9,113.9,0.5],[-12.2,110.0,0.5],[-10.0,105.6,0.5],[-8.3,100.0,0.5],[-6.7,93.9,0.5],[-5.0,88.9,0.5],[-3.3,84.4,0.5],[-2.2,79.4,0.5],[-0.5,73.9,0.5],[1.1,67.2,0.5],[2.2,61.7,0.5],[3.3,57.8,0.5],[4.5,53.9,0.5],[5.0,48.3,0.5],[6.1,43.3,0.5],[6.1,39.4,0.5],[6.7,36.1,0.5],[6.7,33.3,0.5],[6.7,30.6,0.5],[6.7,27.2,0.5],[6.7,23.9,0.5],[6.7,20.0,0.5],[6.7,16.1,0.5],[6.7,13.3,0.5],[6.7,11.1,0.5],[6.7,8.9,0.5],[6.7,7.8,0.5],[6.7,6.7,0.5],[6.7,5.6,0.5],[6.7,4.4,0.5],[6.7,2.8,0.5],[6.7,1.7,0.5],[6.7,0.0,0.5],[6.7,-0.6,0.5],[6.7,-1.1,0.5],[6.7,-1.7,0.5],[6.7,-2.2,0.5]],{"id":"elOPTxpZ","expect":"none","points":[[0,0,0.5],[-2.7,0.0,0.5],[-8.6,0.0,0.5],[-16.7,-1.1,0.5],[-24.8,-2.7,0.5],[-30.1,-3.8,0.5],[-36.1,-4.8,0.5],[-41.4,-4.8,0.5],[-47.9,-5.4,0.5],[-53.8,-5.4,0.5],[-59.8,-5.4,0.5],[-66.2,-5.4,0.5],[-72.7,-4.8,0.5],[-79.7,-3.8,0.5],[-86.7,-2.2,0.5],[-93.7,-0.5,0.5],[-101.2,1.6,0.5],[-108.2,4.3,0.5],[-115.7,7.5,0.5],[-122.7,12.4,0.5],[-128.1,16.7,0.5],[-132.4,21.0,0.5],[-137.3,24.2,0.5],[-142.1,29.1,0.5],[-146.4,35.0,0.5],[-149.1,40.4,0.5],[-151.3,44.7,0.5],[-152.9,48.5,0.5],[-154.0,53.8,0.5],[-154.5,59.2,0.5],[-155.0,63.5,0.5],[-154.5,66.8,0.5],[-154.0,70.5,0.5],[-152.9,74.8,0.5],[-151.3,79.7,0.5],[-148.6,84.0,0.5],[-145.9,87.8,0.5],[-142.7,91.0,0.5],[-139.4,93.7,0.5],[-135.1,96.4,0.5],[-130.3,98.5,0.5],[-124.9,101.2,0.5],[-119.0,104.4,0.5],[-113.6,107.1,0.5],[-107.1,110.4,0.5],[-100.1,112.5,0.5],[-94.2,114.7,0.5],[-87.7,116.8,0.5],[-81.8,117.9,0.5],[-76.4,119.5,0.5],[-71.1,120.6,0.5],[-66.7,122.2,0.5],[-62.4,123.8,0.5],[-59.2,126.0,0.5],[-55.4,128.7,0.5],[-51.7,131.9,0.5],[-48.4,135.7,0.5],[-45.8,138.9,0.5],[-43.1,142.1,0.5],[-40.9,145.4,0.5],[-39.3,148.6,0.5],[-38.2,153.4,0.5],[-37.1,157.2,0.5],[-36.1,161.0,0.5],[-35.5,164.2,0.5],[-35.5,168.0,0.5],[-36.6,172.8,0.5],[-37.7,179.3,0.5],[-40.4,184.7,0.5],[-43.6,190.0,0.5],[-47.9,196.0,0.5],[-53.8,203.0,0.5],[-60.3,209.4,0.5],[-66.7,215.3,0.5],[-73.7,221.3,0.5],[-80.7,227.2,0.5],[-87.7,232.0,0.5],[-94.7,235.3,0.5],[-101.2,238.0,0.5],[-107.1,239.6,0.5],[-112.5,240.1,0.5],[-117.4,240.6,0.5],[-121.7,240.1,0.5],[-126.5,239.6,0.5],[-129.2,238.5,0.5]],{"id":"0zvn8lBy","expect":"none","points":[[0,0,0.5],[0.5,0.0,0.5],[4.3,0.0,0.5],[8.6,0.0,0.5],[12.9,-0.5,0.5],[16.2,-1.1,0.5],[18.8,-1.1,0.5],[21.5,-1.1,0.5],[24.2,-1.1,0.5],[26.9,-1.1,0.5],[30.2,-1.1,0.5],[33.4,-1.1,0.5],[36.6,-1.1,0.5],[39.8,-0.5,0.5],[43.1,0.5,0.5],[46.3,1.6,0.5],[49.5,2.7,0.5],[53.3,4.3,0.5],[56.5,5.9,0.5],[60.3,8.6,0.5],[64.1,11.3,0.5],[67.3,14.5,0.5],[71.1,17.2,0.5],[74.3,20.5,0.5],[77.5,23.2,0.5],[80.8,26.9,0.5],[84.0,31.2,0.5],[86.7,35.5,0.5],[89.4,39.8,0.5],[92.1,43.6,0.5],[94.8,48.5,0.5],[97.4,53.8,0.5],[99.6,59.2,0.5],[102.3,63.5,0.5],[105.0,67.3,0.5],[107.1,72.1,0.5],[108.8,78.1,0.5],[110.9,83.4,0.5],[112.0,87.8,0.5],[113.6,91.5,0.5],[114.7,95.3,0.5],[115.8,100.1,0.5],[116.8,104.4,0.5],[117.4,109.8,0.5],[117.9,113.6,0.5],[117.9,117.9,0.5],[117.9,122.2,0.5],[117.4,127.6,0.5],[116.8,133.5,0.5],[115.2,138.9,0.5],[113.6,144.3,0.5],[110.9,150.7,0.5],[108.2,157.2,0.5],[105.0,162.6,0.5],[101.2,168.5,0.5],[97.4,175.0,0.5],[93.7,180.9,0.5],[88.8,185.7,0.5],[84.5,190.0,0.5],[80.2,193.3,0.5],[76.5,197.0,0.5],[72.7,200.8,0.5],[68.9,204.0,0.5],[64.1,207.3,0.5],[59.8,210.0,0.5],[54.4,212.1,0.5],[50.1,214.3,0.5],[44.7,215.9,0.5],[39.3,217.0,0.5],[33.4,218.6,0.5],[28.5,219.1,0.5],[23.2,220.2,0.5],[17.8,220.7,0.5],[10.8,221.3,0.5],[5.4,221.3,0.5],[0.0,221.3,0.5],[-5.9,221.3,0.5],[-11.8,221.3,0.5],[-17.2,220.7,0.5],[-22.6,220.2,0.5],[-26.9,218.6,0.5],[-32.3,217.0,0.5],[-36.6,215.3,0.5],[-40.9,212.7,0.5],[-44.7,209.4,0.5],[-49.0,206.2,0.5],[-52.8,203.0,0.5],[-56.5,200.3,0.5],[-59.8,197.6,0.5],[-63.0,194.9,0.5],[-66.2,192.2,0.5],[-69.4,189.0,0.5],[-72.7,185.7,0.5],[-75.4,182.5,0.5],[-78.1,179.8,0.5],[-80.2,177.7,0.5],[-82.4,175.5,0.5],[-84.0,174.4,0.5],[-85.6,172.8,0.5],[-87.2,171.7,0.5],[-88.3,170.7,0.5],[-89.4,170.1,0.5],[-90.4,169.6,0.5],[-91.0,169.0,0.5],[-91.5,169.0,0.5],[-92.1,169.0,0.5],[-92.6,169.0,0.5],[-93.1,169.0,0.5],[-93.7,169.0,0.5],[-94.2,169.0,0.5],[-94.7,169.0,0.5],[-95.3,169.0,0.5],[-96.4,169.6,0.5],[-98.0,170.7,0.5]],{"id":"2CyQzS1T","expect":"none","points":[[0,0,0.5],[-2.7,0.0,0.5],[-8.6,0.0,0.5],[-16.7,0.0,0.5],[-22.6,0.0,0.5],[-26.9,0.0,0.5],[-30.7,0.5,0.5],[-35.0,1.1,0.5],[-39.8,2.2,0.5],[-44.1,3.2,0.5],[-48.4,4.3,0.5],[-52.8,5.4,0.5],[-56.5,7.0,0.5],[-60.3,8.6,0.5],[-68.4,11.8,0.5],[-72.1,14.0,0.5],[-76.4,15.1,0.5],[-79.1,16.7,0.5],[-82.4,17.8,0.5],[-85.1,18.8,0.5],[-87.7,19.9,0.5],[-91.0,21.5,0.5],[-93.7,22.1,0.5],[-95.8,23.2,0.5],[-98.5,24.8,0.5],[-101.2,25.8,0.5],[-103.9,27.5,0.5],[-106.6,30.2,0.5],[-109.3,32.8,0.5],[-112.0,36.1,0.5],[-114.1,39.3,0.5],[-116.8,42.5,0.5],[-119.5,46.8,0.5],[-122.7,52.2,0.5],[-125.4,58.1,0.5],[-128.1,63.0,0.5],[-130.3,66.2,0.5],[-131.9,71.1,0.5],[-133.0,75.4,0.5],[-134.0,80.8,0.5],[-134.6,84.5,0.5],[-134.6,88.3,0.5],[-134.6,90.4,0.5],[-134.6,94.2,0.5],[-134.6,98.0,0.5],[-133.5,103.4,0.5],[-132.4,107.1,0.5],[-130.8,110.9,0.5],[-128.7,113.6,0.5],[-127.0,116.3,0.5],[-124.9,118.4,0.5],[-122.7,121.7,0.5],[-120.6,124.9,0.5],[-118.4,128.1,0.5],[-115.7,131.9,0.5],[-113.6,135.1,0.5],[-111.4,137.8,0.5],[-108.7,140.5,0.5],[-106.6,143.2,0.5],[-104.4,145.9,0.5],[-101.7,149.1,0.5],[-99.6,152.4,0.5],[-96.9,155.6,0.5],[-94.7,158.8,0.5],[-92.6,161.5,0.5],[-90.4,163.7,0.5],[-87.7,165.8,0.5],[-85.6,168.0,0.5],[-82.9,170.7,0.5],[-80.2,173.4,0.5],[-77.5,176.6,0.5],[-74.8,179.3,0.5],[-72.7,182.0,0.5],[-70.5,184.1,0.5],[-67.8,185.7,0.5],[-65.7,187.9,0.5],[-63.0,189.5,0.5],[-60.8,191.1,0.5],[-58.1,192.2,0.5],[-56.0,193.8,0.5],[-53.3,195.4,0.5],[-51.1,196.5,0.5],[-49.0,198.1,0.5],[-46.8,199.2,0.5],[-45.2,200.3,0.5],[-43.6,200.8,0.5],[-42.5,201.3,0.5],[-41.4,201.9,0.5],[-40.4,201.9,0.5],[-39.8,201.9,0.5],[-38.8,201.9,0.5],[-38.2,201.9,0.5],[-37.1,200.8,0.5],[-35.5,200.3,0.5],[-34.4,198.7,0.5]],{"id":"2nr0Cz1c","expect":"triangle","points":[[0,0,0.5],[0.0,0.5,0.5],[-1.1,3.8,0.5],[-2.1,9.7,0.5],[-3.8,17.2,0.5],[-5.4,24.2,0.5],[-7.0,30.7,0.5],[-8.1,37.7,0.5],[-9.7,44.7,0.5],[-11.8,51.7,0.5],[-13.5,59.2,0.5],[-15.1,66.8,0.5],[-17.2,74.3,0.5],[-19.4,82.9,0.5],[-21.0,91.0,0.5],[-22.6,99.6,0.5],[-24.2,108.8,0.5],[-26.4,117.4,0.5],[-28.0,127.1,0.5],[-30.1,136.7,0.5],[-31.8,147.0,0.5],[-33.9,157.2,0.5],[-35.5,167.4,0.5],[-37.7,178.2,0.5],[-39.8,187.9,0.5],[-42.0,198.1,0.5],[-43.6,207.8,0.5],[-45.2,217.0,0.5],[-46.8,226.1,0.5],[-47.9,234.7,0.5],[-49.0,242.3,0.5],[-50.1,248.2,0.5],[-50.6,253.6,0.5],[-51.1,257.9,0.5],[-51.7,261.6,0.5],[-51.7,264.3,0.5],[-51.7,267.0,0.5],[-51.7,269.2,0.5],[-51.7,270.8,0.5],[-51.7,272.4,0.5],[-51.7,273.5,0.5],[-51.7,274.6,0.5],[-51.7,275.1,0.5],[-51.7,275.6,0.5],[-51.7,276.2,0.5],[-52.2,276.7,0.5],[-52.2,277.3,0.5],[-52.8,277.3,0.5],[-53.3,277.3,0.5],[-54.4,277.3,0.5],[-56.0,277.3,0.5],[-57.6,277.3,0.5],[-59.2,276.7,0.5],[-60.8,276.2,0.5],[-63.0,275.1,0.5],[-65.1,273.5,0.5],[-66.7,272.4,0.5],[-68.9,270.3,0.5],[-70.5,268.6,0.5],[-72.7,266.5,0.5],[-74.8,263.8,0.5],[-77.0,2
```

```
tests/fixtures/shape user strokes.json
```

```
,31.2,0.5],[50.1,33.4,0.5],[52.8,36.1,0.5],[55.5,39.3,0.5],[58.7,43.1,0.5],[61.9,46.8,0.5],[65.7,51.1,0.5],
,[70.0,56.0,0.5],[73.8,60.8,0.5],[77.5,66.8,0.5],[81.3,71.1,0.5],[85.1,75.9,0.5],[88.8,80.2,0.5],[92.6,84.
0,0.5],[96.4,87.8,0.5],[99.6,91.5,0.5],[102.3,94.2,0.5],[104.4,96.9,0.5],[106.6,99.1,0.5],[108.2,101.2,0.5
],[109.8,102.8,0.5],[111.4,104.4,0.5],[113.1,106.1,0.5],[114.7,107.1,0.5],[116.3,108.8,0.5],[117.4,109.8,0
.5],[119.0,110.9,0.5],[120.1,112.5,0.5],[121.1,113.6,0.5],[122.2,114.7,0.5],[123.3,115.8,0.5],[124.4,116.8
,0.5],[124.9,117.9,0.5],[126.0,118.4,0.5],[126.5,119.5,0.5],[127.1,120.6,0.5],[127.6,121.7,0.5],[128.1,122
.2,0.5],[128.7,122.7,0.5],[129.2,123.3,0.5],[129.8,123.8,0.5],[129.8,124.4,0.5],[130.3,124.9,0.5],[130.8,1
25.4,0.5],[130.8,126.0,0.5],[131.4,126.5,0.5],[131.4,127.1,0.5],[131.9,127.1,0.5],[132.4,127.1,0.5],[132.4
,127.6,0.5]]},{ "id": "p4h5sZtL", "expect": "none", "points": [[0,0,0.5],[0,0,3.2,0.5],[0,0,11.3,0.5],[0,0,22.6,
0.5],[0.5,34.5,0.5],[0,0,44.7,0.5],[-0.5,54.9,0.5],[-4.3,81.8,0.5],[-7.0,94.2,0.5],[-11.3,108.8,0.5],[-17.
2,121.7,0.5],[-21.5,137.3,0.5],[-26.4,151.3,0.5],[-31.8,163.1,0.5],[-37.7,174.4,0.5],[-41.4,185.2,0.5],[-4
5.2,192.2,0.5],[-48.4,200.8,0.5],[-50.6,207.8,0.5],[-53.3,211.0,0.5],[-55.4,213.7,0.5],[-58.1,217.0,0.5],[-
60.8,220.7,0.5],[-62.4,226.7,0.5],[-65.1,230.4,0.5],[-67.8,233.7,0.5],[-70.5,236.9,0.5],[-73.2,239.6,0.5]
],[-77.0,245.5,0.5],[-80.2,252.5,0.5],[-84.0,256.8,0.5],[-90.4,261.6,0.5],[-94.2,264.9,0.5],[-99.1,273.5,0.
5],[-101.7,279.9,0.5],[-106.6,282.6,0.5],[-113.0,285.9,0.5],[-115.7,289.1,0.5],[-119.0,292.3,0.5],[-121.1,
295.6,0.5],[-123.3,297.7,0.5],[-124.4,298.8,0.5],[-125.4,298.3,0.5],[-127.0,297.7,0.5],[-128.1,296.1,0.5],
[-129.7,293.9,0.5],[-130.8,290.7,0.5],[-131.9,287.5,0.5],[-132.4,284.8,0.5],[-133.5,281.6,0.5],[-134.0,276
.2,0.5],[-135.1,268.6,0.5],[-137.3,262.7,0.5],[-138.4,259.5,0.5],[-140.0,253.0,0.5],[-142.1,244.4,0.5],[-1
43.7,238.0,0.5],[-145.4,234.2,0.5],[-147.5,224.5,0.5],[-150.7,215.9,0.5],[-153.4,210.5,0.5],[-157.7,201.9,
0.5],[-161.5,192.7,0.5],[-164.7,186.3,0.5],[-168.0,176.0,0.5],[-171.2,167.4,0.5],[-176.0,160.4,0.5],[-182.
0,149.1,0.5],[-186.3,140.5,0.5],[-190.6,130.8,0.5],[-194.9,120.6,0.5],[-201.9,112.0,0.5],[-207.8,100.7,0.5
],[-212.1,92.1,0.5],[-217.0,82.4,0.5],[-222.3,72.7,0.5],[-227.2,65.1,0.5],[-233.1,54.4,0.5],[-236.3,45.8,0
.5],[-240.1,38.2,0.5],[-243.9,26.9,0.5],[-248.7,19.4,0.5],[-254.6,11.8,0.5],[-257.3,2.2,0.5],[-260.0,-3.2,
0.5],[-262.7,-7.0,0.5],[-265.9,-13.5,0.5],[-268.6,-19.9,0.5],[-270.8,-24.2,0.5],[-273.5,-27.5,0.5],[-275.6
,-29.6,0.5],[-277.2,-30.7,0.5],[-278.3,-33.4,0.5],[-279.4,-35.0,0.5],[-279.4,-36.6,0.5],[-279.9,-37.7,0.5]
],[-279.4,-36.1,0.5]]},{ "id": "DcQ00TW5", "expect": "line", "points": [[0,0,0.5],[-0.5,0.0,0.5],[-1.1,1.1,0.5],
[-3.2,4.3,0.5],[-6.5,10.2,0.5],[-11.8,16.2,0.5],[-16.7,22.1,0.5],[-22.6,28.5,0.5],[-26.9,35.0,0.5],[-32.8,4
0.9,0.5],[-37.7,46.8,0.5],[-42.5,53.8,0.5],[-46.3,59.8,0.5],[-50.1,64.1,0.5],[-53.3,68.9,0.5],[-56.5,73.2,
0.5],[-58.7,78.1,0.5],[-60.8,81.8,0.5],[-62.4,85.1,0.5],[-63.5,87.8,0.5],[-65.1,91.0,0.5],[-66.2,95.8,0.5]
],[-67.8,100.7,0.5],[-69.4,105.5,0.5],[-71.1,109.3,0.5],[-73.2,113.6,0.5],[-75.4,119.0,0.5],[-77.5,124.9,0.
5],[-80.2,129.7,0.5],[-82.4,133.0,0.5],[-84.5,136.2,0.5],[-86.7,139.4,0.5],[-88.8,142.1,0.5],[-90.4,144.8,
0.5],[-92.1,147.5,0.5],[-92.6,150.2,0.5],[-93.7,151.8,0.5],[-94.2,152.9,0.5],[-94.2,153.4,0.5],[-94.2,154.
0,0.5],[-94.2,153.4,0.5],[-93.7,151.8,0.5],[-93.1,149.7,0.5],[-92.6,147.5,0.5]]},{ "id": "8Lt5tBvG", "expect"
: "line", "points": [[0,0,0.5],[-2.7,0.5,0.5],[-8.1,1.1,0.5],[-15.1,2.7,0.5],[-21.0,4.3,0.5],[-24.2,5.4,0.5],
[-26.9,5.9,0.5],[-28.5,6.5,0.5],[-29.6,6.5,0.5],[-30.1,6.5,0.5],[-30.1,5.9,0.5],[-29.1,3.2,0.5],[-28.0,-1.
1,0.5],[-25.3,-5.4,0.5],[-23.1,-9.1,0.5],[-20.5,-12.9,0.5],[-16.7,-16.1,0.5],[-12.4,-19.4,0.5],[-8.1,-23.7
,0.5],[-4.3,-28.5,0.5],[-1.1,-33.4,0.5],[3.2,-36.6,0.5],[7.0,-40.4,0.5],[11.9,-44.7,0.5],[15.6,-50.1,0.5],
[19.4,-54.9,0.5],[22.6,-59.2,0.5],[26.9,-63.0,0.5],[31.8,-66.2,0.5],[36.6,-71.1,0.5],[39.8,-76.4,0.5],[43.
1,-81.8,0.5],[46.3,-85.6,0.5],[49.5,-89.4,0.5],[51.7,-91.0,0.5],[64.6,-106.6,0.5],[71.1,-114.7,0.5],[73.8,
-116.3,0.5],[76.5,-120.1,0.5],[79.7,-126.0,0.5],[82.9,-129.7,0.5],[85.6,-135.1,0.5],[88.8,-137.3,0.5],[91.
5,-140.0,0.5],[92.6,-141.6,0.5],[94.2,-144.8,0.5],[95.8,-148.6,0.5],[97.4,-154.5,0.5],[98.5,-157.2,0.5],[9
9.1,-159.4,0.5],[100.1,-161.5,0.5],[99.1,-159.4,0.5],[99.6,-159.9,0.5],[99.6,-160.4,0.5],[100.1,-161.5,0.5
],[100.1,-162.6,0.5],[100.7,-163.7,0.5],[100.7,-164.7,0.5],[100.7,-166.4,0.5],[100.7,-167.4,0.5],[100.7,-1
68.5,0.5],[100.7,-169.0,0.5],[100.7,-170.1,0.5],[100.7,-170.7,0.5],[100.7,-171.2,0.5],[100.1,-171.7,0.5]]]
},{ "id": "d8LxKjmq", "expect": "line", "points": [[0,0,0.5],[0,0,2.7,0.5],[1.1,8.1,0.5],[4.3,15.6,0.5],[9.2,22.1
,0.5],[13.5,26.9,0.5],[17.2,30.2,0.5],[20.5,33.4,0.5],[23.7,36.1,0.5],[27.5,38.2,0.5],[32.3,40.4,0.5],[37.
7,43.6,0.5],[42.5,46.8,0.5],[46.8,50.6,0.5],[52.8,54.4,0.5],[59.2,57.6,0.5],[65.1,59.8,0.5],[70.5,61.9,0.5]
],[73.5,64.1,0.5],[82.9,66.8,0.5],[89.4,69.5,0.5],[95.3,72.1,0.5],[101.8,75.4,0.5],[106.6,78.6,0.5],[110.9
,80.8,0.5],[114.7,82.9,0.5],[117.9,85.1,0.5],[121.7,86.7,0.5],[124.9,88.3,0.5],[129.2,90.4,0.5],[133.5,92.
1,0.5],[136.2,93.7,0.5]]},{ "id": "ioP3qKqj", "expect": "none", "points": [[0,0,0.5],[2.7,0.0,0.5],[9.2,0.5,0.5]
,[17.2,1.1,0.5],[23.2,1.6,0.5],[27.5,2.2,0.5],[30.2,2.7,0.5],[33.4,3.2,0.5],[37.2,3.8,0.5],[40.4,4.3,0.5],
[43.6,4.8,0.5],[47.4,5.9,0.5],[51.7,6.5,0.5],[56.0,7.5,0.5],[59.8,8.6,0.5],[62.5,9.7,0.5],[66.8,11.3,0.5],
[72.1,12.9,0.5],[77.5,14.5,0.5],[81.8,16.2,0.5],[86.1,18.3,0.5],[90.5,21.0,0.5],[95.8,23.7,0.5],[101.2,26.
4,0.5],[107.1,29.6,0.5],[112.0,33.4,0.5],[117.4,37.7,0.5],[122.8,42.5,0.5],[128.1,47.9,0.5],[133.0,53.3,0.
5],[138.4,58.7,0.5],[143.7,64.1,0.5],[149.1,70.5,0.5],[154.0,77.5,0.5],[158.8,84.5,0.5],[163.7,91.5,0.5],[
169.1,99.1,0.5],[173.9,107.1,0.5],[176.1,108.2,0.5]]},{ "id": "ebg4dcUY", "expect": "none", "points": [[0,0,0.5]
,[0,0,1.1,0.5],[1.1,4.8,0.5],[1.6,11.8,0.5],[2.2,20.5,0.5],[2.7,29.6,0.5],[3.2,39.8,0.5],[3.2,68.4,0.5],[2
.7,84.5,0.5],[2.2,98.0,0.5],[1.6,112.0,0.5],[0,0,124.4,0.5],[-1.1,137.3,0.5],[-2.1,148.6,0.5],[-3.8,160.4,
0.5],[-4.8,171.2,0.5],[-5.9,182.5,0.5],[-7.5,194.4,0.5],[-9.1,205.7,0.5],[-10.8,216.4,0.5],[-11.8,228.3,0.
5],[-13.5,238.0,0.5],[-14.5,247.6,0.5],[-15.6,257.9,0.5],[-17.2,266.5,0.5],[-18.8,276.2,0.5],[-19.4,283.7,
0.5],[-21.0,290.2,0.5],[-22.1,295.0,0.5],[-22.6,301.5,0.5],[-23.1,305.3,0.5],[-23.1,308.5,0.5],[-23.7,311.
2,0.5],[-23.7,313.3,0.5],[-23.7,314.9,0.5],[-23.7,312.8,0.5],[-23.7,313.9,0.5],[-23.7,314.4,0.5],[-24.2,31
4.4,0.5],[-24.8,312.8,0.5],[-26.4,309.6,0.5],[-28.5,304.2,0.5],[-30.1,298.8,0.5],[-31.8,293.4,0.5],[-32.8,
286.9,0.5],[-34.4,279.4,0.5],[-35.5,271.9,0.5],[-36.6,264.3,0.5],[-37.7,256.8,0.5],[-39.3,248.7,0.5],[-40.
4,240.6,0.5],[-42.0,232.0,0.5],[-44.1,223.4,0.5],[-45.8,214.3,0.5],[-47.9,204.6,0.5],[-50.1,195.4,0.5],[-5
2.2,185.7,0.5],[-54.4,175.5,0.5],[-56.5,165.3,0.5],[-58.7,155.1,0.5],[-65.1,127.6,0.5],[-68.4,115.2,0.5],
[-71.6,104.4,0.5],[-75.4,94.2,0.5],[-78.1,84.0,0.5],[-81.3,73.8,0.5],[-84.0,64.1,0.5],[-86.7,53.8,0.5],[-89
.4,45.2,0.5],[-93.1,36.1,0.5],[-96.4,26.9,0.5],[-100.1,19.4,0.5],[-102.8,10.8,0.5],[-106.1,2.7,0.5],[-108.
7,-4.3,0.5],[-110.9,-11.8,0.5],[-113.6,-19.9,0.5],[-116.3,-25.8,0.5],[-119.5,-30.1,0.5],[-121.7,-37.1,0.5],
[-124.4,-44.1,0.5],[-127.0,-47.9,0.5],[-129.2,-53.3,0.5],[-130.3,-57.1,0.5],[-131.9,-63.0,0.5],[-133.0,-6
6.2,0.5],[-134.0,-69.4,0.5],[-135.1,-72.1,0.5],[-136.2,-74.3,0.5],[-137.3,-77.0,0.5],[-137.8,-79.1,0.5],[-
138.4,-80.8,0.5],[-138.9,-82.4,0.5],[-138.9,-84.0,0.5],[-138.4,-81.8,0.5],[-138.9,-82.4,0.5]]},{ "id": "GtIw
z09M", "expect": "line", "points": [[0,0,0.5],[2.2,1.1,0.5],[7.0,3.2,0.5],[12.9,6.5,0.5],[17.2,10.2,0.5],[21.0
,12.9,0.5],[24.2,15.6,0.5],[26.9,17.8,0.5],[30.7,20.5,0.5],[35.0,23.7,0.5],[40.9,26.9,0.5],[46.3,30.7,0.5]
],[51.7,34.5,0.5],[58.1,38.8,0.5],[65.1,43.1,0.5],[71.6,47.9,0.5],[78.6,52.8,0.5],[85.6,57.6,0.5],[92.6,61.
4,0.5],[99.1,65.7,0.5],[106.6,70.0,0.5],[113.6,73.8,0.5],[121.1,79.1,0.5],[128.7,84.5,0.5],[136.8,89.9,0.5]
],[144.3,96.4,0.5],[152.4,102.8,0.5],[159.9,110.4,0.5],[179.8,131.9,0.5],[189.5,144.8,0.5],[197.0,159.4,0.
5],[204.6,173.4,0.5],[211.6,187.9,0.5],[218.6,202.4,0.5],[225.6,214.3,0.5],[232.0,225.6,0.5],[236.9,234.2,
```

```

    0.5], [243.3, 240.6, 0.5], [242.8, 240.1, 0.5]]]]}

=====
// === END FILE: tests/fixtures/shape_user_strokes.json ===
=====

=====
// === START FILE: tests/history.test.ts ===
// File 155 of 194 | Lines: 140 | Size: 4503 bytes
=====

/**
 * Undo history persistence – no DOM, no network.
 *
 * History lives on its own localStorage key (never inside the workspace
 * payload), capped at 100 undo steps, oldest-first shrinking under a byte
 * budget. Fakes `window.localStorage` on globalThis like notebook.test.ts.
 */

import assert from "node:assert/strict";

import {
  saveHistory,
  loadHistory,
  flushHistorySave,
  restorableHistory,
  HISTORY_PAST_MAX,
  type SavedHistoryEntry,
} from "../src/lib/whiteboard/persistence";
import type { WhiteboardElement } from "../src/lib/whiteboard/types";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}\n      ${err as Error}.message`);
  }
}

const g = globalThis as unknown as { window?: unknown };
const prevWindow = g.window;
const store: Record<string, string> = {};
const normalWindow = {
  localStorage: {
    getItem: (k: string) => (k in store ? store[k] : null),
    setItem: (k: string, v: string) => {
      store[k] = String(v);
    },
    removeItem: (k: string) => {
      delete store[k];
    },
  },
};
g.window = normalWindow;

function entry(boardId: string, at: number, fill = "x"): SavedHistoryEntry {
  return {
    boardId,
    pageId: `pg_${at}`,
    elements: [{ id: `e${at}`, type: "stroke", note: fill } as unknown as WhiteboardElement],
    at,
  };
}

console.log(`\n■■ Undo history persistence ■■`);

test("round trip keeps entries in time order", () => {
  saveHistory({ past: [entry("b", 1), entry("b", 2)], future: [entry("b", 3)] });
  const back = loadHistory();
  assert.ok(back, "something must load");
  assert.deepEqual(back.past.map((e) => e.at), [1, 2]);
  assert.deepEqual(back.future.map((e) => e.at), [3]);
});

```

```

    assert.equal(back.past[0].boardId, "b");
  });

test("past caps at 100, newest kept", () => {
  const past = Array.from({ length: 150 }, (_, i) => entry("b", i));
  saveHistory({ past, future: [] });
  const back = loadHistory();
  assert.ok(back, "something must load");
  assert.equal(back.past.length, HISTORY_PAST_MAX);
  assert.equal(back.past[0].at, 50, "oldest dropped first");
  assert.equal(back.past[back.past.length - 1].at, 149, "newest kept");
});

test("byte budget shrinks oldest first and keeps the newest step", () => {
  const big = "y".repeat(30 * 1024);
  const past = Array.from({ length: 100 }, (_, i) => entry("b", i, big));
  saveHistory({ past, future: [] });
  const raw = store["z-whiteboard-history-v1"];
  assert.ok(raw, "something must be written");
  assert.ok(raw.length <= 1_500_000, `payload ${raw.length} must fit the budget`);
  const back = loadHistory();
  assert.ok(back && back.past.length < 100, "oldest steps were shed");
  assert.equal(
    back.past[back.past.length - 1].at,
    99,
    "the newest step always survives"
  );
});

test("corrupt entries are rejected, valid ones kept", () => {
  g.window = {
    localStorage: {
      getItem: () => JSON.stringify({ past: [{ nope: 1 }, entry("b", 7)], future: "junk" }),
      setItem: (k: string, v: string) => {
        store[k] = String(v);
      },
      removeItem: (k: string) => {
        delete store[k];
      },
    },
  };
  const back = loadHistory();
  assert.ok(back, "load must not throw the whole history away");
  assert.equal(back.past.length, 1);
  assert.equal(back.past[0].at, 7);
  assert.deepEqual(back.future, []);
  g.window = normalWindow;
});

test("restorableHistory drops deleted boards and caps", () => {
  const ghost = Array.from({ length: 120 }, (_, i) => entry("gone", i));
  const mine = Array.from({ length: 120 }, (_, i) => entry("b", 1000 + i));
  const out = restorableHistory(["b"], { past: [...ghost, ...mine], future: ghost });
  assert.ok(out.past.every((e) => e.boardId === "b"), "no steps for deleted boards");
  assert.equal(out.past.length, HISTORY_PAST_MAX);
  assert.deepEqual(out.future, [], "future for deleted boards is dropped too");
});

test("flushHistorySave writes through immediately", () => {
  flushHistorySave(() => ({ past: [entry("b", 9)], future: [] }));
  const back = loadHistory();
  assert.equal(back?.past.length, 1);
  assert.equal(back?.past[0].at, 9);
});

test("missing key loads as null", () => {
  delete store["z-whiteboard-history-v1"];
  assert.equal(loadHistory(), null);
});

g.window = prevWindow;

console.log(`\n ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);

```

```

=====
// === END FILE: tests/history.test.ts ===
=====

```

```
=====
// == START FILE: tests/image_placement.test.ts ==
// File 156 of 194 | Lines: 146 | Size: 4919 bytes
=====

/**
 * Inserting an image: where it lands, how big it is, how it resizes and turns.
 *
 * Reported on a Galaxy Tab S9: an added image "is just added to a random
 * place", could not be placed where wanted, and could not be resized or
 * rotated. These pin the placement rules that fix the first half.
 */

import assert from "node:assert/strict";
import {
  placeImage,
  lockAspect,
  snapRotation,
  IMAGE_FIT_FRACTION,
  IMAGE_MIN_SIDE,
} from "../src/lib/whiteboard/placement";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(` ${err as Error}.message`);
  }
}

const A4 = { width: 794, height: 1123 };
const near = (a: number, b: number, eps = 1e-6) => Math.abs(a - b) <= eps;

console.log(`\n■■■ Image placement ■■■`);

test("lands in the middle of what you can SEE on the page", () => {
  // Looking at the lower half of the sheet.
  const r = placeImage({
    naturalWidth: 400, naturalHeight: 300,
    camera: { x: 397, y: 800, zoom: 1 },
    viewport: { width: 794, height: 600 },
    page: A4,
  });
  const cy = r.y + r.height / 2;
  // Visible part of the sheet is y 500..1100 -> centre 800.
  assert.ok(near(cy, 800, 0.5), `centre y ${cy}, expected 800`);
});

test("never lands off the sheet, even when the view is mostly desk", () => {
  // Camera far to the right: only a sliver of the page is visible.
  const r = placeImage({
    naturalWidth: 1000, naturalHeight: 800,
    camera: { x: 1100, y: 500, zoom: 1 },
    viewport: { width: 800, height: 800 },
    page: A4,
  });
  assert.ok(r.x >= 0 && r.y >= 0, "top-left on the paper");
  assert.ok(r.x + r.width <= A4.width + 1e-6, "right edge on the paper");
  assert.ok(r.y + r.height <= A4.height + 1e-6, "bottom edge on the paper");
});

test("a photo larger than the view is fitted, keeping its proportions", () => {
  const r = placeImage({
    naturalWidth: 4000, naturalHeight: 3000,
    camera: { x: 397, y: 561, zoom: 1 },
    viewport: { width: 794, height: 1123 },
    page: A4,
  });
  assert.ok(r.width <= A4.width * IMAGE_FIT_FRACTION + 1e-6, `width ${r.width}`);
  assert.ok(near(r.width / r.height, 4000 / 3000, 1e-9), "aspect ratio preserved");
});
```

```
});

test("a small image is not blown up past its own pixels", () => {
  const r = placeImage({
    naturalWidth: 120, naturalHeight: 80,
    camera: { x: 397, y: 561, zoom: 1 },
    viewport: { width: 794, height: 1123 },
    page: A4,
  });
  assert.equal(r.width, 120);
  assert.equal(r.height, 80);
});

test("a tiny icon is enlarged to something you can actually grab", () => {
  const r = placeImage({
    naturalWidth: 16, naturalHeight: 16,
    camera: { x: 0, y: 0, zoom: 1 },
    viewport: { width: 800, height: 800 },
    page: null,
  });
  assert.ok(Math.min(r.width, r.height) >= IMAGE_MIN_SIDE - 1e-9);
});

test("a paste at the pointer centres on the pointer, then stays on the paper", () => {
  const r = placeImage({
    naturalWidth: 200, naturalHeight: 100,
    camera: { x: 397, y: 561, zoom: 1 },
    viewport: { width: 794, height: 1123 },
    page: A4,
    at: { x: 790, y: 20 }, // right at the top-right corner
  });
  assert.ok(near(r.x + r.width, A4.width), "pushed back inside the right edge");
  assert.ok(near(r.y, 0), "pushed back inside the top edge");
});

test("zoomed out, it still fits the visible page (region is in WORLD units)", () => {
  const r = placeImage({
    naturalWidth: 3000, naturalHeight: 3000,
    camera: { x: 397, y: 561, zoom: 0.5 },
    viewport: { width: 800, height: 800 },
    page: A4,
  });
  // At zoom 0.5 the view spans 1600 world units, so the whole page is visible
  // and the limit is the page itself.
  assert.ok(r.width <= A4.width * IMAGE_FIT_FRACTION + 1e-6);
});

console.log("\n■■■ Resize and rotate ■■■");

test("dragging a corner keeps a photo's proportions", () => {
  const s = lockAspect(400, 300, 600, 310);
  assert.ok(near(s.width / s.height, 400 / 300, 1e-9));
  assert.ok(near(s.width, 600), "follows the axis moved most");
});

test("shrinking below the minimum stops at the minimum, still in proportion", () => {
  const s = lockAspect(400, 200, 2, 1);
  assert.ok(near(Math.min(s.width, s.height), 20));
  assert.ok(near(s.width / s.height, 2, 1e-9));
});

test("rotation snaps to straight when you are within a few degrees", () => {
  const deg = (d: number) => (d * Math.PI) / 180;
  assert.ok(near(snapRotation(deg(2.5)), 0));
  assert.ok(near(snapRotation(deg(88)), deg(90)));
  assert.ok(near(snapRotation(deg(-3)), 0));
});

test("rotation is left alone when you clearly meant an angle", () => {
  const deg = (d: number) => (d * Math.PI) / 180;
  assert.ok(near(snapRotation(deg(30)), deg(30)));
});

console.log(`\n ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/image_placement.test.ts ===
```

```
=====

=====
// === START FILE: tests/image_studio.test.ts ===
// File 157 of 194 | Lines: 160 | Size: 6186 bytes
=====

/**
 * Image editing and locked-items mode.
 * - Edits are descriptions: an unedited picture draws exactly as before.
 * - Crop keeps the picture's scale and centre; flips carry the crop with them.
 * - Locked-items mode picks ONLY locked things; normal mode never does.
 */

import assert from "node:assert/strict";
import {
  boxForCrop,
  centeredAspectCrop,
  dragCrop,
  fitAspect,
  flipCrop,
  imageLayout,
  imageSvgMarkup,
  isFullCrop,
  isUnedited,
  MIN_CROP,
  normalizeCrop,
  quarterTurn,
} from "../src/lib/whiteboard/imageEdit";
import { fitWithin } from "../src/lib/whiteboard/imageImport";
import { pickableNow, type ImageElement } from "../src/lib/whiteboard/types";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` X ${name}`);
    console.error(`   ${err as Error}.message`);
  }
}

const near = (a: number, b: number, eps = 1e-6) => Math.abs(a - b) < eps;

console.log(String.fromCharCode(10) + "■■ Image studio & locked items ■■");

function photo(extra: Partial<ImageElement> = {}): ImageElement {
  return {
    id: "p1",
    type: "image",
    x: 100,
    y: 100,
    width: 400,
    height: 300,
    rotation: 0,
    opacity: 1,
    createdAt: 0,
    updatedAt: 0,
    src: "",
    naturalWidth: 4000,
    naturalHeight: 3000,
    ...extra,
  };
}

test("a picture with no edits counts as unedited (drawn the old way)", () => {
  assert.equal(isUnedited(photo()), true);
  assert.equal(isUnedited(photo({ crop: { x: 0, y: 0, w: 1, h: 1 }, filter: "none", frame: "square" })), true);
  ;
  assert.equal(isUnedited(photo({ flipX: true })), false);
  assert.equal(isUnedited(photo({ filter: "document" })), false);
  assert.equal(isUnedited(photo({ borderWidth: 2 })), false);
});
```

```

test("crops are clamped inside the picture and never vanish", () => {
  const c = normalizeCrop({ x: 0.9, y: -1, w: 0.5, h: 0 });
  assert.ok(c.x + c.w <= 1 + 1e-9 && c.y >= 0 && c.h >= MIN_CROP, JSON.stringify(c));
  assert.deepEqual(normalizeCrop(undefined), { x: 0, y: 0, w: 1, h: 1 });
  assert.deepEqual(normalizeCrop({ x: NaN, y: 0, w: 1, h: 1 }), { x: 0, y: 0, w: 1, h: 1 });
});

test("cropping keeps the scale and the centre: it takes away, never zooms", () => {
  const el = photo();
  const box = boxForCrop(el, { x: 0.25, y: 0.25, w: 0.5, h: 0.5 });
  assert.ok(near(box.width, 200) && near(box.height, 150), JSON.stringify(box));
  assert.ok(near(box.x + box.width / 2, 300) && near(box.y + box.height / 2, 250));
  // Uncropping a cropped picture brings back the whole photo at the same scale.
  const cropped = photo({ ...box, crop: { x: 0.25, y: 0.25, w: 0.5, h: 0.5 } });
  const back = boxForCrop(cropped, { x: 0, y: 0, w: 1, h: 1 });
  assert.ok(near(back.width, 400) && near(back.height, 300), JSON.stringify(back));
});

test("the crop is the inner viewBox, in the photo's own pixels", () => {
  const L = imageLayout(photo({ crop: { x: 0.1, y: 0.2, w: 0.5, h: 0.4 } }));
  assert.ok(near(L.viewBox.x, 400) && near(L.viewBox.y, 600) && near(L.viewBox.w, 2000) && near(L.viewBox.h, 1200));
  assert.equal(L.flip, "");
  assert.match(imageLayout(photo({ flipX: true })).flip, /scale\(-1 1\)/);
});

test("mirroring carries the kept window with the picture", () => {
  const c = { x: 0.1, y: 0.2, w: 0.3, h: 0.4 };
  const fx = flipCrop(c, "x");
  assert.ok(near(fx.x, 0.6) && fx.y === 0.2);
  assert.deepEqual(flipCrop(fx, "x").x.toFixed(6), c.x.toFixed(6));
  assert.ok(near(flipCrop(c, "y").y, 0.4));
});

test("quarter turns snap a hand-rotated picture and stay bounded", () => {
  assert.ok(near(quarterTurn(0, 1), Math.PI / 2));
  assert.ok(near(quarterTurn(0.1, 1), Math.PI / 2), "0.1 rad snaps to 0 first");
  let r = 0;
  for (let i = 0; i < 4; i++) r = quarterTurn(r, 1);
  assert.equal(r, 0);
  assert.ok(near(quarterTurn(0, -1), -Math.PI / 2));
});

test("dragging a corner resizes; dragging the middle moves; both stay inside", () => {
  const start = { x: 0.2, y: 0.2, w: 0.5, h: 0.5 };
  const se = dragCrop(start, "se", 0.1, 0.05, null);
  assert.ok(near(se.w, 0.6) && near(se.h, 0.55) && se.x === 0.2);
  const nw = dragCrop(start, "nw", 0.9, 0.9, null);
  assert.ok(nw.w >= MIN_CROP - 1e-9 && nw.h >= MIN_CROP - 1e-9, JSON.stringify(nw));
  const mv = dragCrop(start, "move", 5, -5, null);
  assert.ok(near(mv.x, 0.5) && near(mv.y, 0) && mv.w === 0.5);
});

test("a fixed ratio holds while dragging and when chosen", () => {
  const sq = centeredAspectCrop(1);
  assert.deepEqual(sq, { x: 0, y: 0, w: 1, h: 1 });
  const wide = centeredAspectCrop(2);
  assert.ok(near(wide.w / wide.h, 2) && near(wide.y, 0.25));
  const d = dragCrop({ x: 0.1, y: 0.1, w: 0.4, h: 0.4 }, "se", 0.3, 0.0, 1.5);
  assert.ok(near(d.w / d.h, 1.5), JSON.stringify(d));
  const f = fitAspect({ x: 0, y: 0, w: 1, h: 1 }, 0.5);
  assert.ok(near(f.w, 0.5) && near(f.h, 1));
  assert.equal(isFullCrop(f), false);
});

test("export draws the same crop, flip, look and frame as the canvas", () => {
  const svg = imageSvgMarkup(
    photo({ crop: { x: 0, y: 0, w: 0.5, h: 0.5 }, flipY: true, filter: "document", frame: "circle", borderWidth: 3 }),
    ""
  );
  assert.match(svg, /viewBox="0 0 2000 1500"/);
  assert.match(svg, /scale\(-1 -1\)/);
  assert.match(svg, /grayscale\(\(1\)/);
  assert.match(svg, /<ellipse/);
  assert.match(svg, /stroke-width="3"/);
});

```

```

test("big camera photos are scaled to 2560 px on the long side, small ones untouched", () => {
  assert.deepEqual(fitWithin(8000, 6000), { w: 2560, h: 1920 });
  assert.deepEqual(fitWithin(1200, 900), { w: 1200, h: 900 });
  assert.deepEqual(fitWithin(3000, 6000), { w: 1280, h: 2560 });
});

test("locked-items mode picks ONLY locked things; normal selection never does", () => {
  assert.equal(pickableNow({ locked: true }, false), false);
  assert.equal(pickableNow({}, false), true);
  assert.equal(pickableNow({ locked: true }, true), true);
  assert.equal(pickableNow({ locked: false }, true), false);
});

console.log(` ${passed} passed, ${failed} failed`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/image_studio.test.ts ===
=====

=====
// === START FILE: tests/interactive_knowledge_graph.test.ts ===
// File 158 of 194 | Lines: 233 | Size: 8698 bytes
=====

/**
 * Interactive Knowledge Graph Intelligence & Gesture Verification Tests
 * =====
 * Tests for:
 * 1. Tag clustering (#tags) node and edge generation in buildGraph
 * 2. Backwards compatibility of buildGraph with existing callers
 * 3. Rich node metadata (snippet, tags, folderName)
 * 4. Neighborhood extraction (filterGraphNeighborhood at 1-hop and 2-hop)
 * 5. Gesture disambiguation: Drag deadband (>6px) vs Tap selection
 * 6. Multi-touch pinch-to-zoom math & focal point preservation
 * 7. Bounding-box Auto-Fit (Zoom to Extents) calculation
 */

import assert from "node:assert/strict";
import {
  buildGraph,
  filterGraphNeighborhood,
  type GraphData,
} from "../src/lib/whiteboard/links";
import type { Board, TextElement, StickyElement, Folder } from "../src/lib/whiteboard/types";

let passed = 0;
let failed = 0;

async function test(name: string, fn: () => void | Promise<void>) {
  try {
    await fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`   ${err as Error}.message`);
  }
}

function makeBoard(id: string, name: string, folderId: string | null = null, tags: string[] = []): Board {
  return {
    id,
    name,
    folderId,
    tags,
    elements: [],
    camera: { x: 0, y: 0, zoom: 1 },
    updatedAt: Date.now(),
    createdAt: Date.now(),
  };
}

function makeTextElement(id: string, text: string): TextElement {
  return {
    id,

```

```

    type: "text",
    x: 100,
    y: 100,
    width: 250,
    height: 80,
    rotation: 0,
    opacity: 1,
    text,
    color: "#1e293b",
    fontSize: 16,
    fontFamily: "Inter",
    fontWeight: 400,
    align: "left",
    createdAt: Date.now(),
    updatedAt: Date.now(),
  });
}

async function runTests() {
  console.log("=== Testing Interactive Knowledge Graph Engine ===");

  await test("1. buildGraph backwards compatibility with 1, 2, or 3 arguments", () => {
    const b1 = makeBoard("b1", "Alpha");
    const b2 = makeBoard("b2", "Beta");
    b1.elements = [makeTextElement("t1", "Check out [[Beta]] for details")];

    // Call with 1 arg
    const g1 = buildGraph([b1, b2]);
    assert.ok(g1.nodes.length >= 2, "g1 has at least 2 nodes");
    assert.ok(g1.edges.some((e) => e.target === "b2"), "edge points to b2");

    // Call with 2 args
    const g2 = buildGraph([b1, b2], []);
    assert.equal(g2.nodes.length, g1.nodes.length);

    // Call with 3 args
    const folders: Folder[] = [{ id: "f1", name: "Projects", createdAt: 1, parentId: null, expanded: true }];
    const g3 = buildGraph([b1, b2], [], folders);
    assert.ok(g3.nodes.length >= 2);
  });

  await test("2. buildGraph extracts rich snippet, tags, and folderName metadata", () => {
    const folders: Folder[] = [{ id: "f1", name: "Physics Department", createdAt: 1, parentId: null, expanded: true }];
    const b1 = makeBoard("b1", "Quantum Mechanics", "f1", ["quantum", "physics"]);
    b1.elements = [
      makeTextElement("t1", "Schrodinger equation describes wave function dynamics."),
    ];

    const g = buildGraph([b1], [], folders);
    const node = g.nodes.find((n) => n.id === "b1");
    assert.ok(node, "b1 node exists");
    assert.equal(node!.folderName, "Physics Department", "folderName correctly resolved");
    assert.deepEqual(node!.tags, ["quantum", "physics"], "tags array preserved");
    assert.ok(node!.snippet?.includes("Schrodinger equation"), "snippet extracted from element text");
  });

  await test("3. buildGraph with includeTags creates emerald tag nuclei & connecting edges", () => {
    const b1 = makeBoard("b1", "Board One", null, ["exam2026", "math"]);
    const b2 = makeBoard("b2", "Board Two", null, ["exam2026", "cs"]);
    b2.elements = [makeTextElement("t2", "Review #algorithms before deadline.")];

    const g = buildGraph([b1, b2], [], [], { includeTags: true });

    // Check tag node existence
    const tagExam = g.nodes.find((n) => n.id === "tag:exam2026");
    assert.ok(tagExam, "tag:exam2026 node created");
    assert.equal(tagExam!.kind, "tag");
    assert.equal(tagExam!.color, "#10b981", "emerald tag color");
    assert.equal(tagExam!.name, "#exam2026");

    const tagAlgo = g.nodes.find((n) => n.id === "tag:algorithms");
    assert.ok(tagAlgo, "tag:algorithms parsed from element text created");

    // Check edges connecting to tag
    const edgesToExam = g.edges.filter((e) => e.target === "tag:exam2026");
    assert.equal(edgesToExam.length, 2, "both b1 and b2 connect to tag:exam2026");
    assert.ok(edgesToExam.some((e) => e.source === "b1"));
  });
}

```

```

    assert.ok(edgesToExam.some((e) => e.source === "b2"));
  });

  await test("4. filterGraphNeighborhood extracts exact 1-hop and 2-hop subgraphs", () => {
    const b1 = makeBoard("b1", "Center");
    const b2 = makeBoard("b2", "Hop 1");
    const b3 = makeBoard("b3", "Hop 2");
    const b4 = makeBoard("b4", "Isolated");

    b1.elements = [makeTextElement("t1", "Links to [[Hop 1]]")];
    b2.elements = [makeTextElement("t2", "Links to [[Hop 2]]")];

    const globalGraph = buildGraph([b1, b2, b3, b4]);

    // 1-Hop around b1
    const sub1 = filterGraphNeighborhood(globalGraph, "b1", 1);
    const sub1Ids = new Set(sub1.nodes.map((n) => n.id));
    assert.ok(sub1Ids.has("b1"), "contains center");
    assert.ok(sub1Ids.has("b2"), "contains 1-hop neighbor b2");
    assert.ok(!sub1Ids.has("b3"), "does not contain 2-hop neighbor b3 at depth 1");
    assert.ok(!sub1Ids.has("b4"), "does not contain isolated node b4");

    // 2-Hop around b1
    const sub2 = filterGraphNeighborhood(globalGraph, "b1", 2);
    const sub2Ids = new Set(sub2.nodes.map((n) => n.id));
    assert.ok(sub2Ids.has("b1"));
    assert.ok(sub2Ids.has("b2"));
    assert.ok(sub2Ids.has("b3"), "contains 2-hop neighbor b3 at depth 2");
    assert.ok(!sub2Ids.has("b4"), "still excludes isolated node b4");
  });

  await test("5. Gesture deadband disambiguates Tap (<6px) vs Drag (>=6px)", () => {
    const isDrag = (dx: number, dy: number): boolean => {
      const travel = Math.hypot(dx, dy);
      return travel > 6;
    };

    // Micro-motion on stylus tap (hand tremors 1-3px)
    assert.equal(isDrag(1.5, 2.0), false, "small jitter (<6px) is NOT a drag -> triggers tap");
    assert.equal(isDrag(0, 5.8), false, "5.8px travel is NOT a drag -> triggers tap");

    // Intentional drag to rearrange node
    assert.equal(isDrag(12, 18), true, "21.6px travel IS a drag -> suppresses tap/navigation");
    assert.equal(isDrag(150, 0), true, "large pan IS a drag -> suppresses tap/navigation");
  });

  await test("6. Multi-touch pinch-to-zoom ratio & focal midpoint tracking", () => {
    // Initial touch: p1 = (100, 200), p2 = (300, 200) -> distance = 200, mid = (200, 200)
    const p1_start = { x: 100, y: 200 };
    const p2_start = { x: 300, y: 200 };
    const initialDist = Math.hypot(p2_start.x - p1_start.x, p2_start.y - p1_start.y);
    assert.equal(initialDist, 200);

    // Finger spread: p1 = (50, 200), p2 = (350, 200) -> distance = 300
    const p1_cur = { x: 50, y: 200 };
    const p2_cur = { x: 350, y: 200 };
    const currentDist = Math.hypot(p2_cur.x - p1_cur.x, p2_cur.y - p1_cur.y);
    assert.equal(currentDist, 300);

    const initialZoom = 1.0;
    const scale = currentDist / initialDist;
    assert.equal(scale, 1.5, "zoom scales up by 1.5x on spread");

    const newZoom = initialZoom * scale;
    assert.equal(newZoom, 1.5);
  });

  await test("7. Auto-Fit (Zoom to Extents) bounding box computation", () => {
    const testNodes = [
      { x: 100, y: 150 },
      { x: 500, y: 450 },
      { x: 300, y: 200 },
    ];
    const containerDims = { width: 1000, height: 700 };
    const padding = 70;

    let minX = Infinity, maxX = -Infinity, minY = Infinity, maxY = -Infinity;
    for (const n of testNodes) {

```

```
    if (n.x < minX) minX = n.x;
    if (n.x > maxX) maxX = n.x;
    if (n.y < minY) minY = n.y;
    if (n.y > maxY) maxY = n.y;
  }

  assert.equal(minX, 100);
  assert.equal(maxX, 500);
  assert.equal(minY, 150);
  assert.equal(maxY, 450);

  const graphWidth = maxX - minX; // 400
  const graphHeight = maxY - minY; // 300
  const availWidth = containerDims.width - padding * 2; // 860
  const availHeight = containerDims.height - padding * 2; // 560

  const targetZoom = Math.min(availWidth / graphWidth, availHeight / graphHeight);
  // 860 / 400 = 2.15; 560 / 300 = 1.866...
  assert.ok(targetZoom > 1.8 && targetZoom < 1.9, "calculates optimal zoom without clipping");
});

console.log(`\nResults: ${passed} passed, ${failed} failed`);
if (failed > 0) process.exit(1);
}

runTests();

=====
// === END FILE: tests/interactive_knowledge_graph.test.ts ===
=====

=====
// === START FILE: tests/m1_adversarial_spen_stress.test.ts ===
// File 159 of 194 | Lines: 667 | Size: 27897 bytes
=====

/**
 * Milestone 1 Empirical Adversarial Stress Test Suite
 * =====
 * Agent: m1_challenger_1 (Empirical Challenger & Adversarial Verifier)
 *
 * Target Verifications:
 * 1. S Pen High-Frequency 240Hz-360Hz Digitizer Burst Streams, Zero-Delay Sub-Events, and Micro-Loops
 * 2. Pointer ID Isolation & Capacitive Palm Contention under Simultaneous Move, Liftoff, and OS Cancel
 * 3. Hardware Prediction (getPredictedEvents) Overlay Isolation & History Non-Persistence
 * 4. Feather-Light Pressure (0.0), Single-Tap Dots, and Auto-Straightening 5px Deadband
 */

import assert from "node:assert/strict";
import {
  calibratePressure,
  computePressureFromSpeed,
  applyPenDynamics,
  buildStrokePath,
  getSvgPathFromStroke,
  getStrokeOptionsForStyle,
  pruneTerminalHooks,
  isStrokeStraight,
  straightenStrokePoints,
} from "../src/lib/whiteboard/stroke";
import {
  screenToWorld,
  type Viewport,
} from "../src/lib/whiteboard/geometry";
import {
  PEN_TYPE_PRESETS,
  type StrokeStyle,
  type StrokeElement,
  type Camera,
  type WhiteboardElement,
} from "../src/lib/whiteboard/types";
import {
  classifyTouch,
  contactLooksLikePalm,
  isRejectedTouch,
  type ContactGeometry,
  type PenProximity,
```

```

} from "../src/lib/whiteboard/palm";
import {
  stylusEraseTransition,
  isStylusEraseRequest,
  STYLUS_BARREL_BUTTON,
  type StylusButtonState,
} from "../src/lib/whiteboard/input";
import { useWhiteboard } from "../src/store/whiteboard";

let passedCount = 0;
let failedCount = 0;
let totalAssertions = 0;

function runTest(name: string, fn: () => void) {
  try {
    fn();
    passedCount++;
    console.log(` ✓ PASS: ${name}`);
  } catch (err: unknown) {
    failedCount++;
    console.error(` ✗ FAIL: ${name}`);
    console.error(`${(err as Error).message}`);
    if ((err as Error).stack) {
      console.error(`${(err as Error).stack?.split("\n").slice(1, 3).join("\n")}`);
    }
  }
}

function expect(condition: boolean, msg: string) {
  totalAssertions++;
  assert.ok(condition, msg);
}

function expectEqual<T>(actual: T, expected: T, msg: string) {
  totalAssertions++;
  assert.equal(actual, expected, msg);
}

console.log("\n=====");
console.log("■ EMPIRICAL ADVERSARIAL CHALLENGER: S PEN INKING & HARDWARE COALESCENCE (M1)");
console.log("=====\\n");

// =====
// SECTION 1: 240Hz-360Hz Digitizer Coalescence & Extreme Burst Ingestion
// =====
console.log("■ SECTION 1: 240Hz-360Hz Coalescence & Extreme Burst Ingestion ■");

runTest("1.1 360Hz continuous burst stream: 1,000 sub-events ingested with strict chronological order", () =>
{
  // Simulate 1,000 coordinate samples generated at 360Hz (dt = 2.77ms per sample)
  // along an Archimedean spiral: r(theta) = a + b*theta
  const totalSamples = 1000;
  const sampleRateHz = 360;
  const dtMs = 1000 / sampleRateHz;
  const startTime = 100000;

  interface RawSample {
    clientX: number;
    clientY: number;
    pressure: number;
    timeStamp: number;
    pointerType: string;
  }

  const generatedStream: RawSample[] = [];
  const cx = 500;
  const cy = 500;
  const b = 0.25;

  for (let i = 0; i < totalSamples; i++) {
    const t = startTime + i * dtMs;
    const theta = i * 0.05;
    const r = 10 + b * theta;
    const clientX = cx + r * Math.cos(theta);
    const clientY = cy + r * Math.sin(theta);
    // Oscillating realistic pen pressure
    const pressure = 0.2 + 0.5 * Math.sin(i * 0.02);
    generatedStream.push({

```

```

        clientX,
        clientY,
        pressure: Math.max(0.01, Math.min(1.0, pressure)),
        timeStamp: t,
        pointerType: "pen",
    });
}

// Simulate batching into 16.6ms display frames (each frame receives ~6 coalesced samples)
const frames: RawSample[][] = [];
let currentIdx = 0;
while (currentIdx < totalSamples) {
    const batchSize = Math.min(6, totalSamples - currentIdx);
    frames.push(generatedStream.slice(currentIdx, currentIdx + batchSize));
    currentIdx += batchSize;
}

expect(frames.length > 0, "Frames must be populated");

// Ingest stream simulating WhiteboardCanvas coalesced event processing
const ingestedPoints: Array<[number, number, number]> = [];
const tStart = performance.now();

for (const frame of frames) {
    // getCoalescedEvents() returns the batch
    const coalescedBatch = frame;
    for (const ev of coalescedBatch) {
        const p = calibratePressure(ev.pressure, 65, "standard");
        ingestedPoints.push([ev.clientX, ev.clientY, p]);
    }
}

const tDuration = performance.now() - tStart;

expectEqual(ingestedPoints.length, 1000, "All 1,000 samples must be ingested without dropping");
expect(tDuration < 50, `1,000 points ingestion must complete under 50ms (took ${tDuration.toFixed(2)}ms)`);

// Verify monotonicity and numerical validity
for (let i = 0; i < ingestedPoints.length; i++) {
    const [x, y, p] = ingestedPoints[i];
    expect(Number.isFinite(x) && Number.isFinite(y), `Coordinates at ${i} must be finite`);
    expect(p >= 0.02 && p <= 1.0, `Pressure at ${i} must be calibrated between 0.02 and 1.0 (got ${p})`);
}

// Tessellate stroke through perfect-freehand and getSvgPathFromStroke
const strokeEl: StrokeElement = {
    id: "burst-stroke",
    type: "stroke",
    x: 0,
    y: 0,
    rotation: 0,
    opacity: 1,
    points: ingestedPoints,
    color: "#000000",
    strokeWidth: 4,
    strokeStyle: "gel",
    smoothing: 50,
    createdAt: Date.now(),
    updatedAt: Date.now(),
};

const svgPath = buildStrokePath(strokeEl, false);
expect(svgPath.startsWith("M") && svgPath.endsWith("Z"), "Tessellated SVG path must be valid closed polygon");
expect(!svgPath.includes("NaN") && !svgPath.includes("Infinity"), "SVG path must contain no NaN or Infinity");
});

runTest("1.2 Zero-delay sub-events: duplicate coordinates (dt=0, dx=0, dy=0) do not cause division by zero", () => {
    // Simulate 50 sub-events where pen rests or digitizer reports duplicate coordinates
    const dupStream: Array<[number, number, number]> = [];
    for (let i = 0; i < 50; i++) {
        dupStream.push([150.0, 200.0, 0.45]);
    }

    // computePressureFromSpeed on zero-distance points
    const pSpeed = computePressureFromSpeed(dupStream);

```

```

expect(Number.isFinite(pSpeed), "computePressureFromSpeed must be finite on duplicate points");
expectEqual(pSpeed, 0.52, "Zero-distance points should yield default speed pressure 0.52");

// applyPenDynamics on duplicate points
const dynPoints = applyPenDynamics(dupStream, "ballpoint");
expectEqual(dynPoints.length, 50, "Dynamics must preserve point count");
for (const pt of dynPoints) {
  expect(Number.isFinite(pt[0]) && Number.isFinite(pt[1]) && Number.isFinite(pt[2]), "All dynamics values must be finite");
}

// getSvgPathFromStroke on duplicate points
const path = getSvgPathFromStroke(dupStream.map(([x, y]) => [x, y]), true);
expect(path.startsWith("M") && path.endsWith("Z"), "Path from duplicate points must be closed");
expect(!path.includes("NaN"), "Path must not contain NaN");
});

runTest("1.3 Micro-loops: ultra-tight 0.5px-2.0px cursive loops across all 8 pen engines", () => {
  const engines: StrokeStyle[] = [
    "ballpoint",
    "gel",
    "fineliner",
    "fountain",
    "brush",
    "marker",
    "highlighter",
    "pencil",
  ];

  // Micro-loop with radius 1.2px (rapid 360-degree direction reversal over 16 points)
  const radius = 1.2;
  const loopPoints: Array<[number, number, number]> = [];
  for (let i = 0; i < 16; i++) {
    const angle = (i / 16) * Math.PI * 2;
    loopPoints.push([100 + Math.cos(angle) * radius, 100 + Math.sin(angle) * radius, 0.5]);
  }

  for (const eng of engines) {
    const strokeEl: StrokeElement = {
      id: `micro-${eng}`,
      type: "stroke",
      x: 0,
      y: 0,
      rotation: 0,
      opacity: 1,
      points: loopPoints,
      color: "#0f172a",
      strokeWidth: 3,
      strokeStyle: eng,
      smoothing: 50,
      createdAt: 1,
      updatedAt: 1,
    };

    const pathData = buildStrokePath(strokeEl, false);
    expect(pathData.startsWith("M"), `${eng}: path must start with M`);
    expect(pathData.endsWith("Z"), `${eng}: path must end with Z`);
    expect(!pathData.includes("NaN"), `${eng}: path must not contain NaN on micro-loop`);
    expect(!pathData.includes("Infinity"), `${eng}: path must not contain Infinity on micro-loop`);
  }
});

runTest("1.4 Auto-straighten deadband under 240Hz micro-jitter: jitter within 5px preserves hold timer", () => {
  {
    // Anchor at (200, 200)
    const anchor = { x: 200, y: 200 };
    const HOLD_DEADBAND_PX = 5;

    // 10 high-frequency digitizer jitter samples around anchor
    const jitters = [
      { x: 200.8, y: 201.2 },
      { x: 202.1, y: 199.4 },
      { x: 198.5, y: 202.3 },
      { x: 203.2, y: 201.8 }, // dist = hypot(3.2, 1.8) = 3.67 <= 5
      { x: 201.0, y: 200.5 },
    ];

    for (const j of jitters) {

```

```

    const dist = Math.hypot(j.x - anchor.x, j.y - anchor.y);
    const exceedsDeadband = dist > HOLD_DEADBAND_PX;
    expect(!exceedsDeadband, `Micro-jitter (${j.x}, ${j.y}) dist=${dist.toFixed(2)} must NOT exceed deadband 5
    px`);
  }

  // Intentional movement exceeds deadband
  const intentionalMove = { x: 208, y: 206 };
  const distMove = Math.hypot(intentionalMove.x - anchor.x, intentionalMove.y - anchor.y);
  expect(distMove > HOLD_DEADBAND_PX, `Intentional move dist=${distMove.toFixed(2)} must exceed deadband`);
});

// =====
// SECTION 2: Drawing Pointer ID Isolation & Palm Contention Stress
// =====
console.log("\n■■ SECTION 2: Drawing Pointer ID Isolation & Palm Contention Stress ■■");

runTest("2.1 Pointer ID isolation: simultaneous palm touches, moves, liftoffs, and cancellations do not corrup
t pen stroke", () => {
  // Realistic simulation of the WhiteboardCanvas state machine and refs
  const state = {
    drawingPointerId: null as number | null,
    penActive: false,
    lastPenAt: 0,
    activeTouches: new Map<number, { x: number; y: number; startX: number; startY: number }>(),
    rejectedPalms: new Set<number>(),
    interaction: { kind: "idle" as "idle" | "drawing" | "panning" | "moving" },
    liveDraftStroke: null as { originX: number; originY: number; points: Array<[number, number, number]> } | n
    ull,
    committedStrokes: [] as Array<{ points: Array<[number, number, number]> }>,
    camera: { x: 0, y: 0, zoom: 1 },
  };

  const handlePointerDown = (e: { pointerId: number; pointerType: string; clientX: number; clientY: number; wi
    dth?: number; height?: number; pressure?: number }) => {
    const isPen = e.pointerType === "pen";
    const isTouch = e.pointerType === "touch";

    if (isPen) {
      state.penActive = true;
      state.lastPenAt = Date.now();
    }

    if (isTouch) {
      const verdict = classifyTouch(
        { width: e.width ?? 20, height: e.height ?? 20 },
        { penDown: state.penActive, lastPenAt: state.lastPenAt, now: Date.now() }
      );

      const isSecondFingertipPinch =
        !state.penActive &&
        verdict === "pen-recent" &&
        state.activeTouches.size >= 1 &&
        !contactLooksLikePalm({ width: e.width ?? 20, height: e.height ?? 20 });

      if ((verdict === "pen-down" || verdict === "pen-recent") && !isSecondFingertipPinch) {
        state.rejectedPalms.add(e.pointerId);
        state.activeTouches.set(e.pointerId, { x: e.clientX, y: e.clientY, startX: e.clientX, startY: e.client
          Y });
        return; // Rejected palm contact returns immediately
      }

      state.activeTouches.set(e.pointerId, { x: e.clientX, y: e.clientY, startX: e.clientX, startY: e.clientY
        });
    }

    // Inking tool touchdown
    if (isPen) {
      state.drawingPointerId = e.pointerId;
      state.interaction = { kind: "drawing" };
      state.liveDraftStroke = {
        originX: e.clientX,
        originY: e.clientY,
        points: [[0, 0, calibratePressure(e.pressure ?? 0.5, 65, "standard")]],
      };
    }
  };
});

```

```

const handlePointerMove = (e: { pointerId: number; pointerType: string; clientX: number; clientY: number; pressure?: number }) => {
  // Multi-touch pan/zoom guard (Defect D5)
  const nonPalmTouches = Array.from(state.activeTouches.entries())
    .filter(([id]) => !state.rejectedPalms.has(id))
    .map(([t]) => t);

  if (
    nonPalmTouches.length === 2 &&
    !state.penActive &&
    state.drawingPointerId === null &&
    state.interaction.kind !== "drawing"
  ) {
    // Pan camera
    state.camera.x += 10;
    return;
  }

  if (state.interaction.kind === "drawing") {
    // R1-DEF-02: Isolate pen stream from palm moves
    if (state.drawingPointerId !== null && e.pointerId !== state.drawingPointerId) {
      return; // Palm movement completely ignored
    }

    const live = state.liveDraftStroke;
    if (!live) return;
    const p = calibratePressure(e.pressure ?? 0.5, 65, "standard");
    live.points.push([e.clientX - live.originX, e.clientY - live.originY, p]);
  }
};

const handlePointerUp = (e: { pointerId: number; pointerType: string }) => {
  state.activeTouches.delete(e.pointerId);
  state.rejectedPalms.delete(e.pointerId);

  if (e.pointerType === "pen") {
    state.penActive = false;
    state.lastPenAt = Date.now();
  }

  if (state.interaction.kind === "drawing") {
    // R1-DEF-02: Ignore pointerup from non-drawing pointers (e.g. palm lifting off)
    if (state.drawingPointerId !== null && e.pointerId !== state.drawingPointerId) {
      return; // Palm liftoff does NOT commit pen stroke
    }

    state.drawingPointerId = null;
    if (state.liveDraftStroke && state.liveDraftStroke.points.length >= 1) {
      state.committedStrokes.push({ points: state.liveDraftStroke.points });
    }
    state.liveDraftStroke = null;
    state.interaction = { kind: "idle" };
  }
};

const handlePointerCancel = (e: { pointerId: number; pointerType: string }) => {
  state.activeTouches.delete(e.pointerId);
  state.rejectedPalms.delete(e.pointerId);

  if (state.interaction.kind === "drawing") {
    // R1-DEF-02: Strict Pointer ID Isolation for Inking Cancellation
    if (state.drawingPointerId !== null && e.pointerId !== state.drawingPointerId) {
      return; // OS cancelled palm touch: DO NOT CANCEL ACTIVE PEN INKING!
    }
    state.drawingPointerId = null;
    state.liveDraftStroke = null;
    state.interaction = { kind: "idle" };
  }
};

// STEP 1: Stylus touchdown at (100, 100)
handlePointerDown({ pointerId: 42, pointerType: "pen", clientX: 100, clientY: 100, pressure: 0.5 });
expectEqual(state.drawingPointerId, 42, "drawingPointerId must be 42");
expectEqual(state.interaction.kind, "drawing", "Interaction must be drawing");
expectEqual(state.liveDraftStroke?.points.length, 1, "Live draft must have 1 point");

// STEP 2: Simultaneous capacitive palm touches land (pointerId 101, 102)
handlePointerDown({ pointerId: 101, pointerType: "touch", clientX: 450, clientY: 350, width: 50, height: 40

```

```

    });
    handlePointerDown({ pointerId: 102, pointerType: "touch", clientX: 470, clientY: 360, width: 55, height: 45
    });
    expect(state.rejectedPalms.has(101), "Palm 101 must be in rejectedPalms");
    expect(state.rejectedPalms.has(102), "Palm 102 must be in rejectedPalms");

    // STEP 3: Palm touches move erratically across screen
    handlePointerMove({ pointerId: 101, pointerType: "touch", clientX: 480, clientY: 390 });
    handlePointerMove({ pointerId: 102, pointerType: "touch", clientX: 510, clientY: 410 });
    // Camera must NOT have panned
    expectEqual(state.camera.x, 0, "Camera must not pan from dual palm contacts during inking");
    // Stroke must NOT have received palm coordinates
    expectEqual(state.liveDraftStroke?.points.length, 1, "Stroke must not include palm move coordinates");

    // STEP 4: Stylus draws valid cursive segment
    handlePointerMove({ pointerId: 42, pointerType: "pen", clientX: 110, clientY: 105, pressure: 0.55 });
    handlePointerMove({ pointerId: 42, pointerType: "pen", clientX: 120, clientY: 115, pressure: 0.60 });
    expectEqual(state.liveDraftStroke?.points.length, 3, "Pen points must be added to live stroke");

    // STEP 5: Palm 101 lifts off (pointerup on palm)
    handlePointerUp({ pointerId: 101, pointerType: "touch" });
    expectEqual(state.drawingPointerId, 42, "drawingPointerId must still be 42 after palm lift");
    expectEqual(state.interaction.kind, "drawing", "Interaction must still be drawing");
    expectEqual(state.committedStrokes.length, 0, "Stroke must not be prematurely committed by palm lift");

    // STEP 6: Android OS sends ACTION_CANCEL on Palm 102 (pointercancel)
    handlePointerCancel({ pointerId: 102, pointerType: "touch" });
    expectEqual(state.drawingPointerId, 42, "drawingPointerId must still be 42 after palm cancel");
    expectEqual(state.interaction.kind, "drawing", "Interaction must still be drawing after palm cancel");
    expect(state.liveDraftStroke !== null, "Live draft must not be destroyed by palm cancel");

    // STEP 7: Stylus continues drawing additional segment
    handlePointerMove({ pointerId: 42, pointerType: "pen", clientX: 130, clientY: 125, pressure: 0.65 });
    expectEqual(state.liveDraftStroke?.points.length, 4, "Stroke continued drawing smoothly");

    // STEP 8: Stylus finally lifts off (pointerup on pen)
    handlePointerUp({ pointerId: 42, pointerType: "pen" });
    expectEqual(state.drawingPointerId, null, "drawingPointerId reset to null");
    expectEqual(state.interaction.kind, "idle", "Interaction reset to idle");
    expectEqual(state.committedStrokes.length, 1, "Exactly 1 stroke committed on pen lift");
    expectEqual(state.committedStrokes[0].points.length, 4, "Committed stroke contains only the 4 pen points");

    // Verify NONE of the coordinates match palm coordinates
    const committedPts = state.committedStrokes[0].points;
    for (const pt of committedPts) {
        const worldX = pt[0] + 100;
        expect(worldX < 200, `Committed point X=${worldX} must belong to pen, not palm (450+)`);
    }
});

runTest("2.2 Context-aware fast-bypass: 800ms suppression protects resting palm but immediately allows 2-finger pinch", () => {
    const now = 20000;
    const lastPenLiftAt = now - 150; // 150ms after pen lift (inside 800ms window)

    // 1. Single resting palm lands (broad contact patch) -> REJECTED
    const palmGeometry: ContactGeometry = { width: 50, height: 45 };
    const palmVerdict = classifyTouch(palmGeometry, { penDown: false, lastPenAt: lastPenLiftAt, now });
    expectEqual(palmVerdict, "pen-recent", "Resting palm within 150ms must be pen-recent");
    expect(isRejectedTouch(palmGeometry, { penDown: false, lastPenAt: lastPenLiftAt, now }), "Palm must be rejected");

    // 2. Fingertip touch lands (small contact patch) within 150ms -> verdict is pen-recent
    const fingerGeometry: ContactGeometry = { width: 16, height: 16 };
    const fingerVerdict = classifyTouch(fingerGeometry, { penDown: false, lastPenAt: lastPenLiftAt, now });
    expectEqual(fingerVerdict, "pen-recent", "Fingertip within 150ms has base verdict pen-recent");

    // 3. Fast-bypass criteria: pen lifted (!penActive), touchVerdict === 'pen-recent', activeTouches >= 1, not palm-sized
    const penActive = false;
    const activeTouchesCount = 1; // 1 contact already present, 2nd arrives
    const isSecondFingertipPinch =
        !penActive &&
        fingerVerdict === "pen-recent" &&
        activeTouchesCount >= 1 &&
        !contactLooksLikePalm(fingerGeometry);

    expect(isSecondFingertipPinch, "Second fingertip must engage context-aware fast bypass immediately");

```

```

});

// =====
// SECTION 3: Hardware Prediction Isolation & History Non-Persistence
// =====
console.log("\n■■■ SECTION 3: Hardware Prediction Isolation & History Non-Persistence ■■■");

runTest("3.1 Predicted points are rendered in live draft but NEVER committed to document history on pointerup"
, () => {
  // Model WhiteboardCanvas predicted event handling
  const live = {
    originX: 100,
    originY: 100,
    points: [[0, 0, 0.5]] as Array<[number, number, number]>,
  };
  let predictedPoints: Array<[number, number, number]> = [];

  // Frame 1: Move with real points + predicted points from getPredictedEvents()
  const realEvent = { clientX: 120, clientY: 100, pressure: 0.6 };
  const predictedEvents = [
    { clientX: 128, clientY: 100, pressure: 0.6 },
    { clientX: 136, clientY: 100, pressure: 0.6 },
    { clientX: 144, clientY: 100, pressure: 0.6 },
  ];

  // Ingestion of real event
  live.points.push([realEvent.clientX - live.originX, realEvent.clientY - live.originY, realEvent.pressure]);

  // Speculative prediction filtering (1px <= dist <= 40px, max 3)
  const lastReal = [realEvent.clientX, realEvent.clientY, realEvent.pressure];
  const newPredictions: Array<[number, number, number]> = [];
  for (const pe of predictedEvents.slice(0, 3)) {
    const dist = Math.hypot(pe.clientX - lastReal[0], pe.clientY - lastReal[1]);
    if (dist >= 1.0 && dist <= 40.0) {
      newPredictions.push([pe.clientX - live.originX, pe.clientY - live.originY, pe.pressure]);
    }
  }
  predictedPoints = newPredictions;

  // Live RAF render points
  const renderPoints = predictedPoints.length > 0 ? [...live.points, ...predictedPoints] : live.points;
  expectEqual(renderPoints.length, 5, "Live render points must combine 2 real + 3 predicted points (5 total)");
  expectEqual(live.points.length, 2, "Real points buffer must contain ONLY 2 real points");

  // Frame 2: Stylus moves to (130, 100) with new predictions
  const realEvent2 = { clientX: 130, clientY: 100, pressure: 0.65 };
  const predictedEvents2 = [
    { clientX: 138, clientY: 100, pressure: 0.65 },
    { clientX: 146, clientY: 100, pressure: 0.65 },
  ];

  live.points.push([realEvent2.clientX - live.originX, realEvent2.clientY - live.originY, realEvent2.pressure]);
  const lastReal2 = [realEvent2.clientX, realEvent2.clientY, realEvent2.pressure];
  const newPredictions2: Array<[number, number, number]> = [];
  for (const pe of predictedEvents2.slice(0, 3)) {
    const dist = Math.hypot(pe.clientX - lastReal2[0], pe.clientY - lastReal2[1]);
    if (dist >= 1.0 && dist <= 40.0) {
      newPredictions2.push([pe.clientX - live.originX, pe.clientY - live.originY, pe.pressure]);
    }
  }
  predictedPoints = newPredictions2; // Replaces previous frame's predictions

  const renderPoints2 = predictedPoints.length > 0 ? [...live.points, ...predictedPoints] : live.points;
  expectEqual(renderPoints2.length, 5, "Live render now has 3 real + 2 predicted (5 total)");
  expectEqual(live.points.length, 3, "Real points buffer contains exactly 3 real points");

  // POINTERUP: Commit stroke
  predictedPoints = []; // Cleared on pointerup
  const cleanPoints = pruneTerminalHooks(live.points);
  const finalStroke: StrokeElement = {
    id: "commit-stroke-1",
    type: "stroke",
    x: live.originX,
    y: live.originY,
    rotation: 0,
    opacity: 1,
  };

```

```

    points: cleanPoints,
    color: "#000",
    strokeWidth: 3,
    strokeStyle: "gel",
    smoothing: 50,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  });

  // Assertions on committed stroke
  expectEqual(finalStroke.points.length, 3, "Committed stroke MUST have exactly 3 points");
  expectEqual(finalStroke.points[0][0], 0, "Origin X is 0");
  expectEqual(finalStroke.points[1][0], 20, "Point 1 X is 20");
  expectEqual(finalStroke.points[2][0], 30, "Point 2 X is 30 (last real point at 130 - 100)");

  // Verify none of the predicted X coordinates (28, 36, 44, 38, 46) exist in final stroke
  const forbiddenPredictedX = [28, 36, 44, 38, 46];
  for (const pt of finalStroke.points) {
    expect(!forbiddenPredictedX.includes(pt[0]), `Predicted coordinate X=${pt[0]} must NOT be in committed stroke`);
  }
});

runTest("3.2 Predicted points deadband & outlier clamping: discards < 1px jitter and > 40px outliers", () => {
  const lastReal: [number, number, number] = [100, 100, 0.5];
  const candidates = [
    { clientX: 100.4, clientY: 100.3, pressure: 0.5 }, // dist = 0.5px (< 1.0px) -> DISCARDED
    { clientX: 110.0, clientY: 100.0, pressure: 0.5 }, // dist = 10.0px (valid) -> KEPT
    { clientX: 125.0, clientY: 100.0, pressure: 0.5 }, // dist = 25.0px (valid) -> KEPT
    { clientX: 170.0, clientY: 100.0, pressure: 0.5 }, // dist = 70.0px (> 40.0px) -> DISCARDED
  ];

  const filteredPredictions: Array<[number, number, number]> = [];
  for (const pe of candidates.slice(0, 3)) {
    const dist = Math.hypot(pe.clientX - lastReal[0], pe.clientY - lastReal[1]);
    if (dist >= 1.0 && dist <= 40.0) {
      filteredPredictions.push([pe.clientX - lastReal[0], pe.clientY - lastReal[1], pe.pressure]);
    }
  }

  expectEqual(filteredPredictions.length, 2, "Exactly 2 valid predictions kept (jitter and wild outlier discarded)");
  expectEqual(filteredPredictions[0][0], 10.0, "First valid prediction at 10px");
  expectEqual(filteredPredictions[1][0], 25.0, "Second valid prediction at 25px");
});

// =====
// SECTION 4: Feather-Light Pressure & Touchdown Geometry
// =====
console.log("\n■■ SECTION 4: Feather-Light Pressure & Touchdown Geometry ■■");

runTest("4.1 S Pen rawPressure === 0.0 preserves hairline (0.02) without falling back to speed pressure (0.50)", () => {
  // S Pen feather-light touch
  const isPen = true;
  const rawEvPressure = 0.0;
  const rawP = isPen
    ? (typeof rawEvPressure === "number" ? rawEvPressure : 0.5)
    : (rawEvPressure > 0 ? rawEvPressure : 0.5);

  expectEqual(rawP, 0.0, "Raw pressure for pen must preserve 0.0");

  const calibrated = calibratePressure(rawP, 65, "standard");
  expectEqual(calibrated, 0.02, "calibrated pressure for 0.0 must be 0.02 hairline");
  expect(calibrated < 0.1, "Hairline pressure must be delicate (< 0.1), not thick fallback (0.5)");
});

runTest("4.2 Single-point tap touchdown (dot / niqqud / period) renders valid circular SVG dot", () => {
  const dotStroke: StrokeElement = {
    id: "dot-1",
    type: "stroke",
    x: 200,
    y: 300,
    rotation: 0,
    opacity: 1,
    points: [[0, 0, 0.5]], // Single point
    color: "#1e293b",
    strokeWidth: 4,
  };

```

```
        strokeStyle: "ballpoint",
        smoothing: 50,
        createdAt: Date.now(),
        updatedAt: Date.now(),
    };

    const dotPath = buildStrokePath(dotStroke, false);
    expect(dotPath.startsWith("M"), "Dot path must start with M");
    expect(dotPath.endsWith("Z"), "Dot path must end with Z");
    expect(dotPath.includes(" a ") || dotPath.includes("a "), "Dot path must contain arc commands for circle");
    expect(!dotPath.includes("NaN"), "Dot path must not contain NaN");
});

// =====
// SUMMARY & VERDICT EVALUATION
// =====
console.log("\n=====");
console.log(`■ ADVERSARIAL CHALLENGE EXECUTION SUMMARY`);
console.log(`  Tests Passed:      ${passedCount}`);
console.log(`  Tests Failed:      ${failedCount}`);
console.log(`  Total Assertions:   ${totalAssertions}`);
console.log("=====\\n");

if (failedCount > 0) {
    process.exit(1);
}

=====
// === END FILE: tests/m1_adversarial_spen_stress.test.ts ===
=====

=====
// === START FILE: tests/m1_challenger_2_adversarial.test.ts ===
// File 160 of 194 | Lines: 739 | Size: 27474 bytes
=====

/**
 * M1 CHALLENGER 2 EMPIRICAL ADVERSARIAL STRESS TEST SUITE
 * =====
 *
 * Verifies Milestone 1 implementation against:
 * 1. Defect D5: Dual palm rejection contention during active S Pen writing.
 *    Simulate two simultaneous palm contacts landing on glass while S Pen is inking.
 *    Verify 2-finger pan & pinch zoom are strictly blocked, canvas camera never moves,
 *    and pen inking stroke is never corrupted, truncated, or cancelled.
 * 2. R1-DEF-03: Post-stylus multi-touch unfreezing.
 *    Stress-test immediate responsiveness of two-finger pinch-to-zoom upon pen liftoff
 *    across multiple sub-800ms delays (10ms, 50ms, 150ms, 300ms, 790ms), while
 *    maintaining strict resting palm rejection for broad palm contacts.
 * 3. R1-DEF-05: Feather-light pressure transitions.
 *    Stress-test near-zero pressure values (0.0, 0.0001, 0.001, 0.005, 0.01) across
 *    all 8 pen engines and all response curves, verifying continuous hairline rendering
 *    without sudden jumps to 0.5 speed-derived pressure or NaN/infinite geometry.
 */

import assert from "node:assert/strict";
import {
    calibratePressure,
    applyPenDynamics,
    getStrokeOptionsForStyle,
    getSvgPathFromStroke,
} from "../src/lib/whiteboard/stroke";
import {
    classifyTouch,
    contactLooksLikePalm,
    PEN_SUPPRESSION_MS,
    PEN_FAST_BYPASS_MS,
    PALM_CONTACT_PX,
    type ContactGeometry,
    type PenProximity,
} from "../src/lib/whiteboard/palm";
import {
    PEN_TYPE_PRESETS,
    type StrokeStyle,
    type Camera,
} from "../src/lib/whiteboard/types";
import type { Viewport } from "../src/lib/whiteboard/geometry";
```

```

import { getStroke } from "perfect-freehand";

let totalAssertions = 0;
let passedAssertions = 0;

function check(condition: boolean, msg: string) {
  totalAssertions++;
  if (!condition) {
    console.error(` ■ FAILED [${totalAssertions}]: ${msg}`);
    throw new Error(`Assertion failed: ${msg}`);
  }
  passedAssertions++;
}

console.log("\n=====");
console.log(" ■ M1 CHALLENGER 2: ADVERSARIAL VERIFICATION & STRESS HARNESS");
console.log("=====\\n");

// =====
// SUITE 1: DEFECT D5 – DUAL PALM REJECTION CONTENTION DURING ACTIVE S PEN INKING
// =====
console.log("■■ Suite 1: Defect D5 – Palm Contention During S Pen Inking ■■");

/**
 * Canvas Inking & Gesture State Machine Harness
 * Replicates the exact state machine in WhiteboardCanvas.tsx (lines 1735–3450)
 */
class CanvasGestureStateMachine {
  penActive = false;
  lastPenAt = 0;
  drawingPointerId: number | null = null;
  interactionKind: "idle" | "drawing" | "panning" | "erasing" = "idle";

  // Camera state
  camera: Camera = { x: 0, y: 0, zoom: 1.0 };
  viewport: Viewport = { width: 1200, height: 800 };

  // Touches & Palms tracking
  activeTouches = new Map<number, { x: number; y: number; startX: number; startY: number }>();
  rejectedPalms = new Set<number>();
  lastTouchMid: { x: number; y: number } | null = null;
  pinchStartDistance: number | null = null;
  pinchStartZoom = 1.0;
  pinchEngaged = false;

  // Inking stroke buffer
  liveStrokePoints: Array<[number, number, number]> = [];
  committedStrokes: Array<Array<[number, number, number]>> = [];
  cancelledStrokeCount = 0;

  // Track pan calls
  panQueueCount = 0;
  totalPanDx = 0;
  totalPanDy = 0;

  handlePointerDown(e: {
    pointerId: number;
    pointerType: string;
    clientX: number;
    clientY: number;
    width?: number;
    height?: number;
    pressure?: number;
  }) {
    const isPen = e.pointerType === "pen";
    const isTouch = e.pointerType === "touch";

    if (isPen) {
      this.penActive = true;
      this.lastPenAt = Date.now();
    }

    let touchVerdict: string = "allow";
    if (isTouch) {
      touchVerdict = classifyTouch(
        { width: e.width, height: e.height },
        { penDown: this.penActive, lastPenAt: this.lastPenAt, now: Date.now() }
      );
    }
  }
}

```

```
const isSecondFingertipPinch =
  !this.penActive &&
  touchVerdict === "pen-recent" &&
  this.activeTouches.size >= 1 &&
  !contactLooksLikePalm({ width: e.width, height: e.height });

if ((touchVerdict === "pen-down" || touchVerdict === "pen-recent") && !isSecondFingertipPinch) {
  this.rejectedPalms.add(e.pointerId);
  this.activeTouches.set(e.pointerId, {
    x: e.clientX,
    y: e.clientY,
    startX: e.clientX,
    startY: e.clientY,
  });
  return; // Suppressed palm!
}

if (isSecondFingertipPinch) {
  for (const [id] of this.activeTouches.entries()) {
    this.rejectedPalms.delete(id);
  }
}

if (isTouch) {
  this.activeTouches.set(e.pointerId, {
    x: e.clientX,
    y: e.clientY,
    startX: e.clientX,
    startY: e.clientY,
  });
}

const validTouches = Array.from(this.activeTouches.entries())
  .filter(([id]) => !this.rejectedPalms.has(id))
  .map([, t] => t);

const currentTouchCount = validTouches.length;

// Multi-touch Pinch to Zoom and 2-Finger Pan Setup (Defect D5 guard)
if (
  currentTouchCount >= 2 &&
  !this.penActive &&
  this.drawingPointerId === null &&
  this.interactionKind !== "drawing"
) {
  const touches = validTouches;
  this.pinchStartDistance = Math.hypot(touches[0].x - touches[1].x, touches[0].y - touches[1].y);
  this.pinchStartZoom = this.camera.zoom;
  this.lastTouchMid = {
    x: (touches[0].x + touches[1].x) / 2,
    y: (touches[0].y + touches[1].y) / 2,
  };
}
return;
}

// Stylus Drawing Touchdown
if (isPen) {
  this.drawingPointerId = e.pointerId;
  const rawP = typeof e.pressure === "number" ? e.pressure : 0.5;
  const p = calibratePressure(rawP, 65, "standard");
  this.liveStrokePoints = [[e.clientX, e.clientY, p]];
  this.interactionKind = "drawing";
}
}

handlePointerMove(e: {
  pointerId: number;
  pointerType: string;
  clientX: number;
  clientY: number;
  pressure?: number;
}) {
  if (this.activeTouches.has(e.pointerId)) {
    const orig = this.activeTouches.get(e.pointerId)!;
    this.activeTouches.set(e.pointerId, { ...orig, x: e.clientX, y: e.clientY });
  }
}
```

```

// Multi-touch pinch & pan handling
const nonPalmTouches = Array.from(this.activeTouches.entries())
  .filter(([id]) => !this.rejectedPalms.has(id))
  .map(([, t]) => t);

if (
  nonPalmTouches.length === 2 &&
  !this.penActive &&
  this.drawingPointerId === null &&
  this.interactionKind !== "drawing"
) {
  const touches = nonPalmTouches;
  const currDist = Math.hypot(touches[0].x - touches[1].x, touches[0].y - touches[1].y);
  const midX = (touches[0].x + touches[1].x) / 2;
  const midY = (touches[0].y + touches[1].y) / 2;

  if (this.lastTouchMid) {
    const dx = midX - this.lastTouchMid.x;
    const dy = midY - this.lastTouchMid.y;
    this.panQueueCount++;
    this.totalPanDx += dx;
    this.totalPanDy += dy;
    this.camera.x += dx / this.camera.zoom;
    this.camera.y += dy / this.camera.zoom;
  }
  this.lastTouchMid = { x: midX, y: midY };

  if (this.pinchStartDistance && this.pinchStartDistance > 0) {
    const scale = currDist / this.pinchStartDistance;
    this.camera.zoom = Math.max(0.1, Math.min(8.0, this.pinchStartZoom * scale));
    this.pinchEngaged = true;
  }
  return;
}

// Inking handling
if (this.interactionKind === "drawing") {
  if (this.drawingPointerId !== null && e.pointerId !== this.drawingPointerId) {
    return; // R1-DEF-02: Isolated from palm moves
  }
  const rawP = e.pointerType === "pen" && typeof e.pressure === "number" ? e.pressure : 0.5;
  const p = calibratePressure(rawP, 65, "standard");
  this.liveStrokePoints.push([e.clientX, e.clientY, p]);
}

handlePointerUp(e: { pointerId: number; pointerType: string }) {
  this.activeTouches.delete(e.pointerId);
  this.rejectedPalms.delete(e.pointerId);
  if (e.pointerType === "pen") {
    this.penActive = false;
    this.lastPenAt = Date.now();
  }

  if (this.interactionKind === "drawing") {
    if (this.drawingPointerId !== null && e.pointerId !== this.drawingPointerId) {
      return; // Ignore non-drawing pointer liftoff
    }
    this.drawingPointerId = null;
    this.committedStrokes.push([...this.liveStrokePoints]);
    this.liveStrokePoints = [];
    this.interactionKind = "idle";
  }
}

handlePointerCancel(e: { pointerId: number; pointerType: string }) {
  this.activeTouches.delete(e.pointerId);
  this.rejectedPalms.delete(e.pointerId);
  if (e.pointerType === "pen") {
    this.penActive = false;
    this.lastPenAt = Date.now();
  }

  if (this.interactionKind === "drawing") {
    if (this.drawingPointerId !== null && e.pointerId !== this.drawingPointerId) {
      return; // Ignore palm cancellation
    }
  }
}

```

```

        this.drawingPointerId = null;
        this.liveStrokePoints = [];
        this.cancelledStrokeCount++;
        this.interactionKind = "idle";
    }
}
}

// Test 1.1: Two simultaneous palm contacts land while S Pen is active writing
(() => {
    const canvas = new CanvasGestureStateMachine();
    const PEN_ID = 1;
    const PALM_A_ID = 10;
    const PALM_B_ID = 11;

    // 1. S Pen touches down and starts writing
    canvas.handlePointerDown({
        pointerId: PEN_ID,
        pointerType: "pen",
        clientX: 100,
        clientY: 100,
        pressure: 0.45,
    });

    check(canvas.penActive === true, "penActive is true after S Pen touchdown");
    check(canvas.drawingPointerId === PEN_ID, "drawingPointerId bound to S Pen ID");
    check(canvas.interactionKind === "drawing", "Interaction kind is 'drawing'");
    check(canvas.liveStrokePoints.length === 1, "Initial stroke point registered");

    // Pen moves drawing a stroke
    canvas.handlePointerMove({
        pointerId: PEN_ID,
        pointerType: "pen",
        clientX: 110,
        clientY: 105,
        pressure: 0.50,
    });
    check(canvas.liveStrokePoints.length === 2, "Second stroke point recorded");

    // 2. TWO simultaneous palm contacts land on the glass while pen is writing
    // Palm A: broad fleshy thenar eminence (60x80px)
    canvas.handlePointerDown({
        pointerId: PALM_A_ID,
        pointerType: "touch",
        clientX: 500,
        clientY: 400,
        width: 60,
        height: 80,
    });

    // Palm B: side of hypothenar palm / wrist (55x70px)
    canvas.handlePointerDown({
        pointerId: PALM_B_ID,
        pointerType: "touch",
        clientX: 560,
        clientY: 450,
        width: 55,
        height: 70,
    });

    // Both palms must be marked as rejected
    check(canvas.rejectedPalms.has(PALM_A_ID), "Palm A added to rejectedPalms");
    check(canvas.rejectedPalms.has(PALM_B_ID), "Palm B added to rejectedPalms");
    check(canvas.pinchEngaged === false, "Pinch zoom not engaged on dual palm touchdown");

    // Initial camera coordinates before movement
    const initialCamX = canvas.camera.x;
    const initialCamY = canvas.camera.y;
    const initialZoom = canvas.camera.zoom;

    // 3. Both palms slide/wobble across glass while pen continues writing
    for (let step = 1; step <= 20; step++) {
        // Palm A moves
        canvas.handlePointerMove({
            pointerId: PALM_A_ID,
            pointerType: "touch",
            clientX: 500 + step * 5,
            clientY: 400 + step * 3,

```

```

});
// Palm B moves in opposite direction (simulating pinch/pan gesture)
canvas.handlePointerMove({
  pointerId: PALM_B_ID,
  pointerType: "touch",
  clientX: 560 - step * 5,
  clientY: 450 - step * 4,
});
// Pen moves normally writing cursive
canvas.handlePointerMove({
  pointerId: PEN_ID,
  pointerType: "pen",
  clientX: 110 + step * 2,
  clientY: 105 + Math.sin(step) * 5,
  pressure: 0.48,
});
}

// Camera must be 100% stationary! Canvas never moved under the pen!
check(canvas.camera.x === initialCamX, "Camera X strictly unchanged under dual palm motion");
check(canvas.camera.y === initialCamY, "Camera Y strictly unchanged under dual palm motion");
check(canvas.camera.zoom === initialZoom, "Camera zoom strictly unchanged under dual palm motion");
check(canvas.panQueueCount === 0, "Zero pan actions queued while inking");
check(canvas.pinchEngaged === false, "Pinch zoom never engaged");

// Inking stroke points must NOT contain any palm coordinates (500+, 400+)
check(canvas.liveStrokePoints.length === 22, "Stroke received only pen points (22 total)");
const hasPalmCoords = canvas.liveStrokePoints.some([x, y] => x >= 400 || y >= 300);
check(!hasPalmCoords, "Inking stroke contains ZERO palm coordinates");

// 4. Palm A lifts off while S Pen continues inking
canvas.handlePointerUp({ pointerId: PALM_A_ID, pointerType: "touch" });
check(canvas.interactionKind === "drawing", "Interaction remains 'drawing' after Palm A liftoff");
check(canvas.drawingPointerId === PEN_ID, "drawingPointerId remains PEN_ID after Palm A liftoff");
check(canvas.committedStrokes.length === 0, "Stroke NOT prematurely committed by Palm A liftoff");

// 5. Android sends ACTION_CANCEL (pointercancel) on Palm B
canvas.handlePointerCancel({ pointerId: PALM_B_ID, pointerType: "touch" });
check(canvas.interactionKind === "drawing", "Interaction remains 'drawing' after Palm B cancellation");
check(canvas.drawingPointerId === PEN_ID, "drawingPointerId remains PEN_ID after Palm B cancellation");
check(canvas.cancelledStrokeCount === 0, "Active pen stroke NOT destroyed by palm cancellation");

// 6. Pen writes final flourish and lifts off
canvas.handlePointerMove({
  pointerId: PEN_ID,
  pointerType: "pen",
  clientX: 160,
  clientY: 110,
  pressure: 0.30,
});
canvas.handlePointerUp({ pointerId: PEN_ID, pointerType: "pen" });

check(canvas.interactionKind === "idle", "Interaction returns to idle after pen lift");
check(canvas.committedStrokes.length === 1, "Stroke committed cleanly on pen lift");
check(canvas.committedStrokes[0].length === 23, "Committed stroke has all 23 pen points");
})();
console.log(" ✓ Defect D5: Dual palm landing during inking strictly blocked; camera stationary; stroke clean"
);

// Test 1.2: Fingertip-sized dual contacts (20x20px) landing while S Pen is writing
(() => {
  const canvas = new CanvasGestureStateMachine();
  const PEN_ID = 2;
  const FINGER_1 = 30;
  const FINGER_2 = 31;

  canvas.handlePointerDown({
    pointerId: PEN_ID,
    pointerType: "pen",
    clientX: 200,
    clientY: 200,
    pressure: 0.6,
  });

  // Two accidental small fingertip contacts arrive (e.g. knuckles)
  canvas.handlePointerDown({
    pointerId: FINGER_1,
    pointerType: "touch",
  });

```

```

    clientX: 400,
    clientY: 400,
    width: 20,
    height: 20,
  });
  canvas.handlePointerDown({
    pointerId: FINGER_2,
    pointerType: "touch",
    clientX: 450,
    clientY: 450,
    width: 22,
    height: 22,
  });

  // Even though width < 45, penDown === true forces "pen-down" verdict, blocking them
  check(canvas.rejectedPalms.has(FINGER_1), "Small touch 1 rejected while pen is active");
  check(canvas.rejectedPalms.has(FINGER_2), "Small touch 2 rejected while pen is active");

  // Simulated pinch gesture between finger 1 and finger 2
  canvas.handlePointerMove({ pointerId: FINGER_1, pointerType: "touch", clientX: 350, clientY: 350 });
  canvas.handlePointerMove({ pointerId: FINGER_2, pointerType: "touch", clientX: 500, clientY: 500 });

  check(canvas.camera.zoom === 1.0, "Zoom locked while pen is active, even for fingertip touches");
  check(canvas.panQueueCount === 0, "Pan locked while pen is active, even for fingertip touches");
})();
console.log(" ✓ Defect D5: Fingertip-sized dual contacts also strictly blocked while pen is active");

// =====
// SUITE 2: R1-DEF-03 – POST-STYLUS MULTI-TOUCH UNFREEZING & FAST-BYPASS
// =====
console.log("\n Suite 2: R1-DEF-03 – Post-Stylus Multi-Touch Unfreezing ");

// Test 2.1: Immediate responsiveness of two-finger pinch-to-zoom upon pen liftoff
(() => {
  const delaySamplesMs = [5, 15, 50, 100, 200, 350, 500, 750]; // All within 800ms suppression window

  for (const delayMs of delaySamplesMs) {
    const canvas = new CanvasGestureStateMachine();
    const PEN_ID = 5;
    const F1_ID = 51;
    const F2_ID = 52;

    // 1. S Pen inking
    canvas.handlePointerDown({
      pointerId: PEN_ID,
      pointerType: "pen",
      clientX: 100,
      clientY: 100,
      pressure: 0.5,
    });
    // Pen lifts off
    canvas.handlePointerUp({ pointerId: PEN_ID, pointerType: "pen" });

    // Artificially advance time by delayMs
    const tLift = canvas.lastPenAt;
    const now = tLift + delayMs;

    // Classify touch directly at this timestamp
    const prox: PenProximity = { penDown: false, lastPenAt: tLift, now };
    const verdict = classifyTouch({ width: 20, height: 20 }, prox);
    check(verdict === "pen-recent", `Standard classification at ${delayMs}ms is 'pen-recent'`);

    // 2. User immediately performs two-finger pinch
    // First finger lands:
    canvas.handlePointerDown({
      pointerId: F1_ID,
      pointerType: "touch",
      clientX: 300,
      clientY: 300,
      width: 22,
      height: 22,
    });

    // Second finger lands 5ms later:
    canvas.handlePointerDown({
      pointerId: F2_ID,
      pointerType: "touch",

```

```

    clientX: 400,
    clientY: 300,
    width: 24,
    height: 24,
  });

  // Verify fast-bypass un-rejected both fingers!
  check(!canvas.rejectedPalms.has(F1_ID), `F1 un-rejected by fast bypass at ${delayMs}ms`);
  check(!canvas.rejectedPalms.has(F2_ID), `F2 allowed by fast bypass at ${delayMs}ms`);
  check(canvas.pinchStartDistance === 100, `Pinch start distance initialized to 100px at ${delayMs}ms`);

  // 3. User pinches outward from 100px to 200px (2x zoom)
  canvas.handlePointerMove({
    pointerId: F1_ID,
    pointerType: "touch",
    clientX: 250,
    clientY: 300,
  });
  canvas.handlePointerMove({
    pointerId: F2_ID,
    pointerType: "touch",
    clientX: 450,
    clientY: 300,
  });

  // Zoom must respond immediately!
  check(Math.abs(canvas.camera.zoom - 2.0) < 1e-4, `Pinch zoom is immediately responsive at ${delayMs}ms (zoom: ${canvas.camera.zoom})`);
  check(canvas.pinchEngaged === true, `Pinch engaged flag is true at ${delayMs}ms`);
}
})();
console.log(" ✓ R1-DEF-03: Two-finger pinch-to-zoom unfreezes immediately across all sub-800ms delays (5ms-750ms)");

// Test 2.2: Resting palm rejection remains strictly preserved after pen lift
(() => {
  const canvas = new CanvasGestureStateMachine();
  const PEN_ID = 6;
  const PALM_1 = 61;
  const PALM_2 = 62;

  // Pen writes and lifts off
  canvas.handlePointerDown({ pointerId: PEN_ID, pointerType: "pen", clientX: 100, clientY: 100, pressure: 0.5 });
  canvas.handlePointerUp({ pointerId: PEN_ID, pointerType: "pen" });

  // 150ms after pen lift, user's resting palm lands between words (55x70px)
  canvas.handlePointerDown({
    pointerId: PALM_1,
    pointerType: "touch",
    clientX: 600,
    clientY: 500,
    width: 55,
    height: 70,
  });

  // Second palm contact from same hand
  canvas.handlePointerDown({
    pointerId: PALM_2,
    pointerType: "touch",
    clientX: 640,
    clientY: 540,
    width: 60,
    height: 75,
  });

  // Broad contacts MUST NOT fast-bypass! They must remain rejected!
  check(canvas.rejectedPalms.has(PALM_1), "Resting palm 1 remains rejected after pen lift");
  check(canvas.rejectedPalms.has(PALM_2), "Resting palm 2 remains rejected after pen lift");

  // Palms shift on the glass
  canvas.handlePointerMove({ pointerId: PALM_1, pointerType: "touch", clientX: 620, clientY: 510 });
  canvas.handlePointerMove({ pointerId: PALM_2, pointerType: "touch", clientX: 650, clientY: 560 });

  check(canvas.camera.zoom === 1.0, "Resting palm shifts do not zoom canvas");
  check(canvas.panQueueCount === 0, "Resting palm shifts do not pan canvas");
})();
console.log(" ✓ R1-DEF-03: Broad resting palm remains strictly rejected after pen lift (no false-positive byp
```

```

    ass)");

// =====
// SUITE 3: R1-DEF-05 — FEATHER-LIGHT PRESSURE CONTINUITY & HAIRLINE RENDERING
// =====
console.log("\n■ Suite 3: R1-DEF-05 — Feather-Light Pressure Transitions ■");

// Test 3.1: Raw pressure === 0.0 with S Pen digitizer
(() => {
    // S Pen hovering or ultra-feather touch reporting raw pressure 0.0
    const isPen = true;
    const rawP_pen = isPen ? (typeof (0.0 as number) === "number" ? 0.0 : 0.5) : (0.0 > 0 ? 0.0 : 0.5);
    check(rawP_pen === 0.0, "rawP preserves 0.0 for pen pointerType");

    const calP_0 = calibratePressure(rawP_pen, 65, "standard");
    check(calP_0 === 0.02, `calibratePressure(0.0) yields minimum hairline 0.02 (actual: ${calP_0})`);

    // Compare with defective legacy logic:
    const legacy_rawP = 0.0 > 0 ? 0.0 : 0.5; // Evaluates to 0.5!
    const legacy_calP = calibratePressure(legacy_rawP, 65, "standard");
    check(legacy_calP === 0.5, "Legacy logic produced sudden 25x jump to 0.50");
    check(calP_0 < 0.10, "R1-DEF-05 successfully keeps hairline pressure < 0.10");

    // Contrast with mouse/touch where pressure is 0:
    const isMouse = false;
    const rawP_mouse = isMouse ? 0.0 : (0.0 > 0 ? 0.0 : 0.5);
    check(rawP_mouse === 0.5, "Mouse/touch without pressure cleanly falls back to 0.5");
})();
console.log(" ✓ R1-DEF-05: Raw pressure 0.0 preserved for S Pen without 0.5 jump");

// Test 3.2: High-resolution pressure sweep (0.0000 to 0.1000) continuity & monotonicity
(() => {
    const SENSITIVITIES = [30, 50, 65, 80, 100];
    const CURVES: Array<"soft" | "standard" | "firm"> = ["soft", "standard", "firm"];

    for (const curve of CURVES) {
        for (const sens of SENSITIVITIES) {
            let prevCalP = calibratePressure(0.0, sens, curve);
            check(prevCalP >= 0.02, `Floor pressure >= 0.02 for ${curve}/${sens}`);

            // 10,000 sub-steps in the feather-light range [0, 0.1]
            for (let i = 1; i <= 1000; i++) {
                const rawP = i * 0.0001; // 0.0001, 0.0002, ..., 0.1000
                const calP = calibratePressure(rawP, sens, curve);

                // 1. Monotonicity: Calibrated pressure must be non-decreasing
                check(
                    calP >= prevCalP - 1e-9,
                    `Monotonicity violation at rawP=${rawP}: calP=${calP} < prev=${prevCalP} (${curve}/${sens})`
                );

                // 2. Continuity: No sudden jumps (> 0.01 between adjacent 0.0001 raw steps)
                const stepDelta = Math.abs(calP - prevCalP);
                check(
                    stepDelta < 0.01,
                    `Step discontinuity ${stepDelta} at rawP=${rawP} (${curve}/${sens})`
                );

                // 3. No abrupt jump from the 0.0 baseline in the feather-light region
                if (rawP < 0.02) {
                    const zeroBaseline = calibratePressure(0.0, sens, curve);
                    check(
                        Math.abs(calP - zeroBaseline) < 0.05,
                        `Feather-light pressure at ${rawP} unexpectedly jumped from 0.0 baseline ${zeroBaseline} to ${calP} (${curve}/${sens})`
                    );
                }
                prevCalP = calP;
            }
        }
    }
})();
console.log(" ✓ R1-DEF-05: 10,000-point pressure sweep proves strict monotonicity & sub-0.01 continuity");

// Test 3.3: All 8 Pen Engine Geometries under feather-light pressure (0.0, 0.001, 0.005)
(() => {

```

```

const ALL_ENGINES: StrokeStyle[] = [
  "ballpoint",
  "gel",
  "fineliner",
  "fountain",
  "brush",
  "marker",
  "highlighter",
  "pencil",
];

const nearZeroPressures = [0.0, 0.0005, 0.001, 0.005, 0.01];

for (const style of ALL_ENGINES) {
  for (const rawP of nearZeroPressures) {
    const calP = calibratePressure(rawP, 65, "standard");

    // Construct a 15-point cursive stroke with near-zero pressure
    const inputPoints: Array<number, number, number> = [];
    for (let i = 0; i < 15; i++) {
      const x = 50 + i * 8;
      const y = 50 + Math.sin(i * 0.5) * 10;
      inputPoints.push([x, y, calP]);
    }

    // 1. Apply pen dynamics
    const dynamicPoints = applyPenDynamics(inputPoints, style);
    check(dynamicPoints.length === 15, `Dynamics preserved 15 points for ${style}`);

    for (const pt of dynamicPoints) {
      check(Number.isFinite(pt[0]) && Number.isFinite(pt[1]) && Number.isFinite(pt[2]), `Finite coords for ${style}`);
      check(pt[2] >= 0.02 && pt[2] <= 1.0, `Pressure ${pt[2]} in valid [0.02, 1.0] range for ${style}`);
      check(pt[2] < 0.20, `Feather-light dynamics pressure ${pt[2]} remains hairline (<0.20) for ${style}`);
    }

    // 2. Generate polygon outline with perfect-freehand
    const strokeOpts = getStrokeOptionsForStyle(style, 4);
    const polygon = getStroke(dynamicPoints, strokeOpts as any);
    check(polygon.length >= 3, `Polygon outline generated for ${style} at rawP=${rawP}`);

    for (const polyPt of polygon) {
      check(Number.isFinite(polyPt[0]) && Number.isFinite(polyPt[1]), `Finite polygon vertex for ${style}`);
    }

    // 3. Generate SVG path
    const svgPath = getSvgPathFromStroke(polygon, true);
    check(svgPath.startsWith("M") && svgPath.endsWith("Z"), `Valid SVG path format for ${style}`);
    check(!svgPath.includes("NaN"), `No NaN in SVG path for ${style}`);
    check(!svgPath.includes("Infinity"), `No Infinity in SVG path for ${style}`);
    check(!svgPath.includes("undefined"), `No undefined in SVG path for ${style}`);
    check(svgPath.length > 20, `Non-empty SVG path for ${style}`);
  }
}

})();
console.log(" ✓ R1-DEF-05: All 8 pen engines render valid, non-degenerate hairline geometry under near-zero pressure");

// Test 3.4: High-speed flick with feather-light touch
(() => {
  // S Pen flick: 60px distance in single step with pressure 0.001
  const flickPoints: Array<number, number, number> = [
    [100, 100, 0.02],
    [160, 100, 0.02],
  ];

  const dynFlick = applyPenDynamics(flickPoints, "fountain");
  // Velocity thinning reduces pressure, but floor 0.02 must prevent vanishing
  check(dynFlick[1][2] >= 0.02, "Flick velocity thinning respects minimum floor 0.02");

  const opts = getStrokeOptionsForStyle("fountain", 4);
  const poly = getStroke(dynFlick, opts as any);
  const path = getSvgPathFromStroke(poly, true);

  check(path.startsWith("M") && path.endsWith("Z"), "High-speed flick produces valid closed SVG path");
  check(!path.includes("NaN"), "High-speed flick has no NaN");
})();
console.log(" ✓ R1-DEF-05: High-speed flick with feather-light pressure stays above floor without polygon col

```

```
lapse");

console.log("\n=====");
console.log(`    ■ ALL ADVERSARIAL STRESS TESTS PASSED: ${passedAssertions}/${totalAssertions} ASSERTIONS (100%)`);
console.log("=====\\n");

=====

// === END FILE: tests/m1_challenger_2_adversarial.test.ts ===

=====

// === START FILE: tests/milestone1.test.ts ===
// File 161 of 194 | Lines: 407 | Size: 16015 bytes
=====

import assert from "node:assert/strict";
import { getSvgPathFromStroke, buildStrokePath, pruneTerminalHooks, getStrokeOptionsForStyle } from "../src/lib/whiteboard/stroke";
import {
  hitTestElement,
  getElementBounds,
  getElementCenter,
  getNodeAnchorPorts,
  getResolvedAnchorPoint,
  getCurvedBezierPath,
  getOrthogonalPath,
  getStraightPath,
  getArrowheadPath,
  alignElements,
  distributeElements,
  screenToWorld,
  worldToScreen,
  clamp,
} from "../src/lib/whiteboard/geometry";
import { PEN_TYPE_PRESETS, type StrokeElement, type RectangleElement, type EllipseElement, type DiamondElement, type TriangleElement, type StarElement, type CloudElement, type LineElement, type ArrowElement } from "../src/lib/whiteboard/types";

console.log("=== Running Milestone 1 Test Suite ===");

// -----
// Test 1: getSvgPathFromStroke - Closed Midpoint B-Splines (Anti-Flap Geometry)
// -----
console.log("\\n[Test 1] Freehand Stroke Flap Fix & Midpoint B-Splines...");

// Empty points
assert.equal(getSvgPathFromStroke([]), "", "Empty points should produce empty string");

// 1 point
const p1Path = getSvgPathFromStroke([[10, 20]]);
assert.ok(p1Path.startsWith("M 9.00,20.00"), "1-point path should generate mini-circle");

// 2 points
const p2Path = getSvgPathFromStroke([[0, 0], [10, 10]]);
assert.equal(p2Path, "M 0.00,0.00 L 10.00,10.00 Z", "2-point closed path should be line with Z");

// Multi-point closed polygon outline (e.g. 4 points)
const poly = [
  [0, 0],
  [100, 0],
  [100, 100],
  [0, 100],
];
const closedPath = getSvgPathFromStroke(poly, true);

// Check starting point is midpoint of poly[0] and poly[3] = (0, 50)
assert.ok(closedPath.startsWith("M 0.00,50.00"), "Closed spline must start at midpoint of last and first vertices");

// This square's vertices are 90 degree turns – SHARP. A midpoint quadratic
// never passes through its control point, so rounding these would cut each
// corner off (measured at 17% of stroke width at a real pen tip, which is what
// made stroke ends look chopped). Sharp vertices must be routed THROUGH.
assert.ok(closedPath.includes("L 0.00,0.00"), "sharp vertex p0 is reached, not cut");
assert.ok(closedPath.includes("L 100.00,0.00"), "sharp vertex p1 is reached, not cut");
assert.ok(closedPath.includes("L 100.00,100.00"), "sharp vertex p2 is reached, not cut");
```

```

assert.ok(closedPath.includes("L 0.00,100.00"), "sharp vertex p3 is reached, not cut");
assert.ok(closedPath.endsWith("Z"), "Closed spline must end with Z");

// ...and gentle curvature must STILL be smoothed, or strokes get faceted.
// A many-sided polygon approximates a circle: every turn is shallow.
const circle: number[][] = [];
for (let i = 0; i < 32; i++) {
  circle.push([100 + Math.cos((i / 32) * Math.PI * 2) * 50, 100 + Math.sin((i / 32) * Math.PI * 2) * 50]);
}
const circlePath = getSvgPathFromStroke(circle, true);
assert.equal(
  (circlePath.match(/ L /g) || []).length,
  0,
  "a smooth outline must use no straight segments – every shallow turn stays quadratic"
);
assert.ok((circlePath.match(/ Q /g) || []).length >= 32, "smooth outline keeps its quadratic segments");
console.log(" ✓ Path builder verified: sharp vertices preserved, shallow curvature still smoothed");

// -----
// Test 2: buildStrokePath - Single-Tap Dots & 6 Pen Types
// -----
console.log("\n[Test 2] Single-Tap Dots & 6 Pen Presets Rendering...");

// Single-tap dot with strokeWidth = 2 (radius should be max(2/2, 2) = 2)
const dotEl2: StrokeElement = {
  id: "dot1",
  type: "stroke",
  x: 50,
  y: 50,
  rotation: 0,
  opacity: 1,
  points: [[0, 0, 0.5]],
  color: "#000000",
  strokeWidth: 2,
  strokeStyle: "pen",
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
const dotPath2 = buildStrokePath(dotEl2);
assert.equal(dotPath2, "M -2.00,0.00 a 2.00,2.00 0 1,0 4.00,0 a 2.00,2.00 0 1,0 -4.00,0 Z", "Dot radius must be 2 for width 2");

// Single-tap dot with strokeWidth = 10 (radius should be max(10/2, 2) = 5)
const dotEl10: StrokeElement = {
  ...dotEl2,
  strokeWidth: 10,
};
const dotPath10 = buildStrokePath(dotEl10);
assert.equal(dotPath10, "M -5.00,0.00 a 5.00,5.00 0 1,0 10.00,0 a 5.00,5.00 0 1,0 -10.00,0 Z", "Dot radius must be 5 for width 10");

// Micro-jitter tap (2 points within 2px)
const jitterDotEl: StrokeElement = {
  ...dotEl2,
  points: [[0, 0, 0.5], [1.2, 0.8, 0.5]],
  strokeWidth: 4,
};
const jitterDotPath = buildStrokePath(jitterDotEl);
assert.ok(jitterDotPath.includes("a 2.00,2.00 0 1,0 4.00,0"), "Micro-jitter tap must render solid circle");

// Verify all 6 pen styles produce valid non-empty paths for single dots and multi-point strokes
const styles = ["pen", "highlighter", "marker", "brush", "calligraphy", "pencil"] as const;
for (const st of styles) {
  // Dot test
  const testDot: StrokeElement = { ...dotEl2, strokeStyle: st, strokeWidth: 6 };
  const dPath = buildStrokePath(testDot);
  assert.ok(dPath.length > 0 && dPath.includes("a 3.00,3.00"), `${st} dot must produce circle path`);

  // Stroke test
  const testStroke: StrokeElement = {
    ...dotEl2,
    strokeStyle: st,
    strokeWidth: 4,
    points: [
      [0, 0, 0.5],
      [20, 10, 0.6],
      [50, 30, 0.7],
      [80, 20, 0.5],
    ],
  };

```

```
    [120, 40, 0.4],
  ],
};
const sPath = buildStrokePath(testStroke);
assert.ok(sPath.length > 0 && sPath.startsWith("M ") && sPath.endsWith("Z"), `${st} stroke must produce valid closed SVG path`);
}
console.log(" ✓ All 6 pen presets (pen, highlighter, marker, brush, calligraphy, pencil) produce valid SVG stroke paths");

// -----
// Test 3: Hit-Testing for Dots and All 8 Shapes
// -----
console.log("\n[Test 3] Hit-Testing for Single Dots and 8 Vector Shapes...");

// Single-dot stroke at world (100, 100), strokeWidth = 4 (radius = 2 + tolerance 8 = 10)
const dotElement: StrokeElement = {
  id: "dot-test",
  type: "stroke",
  x: 100,
  y: 100,
  rotation: 0,
  opacity: 1,
  points: [[0, 0, 0.5]],
  color: "#ff0000",
  strokeWidth: 4,
  strokeStyle: "pen",
  createdAt: Date.now(),
  updatedAt: Date.now(),
};

assert.equal(hitTestElement(dotElement, 100, 100), true, "Direct hit on dot must return true");
assert.equal(hitTestElement(dotElement, 106, 100), true, "Point within radius (dist 6 <= 10) must return true");
assert.equal(hitTestElement(dotElement, 109, 100), true, "Point near boundary (dist 9 <= 10) must return true");
assert.equal(hitTestElement(dotElement, 120, 100), false, "Point outside radius (dist 20 > 10) must return false");
console.log(" ✓ Single-tap dot hit-testing passed (selectable, clickable, and erasable)");

// Rectangle
const rect: RectangleElement = {
  id: "rect1",
  type: "rectangle",
  x: 200,
  y: 200,
  width: 100,
  height: 60,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  fillColor: "transparent",
  fillStyle: "transparent",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
assert.equal(hitTestElement(rect, 250, 230), true, "Inside rectangle hits");
assert.equal(hitTestElement(rect, 150, 230), false, "Outside rectangle misses");

// Ellipse
const ellipse: EllipseElement = {
  id: "el1",
  type: "ellipse",
  x: 400,
  y: 200,
  width: 100,
  height: 60,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  fillColor: "transparent",
  fillStyle: "transparent",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
```

```
};
assert.equal(hitTestElement(ellipse, 450, 230), true, "Inside ellipse center hits");
assert.equal(hitTestElement(ellipse, 401, 201), false, "Corner outside ellipse misses");

// Diamond
const diamond: DiamondElement = {
  id: "dia1",
  type: "diamond",
  x: 600,
  y: 200,
  width: 80,
  height: 80,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  fillColor: "transparent",
  fillStyle: "transparent",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
assert.equal(hitTestElement(diamond, 640, 240), true, "Diamond center hits");
assert.equal(hitTestElement(diamond, 605, 205), false, "Diamond corner outside misses");

// Triangle, Star, Cloud, Line, Arrow
const line: LineElement = {
  id: "l1",
  type: "line",
  x: 0,
  y: 0,
  x2: 100,
  y2: 100,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  strokeWidth: 4,
  dashStyle: "solid",
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
assert.equal(hitTestElement(line, 50, 50), true, "Line midpoint hits");
assert.equal(hitTestElement(line, 50, 80), false, "Far from line misses");

const arrow: ArrowElement = {
  id: "a1",
  type: "arrow",
  x: 0,
  y: 0,
  x2: 100,
  y2: 0,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  strokeWidth: 4,
  dashStyle: "solid",
  startArrowhead: "none",
  endArrowhead: "arrow",
  arrowheadSize: 12,
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
assert.equal(hitTestElement(arrow, 50, 2), true, "Arrow shaft hits");
assert.equal(hitTestElement(arrow, 50, 30), false, "Far from arrow misses");
console.log(" ✓ All 8 shape tools hit-testing verified");

// -----
// Test 4: Smart Connectors & 3 Routing Modes
// -----
console.log("\n[Test 4] Smart Connectors Docking & Routing Modes...");

const nodeA: RectangleElement = { ...rect, id: "nodeA", x: 100, y: 100, width: 100, height: 50 };
const nodeB: RectangleElement = { ...rect, id: "nodeB", x: 400, y: 100, width: 100, height: 50 };

const portsA = getNodeAnchorPorts(nodeA);
assert.equal(portsA.top.x, 150);
assert.equal(portsA.top.y, 100);
assert.equal(portsA.right.x, 200);
```

```
assert.equal(portsA.right.y, 125);
assert.equal(portsA.bottom.x, 150);
assert.equal(portsA.bottom.y, 150);
assert.equal(portsA.left.x, 100);
assert.equal(portsA.left.y, 125);

// 3 Routing Modes
const curved = getCurvedBezierPath(portsA.right, portsA.left);
assert.ok(curved.startsWith("M 200,125 C"), "Curved route must be cubic Bezier");

const ortho = getOrthogonalPath(portsA.right, portsA.left);
assert.ok(ortho.startsWith("M 200,125 L"), "Orthogonal route must be stepped line");

const straight = getStraightPath(portsA.right, portsA.left);
assert.equal(straight, "M 200,125 L 100,125", "Straight route must be direct M...L");

// Arrowheads
const arrowHead = getArrowheadPath(100, 125, 120, 125, "arrow", 12);
assert.ok(arrowHead !== null && arrowHead.path.startsWith("M 100,125 L"), "Arrowhead path generated");

const stealthHead = getArrowheadPath(100, 125, 120, 125, "stealth", 12);
assert.ok(stealthHead !== null && stealthHead.path.startsWith("M 100,125 L"), "Stealth arrowhead generated");

const circleHead = getArrowheadPath(100, 125, 120, 125, "circle", 12);
assert.ok(circleHead !== null && circleHead.path.includes("a "), "Circle arrowhead generated");
console.log(" ✓ Smart Connectors docking and 3 routing modes (curved, orthogonal, straight) verified");

// -----
// Test 5: Zoom Geometry & Viewport Offset Calculations
// -----
console.log("\n[Test 5] Zoom Geometry & Container Viewport Calculations...");

// Container Viewport: width=1280, height=800
const containerVp = { width: 1280, height: 800 };
// Window Viewport (e.g. sidebar width 256px -> 1536):
const windowVp = { width: 1536, height: 800 };

const cameraCenter = { x: 0, y: 0, zoom: 1.0 };
// Mouse at center of canvas container: centerX = 640, centerY = 400
const screenPt = { x: 640, y: 400 };

// With container viewport:
const worldPtCorrect = screenToWorld(screenPt.x, screenPt.y, cameraCenter, containerVp);
assert.equal(worldPtCorrect.x, 0, "Container center must map to world center (0,0)");
assert.equal(worldPtCorrect.y, 0, "Container center must map to world center (0,0)");

// When zooming to 2.0 with container viewport:
const newZoom = 2.0;
const newCameraXCorrect = worldPtCorrect.x - (screenPt.x - containerVp.width / 2) / newZoom;
const newCameraYCorrect = worldPtCorrect.y - (screenPt.y - containerVp.height / 2) / newZoom;
assert.equal(newCameraXCorrect, 0, "Camera position must remain exactly 0 with container viewport");
assert.equal(newCameraYCorrect, 0, "Camera position must remain exactly 0 with container viewport");

// If window.innerWidth had been used instead:
const worldPtBuggy = (screenPt.x - windowVp.width / 2) / cameraCenter.zoom + cameraCenter.x;
const newCameraXBuggy = worldPtBuggy - (screenPt.x - windowVp.width / 2) / newZoom;
assert.equal(worldPtBuggy, -128, "Buggy window width shifted focus by -128px");
assert.equal(newCameraXBuggy, -64, "Buggy window width shifted camera by -64px");
console.log(" ✓ Sidebar offset fix verified: Passing container viewport eliminates the 128px zoom shift");

// -----
// Test 6: Alignment and Distribution
// -----
console.log("\n[Test 6] Element Alignment & Distribution...");

const el1: RectangleElement = { ...rect, id: "e1", x: 10, y: 10, width: 50, height: 50 };
const el2: RectangleElement = { ...rect, id: "e2", x: 100, y: 80, width: 60, height: 40 };
const el3: RectangleElement = { ...rect, id: "e3", x: 250, y: 40, width: 40, height: 70 };

// Align Left
const alignedLeft = alignElements([el1, el2, el3], "left");
assert.equal(alignedLeft[0].x, 10);
assert.equal(alignedLeft[1].x, 10);
assert.equal(alignedLeft[2].x, 10);

// Align Top
const alignedTop = alignElements([el1, el2, el3], "top");
assert.equal(alignedTop[0].y, 10);
```

```
assert.equal(alignedTop[1].y, 10);
assert.equal(alignedTop[2].y, 10);

// Distribute Horizontally
const distributedH = distributeElements([el1, el2, el3], "horizontal");
assert.equal(distributedH[0].x, 10);
assert.equal(distributedH[2].x, 250);
// Even spacing between (10, 60) -> (125, 185) -> (250, 290)
assert.equal(distributedH[1].x, 125);

console.log(" ✓ Alignment and distribution functions verified");

// -----
// Test 7: Panning Gesture Sign & Camera Translation Math
// -----
console.log("\n[Test 7] Panning Direction & Multi-Touch Drag Calculations...");

const testCam = { x: 100, y: 100, zoom: 1.5 };
const testVp = { width: 1000, height: 800 };
const testElementPos = { x: 100, y: 100 }; // initially renders exactly at center (500, 400)

const screenInitial = worldToScreen(testElementPos.x, testElementPos.y, testCam, testVp);
assert.equal(screenInitial.x, 500);
assert.equal(screenInitial.y, 400);

// User drags pointer to the right by +60px and down by +45px:
const dragDeltaX = 60;
const dragDeltaY = 45;

// Correct camera update:
const pannedCam = {
  ...testCam,
  x: testCam.x - dragDeltaX / testCam.zoom,
  y: testCam.y - dragDeltaY / testCam.zoom,
};

const screenPanned = worldToScreen(testElementPos.x, testElementPos.y, pannedCam, testVp);
assert.equal(Math.round(screenPanned.x), 500 + dragDeltaX, "Element must move right by exactly dragDeltaX on screen");
assert.equal(Math.round(screenPanned.y), 400 + dragDeltaY, "Element must move down by exactly dragDeltaY on screen");
console.log(" ✓ Pan direction verified: Canvas elements track dragging pointer direction with 1:1 screen displacement");

console.log("\n=====");
console.log("ALL MILESTONE 1 AUTOMATED TESTS PASSED SUCCESSFULLY! (100%)");
console.log("=====");

=====
// === END FILE: tests/milestone1.test.ts ===
=====

=====
// === START FILE: tests/milestone2_tools_qa.test.ts ===
// File 162 of 194 | Lines: 429 | Size: 13772 bytes
=====

import assert from "node:assert/strict";
import {
  hitTestElement,
  getElementBounds,
  getElementCenter,
  getNodeAnchorPorts,
  getResolvedAnchorPoint,
  getOptimalConnectionAnchors,
  getConnectionPath,
  isPointInPolygon,
  isElementInPolygon,
  worldToScreen,
  screenToWorld,
} from "../src/lib/whiteboard/geometry";
import type {
  StrokeElement,
  RectangleElement,
  EllipseElement,
  DiamondElement,
```

```

    LineElement,
    ArrowElement,
    ConnectionElement,
    TextElement,
    StickyElement,
    ImageElement,
    Board,
  } from "../src/lib/whiteboard/types";

console.log("=====");
console.log("=== RUNNING MILESTONE 2 (R2) QA & TOOL BUG FIXING TEST SUITE ===");
console.log("=====");

let passCount = 0;
function pass(msg: string) {
  passCount++;
  console.log(`  ✓ ${msg}`);
}

// -----
// 1. True Lasso Freeform Polygon Selection & Ray-Casting Point-in-Polygon Tests
// -----
console.log(`\n[Test 1] Lasso Ray-Casting & Element Containment Tests...`);

// 1.1 Triangle Polygon
const trianglePoly = [
  { x: 0, y: 0 },
  { x: 100, y: 0 },
  { x: 50, y: 100 },
];
assert.equal(isPointInPolygon(50, 30, trianglePoly), true, "Interior point in triangle");
assert.equal(isPointInPolygon(50, 150, trianglePoly), false, "Exterior point below triangle");
assert.equal(isPointInPolygon(-10, 50, trianglePoly), false, "Exterior point left of triangle");
pass("Triangle polygon ray casting point-in-polygon containment verified");

// 1.2 Concave / L-shaped Polygon
const lPoly = [
  { x: 0, y: 0 },
  { x: 100, y: 0 },
  { x: 100, y: 50 },
  { x: 50, y: 50 },
  { x: 50, y: 100 },
  { x: 0, y: 100 },
];
assert.equal(isPointInPolygon(20, 20, lPoly), true, "Inside top-left arm of L");
assert.equal(isPointInPolygon(20, 80, lPoly), true, "Inside bottom-left foot of L");
assert.equal(isPointInPolygon(75, 75, lPoly), false, "In the cut-out corner of L (must be outside)");
pass("Concave polygon ray casting handles non-convex geometries");

// 1.3 Element Containment in Polygon
const rectInside: RectangleElement = {
  id: "rect-in",
  type: "rectangle",
  x: 20,
  y: 20,
  width: 40,
  height: 40,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  fillColor: "#fff",
  fillStyle: "solid",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: Date.now(),
  updatedAt: Date.now(),
};

const rectOutside: RectangleElement = {
  ...rectInside,
  id: "rect-out",
  x: 200,
  y: 200,
};

const lassoLoop = [
  { x: 0, y: 0 },
  { x: 120, y: 0 },

```

```
{ x: 120, y: 120 },
{ x: 0, y: 120 },
];

assert.equal(isElementInPolygon(rectInside, lassoLoop), true, "Element inside lasso polygon returns true");
assert.equal(isElementInPolygon(rectOutside, lassoLoop), false, "Element outside lasso polygon returns false")
;

// 1.4 Freehand Stroke containment in lasso
const strokeInLasso: StrokeElement = {
  id: "stroke-in",
  type: "stroke",
  x: 30,
  y: 30,
  rotation: 0,
  opacity: 1,
  points: [[0, 0, 0.5], [20, 20, 0.5]],
  color: "#000",
  strokeWidth: 2,
  strokeStyle: "pen",
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
assert.equal(isElementInPolygon(strokeInLasso, lassoLoop), true, "Stroke inside lasso returns true");
pass("Element and stroke containment in freeform lasso polygons verified");

// -----
// 2. Line & Arrow Resizing Endpoint Vector Updates
// -----
console.log("\n[Test 2] Line & Arrow Endpoint (x2, y2) Vector Resizing...");

const testLine: LineElement = {
  id: "line-1",
  type: "line",
  x: 100,
  y: 100,
  x2: 150,
  y2: 80,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: Date.now(),
  updatedAt: Date.now(),
};

const lineBounds = getElementBounds(testLine);
assert.equal(lineBounds.x, 100);
assert.equal(lineBounds.y, 100);
assert.equal(lineBounds.width, 150);
assert.equal(lineBounds.height, 80);

// Simulate handle 'e' resize (+50px dx)
const dx = 50;
const dy = 0;
const p1 = { x: testLine.x, y: testLine.y };
const p2 = { x: testLine.x + testLine.x2, y: testLine.y + testLine.y2 };
const oldW = Math.max(lineBounds.width, 1);
const oldH = Math.max(lineBounds.height, 1);

const newX = lineBounds.x;
const newY = lineBounds.y;
const newW = lineBounds.width + dx;
const newH = lineBounds.height;

const u1 = (p1.x - lineBounds.x) / oldW;
const v1 = (p1.y - lineBounds.y) / oldH;
const u2 = (p2.x - lineBounds.x) / oldW;
const v2 = (p2.y - lineBounds.y) / oldH;

const newP1x = newX + u1 * newW;
const newP1y = newY + v1 * newH;
const newP2x = newX + u2 * newW;
const newP2y = newY + v2 * newH;

const updatedLine: LineElement = {
  ...testLine,
```

```
x: newP1x,
y: newP1y,
x2: newP2x - newP1x,
y2: newP2y - newP1y,
};

assert.equal(updatedLine.x, 100);
assert.equal(updatedLine.y, 100);
assert.equal(updatedLine.x2, 200);
assert.equal(updatedLine.y2, 80);
pass("Line and Arrow endpoint (x2, y2) vector updates accurately during resize");

// -----
// 3. Rotated Element Resizing in Local Coordinate Space
// -----
console.log("\n[Test 3] Rotated Element Resizing along Local Axes...");

const rotRect: RectangleElement = {
  ...rectInside,
  id: "rot-rect",
  x: 100,
  y: 100,
  width: 100,
  height: 60,
  rotation: Math.PI / 2, // 90 deg clockwise
};

const rot = rotRect.rotation;
const cos = Math.cos(rot);
const sin = Math.sin(rot);

// Dragging handle 'e' in world space (+40px in Y direction corresponds to +40px in local X direction)
const worldDx = 0;
const worldDy = 40;
const localDx = worldDx * cos + worldDy * sin; // 0 * 0 + 40 * 1 = +40
const localDy = -worldDx * sin + worldDy * cos; // 0 + 0 = 0

assert.equal(Math.round(localDx), 40, "World Y displacement maps to local X displacement for 90-deg rotation");
;
assert.equal(Math.round(localDy), 0, "World X displacement maps to local Y displacement");

const origW = rotRect.width;
const origH = rotRect.height;
const origCx = rotRect.x + origW / 2;
const origCy = rotRect.y + origH / 2;

const newWidth = Math.max(origW + localDx, 20); // 140
const localShiftX = (newWidth - origW) / 2; // +20
const localShiftY = 0;

const worldShiftX = localShiftX * cos - localShiftY * sin; // 20 * 0 - 0 = 0
const worldShiftY = localShiftX * sin + localShiftY * cos; // 20 * 1 + 0 = 20

const newRotCx = origCx + worldShiftX;
const newRotCy = origCy + worldShiftY;

assert.equal(newWidth, 140, "Local width expanded along rotated X-axis");
assert.equal(Math.round(newRotCy), Math.round(origCy + 20), "Center translated along rotated orientation");
pass("Rotated element local coordinate space resize calculation verified");

// -----
// 4. Eraser Size Tolerance Hit-Testing
// -----
console.log("\n[Test 4] Continuous Eraser Size Tolerance Hit-Testing...");

const dotForEraser: StrokeElement = {
  id: "dot-eraser",
  type: "stroke",
  x: 50,
  y: 50,
  rotation: 0,
  opacity: 1,
  points: [[0, 0, 0.5]],
  color: "#000",
  strokeWidth: 2,
  strokeStyle: "pen",
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
```